

Struct_Sept handbook

Lattice structures with a neural network as geometry:
how the code is organised and how the work flows

This handbook is for someone joining the project. It explains what the code does, why it is organised the way it is, and how a day of work actually runs: build a dataset, train a decoder, look at what it learned, and push a geometry through to a finite element model.

If you are new, start with Chapter 1. Chapters 2–4 are background (method, repository, library), Chapters 5–8 follow the code (data, training, GUI, FEM), Chapter 9 has the recipes and Chapter 10 the state of the research.

Open `board.html` (same folder) in a browser for the visual version: a Miro-style board with boxes and arrows, a guided tour, and links that open VS Code at the right line.

Written	2 October 2026
Code state	branch <code>gui-train-tab</code> at commit <code>6c07cfc</code> , plus the uncommitted work on disk (triangle family, 166-shape hole dataset); submodule <code>DeepSDFStruct</code> at <code>bbe9881</code>
Companion	<code>docs/handbook/board.html</code> — the interactive board (open it in a browser)
Reference paper	Kofler, Giritsch, Elgeti, <i>Structural optimization of lattice structures using deep neural networks as geometry representation</i> , Graphical Models 142 (2025) 101307

Contents

1	The project at a glance	4
1.1	What the project is	4
1.2	The pipeline in six steps	4
1.3	Where this repo stands	4
1.4	The current research question	6
1.5	The code bases at a glance	6
1.6	How to read this handbook	6
2	The method, concept by concept	8
2.1	Signed distance functions	8
2.2	DeepSDF: the unit cell as an auto-decoder	8
2.3	The transformation function $T(\mathbf{x})$	9
2.4	The latent field $\lambda(\mathbf{x})$: the design variables	10
2.5	Closing the borders: <code>CappedBorderSDF</code>	10
2.6	FlexiCubes: from the SDF to a surface mesh	11
2.7	Freeform deformation: from the parametric cube to metres	11
2.8	Tetrahedralization: the open trade-off	11
2.9	FEM: linear elasticity, stiffness, compliance	12
2.10	MMA: updating the design	12
2.11	The gradient chain	13
3	The repository	14
3.1	The map	14
3.2	The one import rule	14
3.3	Where does my new file go?	15
3.4	Packages and installs	16
3.5	Where output lands	16
3.6	Naming conventions	17
3.7	The <code>docs/</code> folder	17
3.8	Git state	18
3.9	Docs that have drifted	18
4	The DeepSDFStruct library	19
4.1	Module map	19
4.2	The geometry side	19
4.3	The learning side	23
4.4	The optimisation side: <code>MMA</code>	24
4.5	The template: <code>test_structural_optimization.py</code>	24
4.6	Library-level traps	25
5	Training data: the <code>datagen</code> package	26
5.1	Why a separate package	26
5.2	Parameters, field, glue	26
5.3	The hole family	27
5.4	The exact signed distance	28
5.5	Where the samples go	29
5.6	The triangle family	29
5.7	The on-disk contract	30
5.8	The datasets on disk	31
5.9	Gotchas	32

6	Training a decoder	33
6.1	What a run is	33
6.2	The hyperparameter schema	33
6.3	The supervisor's sheet	34
6.4	What pressing <i>Train</i> does	35
6.5	Anatomy of a run folder	36
6.6	Presets	37
6.7	Unattended training	37
6.8	Analysing a run	39
6.9	Trainer traps	39
7	The GUI	41
7.1	Two windows and how to start them	41
7.2	Architecture: one dict, one queue, no classes	41
7.3	Explore: one optimizer iteration by hand	43
7.4	Explore 2-D	45
7.5	How a run reaches the explorers	45
7.6	The SDF maker	46
7.7	The desktop launcher	46
7.8	Known limits and tests	47
8	From geometry to stiffness	48
8.1	Three coordinate spaces	48
8.2	The test geometry: <code>plate_with_hole()</code>	49
8.3	<code>fem.py</code> : from an SDF to K	50
8.4	The experiment ladder	51
8.5	Point cloud in, lattice out	52
8.6	The theory document	53
8.7	What is missing before an optimization loop	54
9	Recipes	55
9.1	Set up a fresh clone	55
9.2	The daily research loop	56
9.3	Add a new parametric family	56
9.4	Train overnight without the GUI	57
9.5	Compute the stiffness matrix of a geometry	58
9.6	Start a real optimization run	58
9.7	Testing and conventions	59
10	Where the research stands	61
10.1	The research question	61
10.2	Timeline	61
10.3	Families, datasets and runs	62
10.4	Rules of thumb learned so far	62
10.5	Open threads and next steps	63
10.6	Broken or incomplete	64
10.7	Open questions for the supervisor	64
10.8	Upstream bug candidates	65
A	Glossary and notation	66
B	On-disk formats	68
C	Command cheat sheet	71
D	Gotchas checklist	73

Chapter 1

The project at a glance

1.1 What the project is

Struct_Sept is a HiWi project at the Institute of Lightweight Design and Structural Biomechanics, TU Wien. It builds on one paper: M. Kofler, M. Giritsch, S. Elgeti, *Structural optimization of lattice structures using deep neural networks as geometry representation*, Graphical Models 142 (2025) 101307, doi:10.1016/j.gmod.2025.101307. The method of the paper is implemented in the library **DeepSDFStruct**, written by the paper's first author and included in this repo as a git submodule. Paper notation and code names line up closely, so reading one helps with the other.

The problem. A lattice structure fills a part with many small, repeated unit cells. Grading the cells (thick struts where the part carries load, thin ones elsewhere) gives stiff, light parts that additive manufacturing can build. The classical ways to optimize such a part either *homogenize* it, which assumes the cells are much smaller than the part and periodic, or reduce every cell to beams or shells. Both limit which geometries can appear.

The method. The paper changes how the geometry itself is described:

- **The unit cell is a neural network.** A DeepSDF decoder $f_\theta(\lambda, \mathbf{x})$ returns the signed distance from a point $\mathbf{x}$ to the cell surface: negative inside, positive outside, zero on the surface. A short latent vector λ says *which* cell. The decoder is trained on a family of cells, so the design space is continuous, low-dimensional, and limited to shapes like the training shapes.
- **The latent vector becomes a field.** Instead of one λ for the whole part, λ varies over space as a B-spline, $\lambda(\mathbf{x}) = \sum_i \varphi_i(\mathbf{x}) \hat{\lambda}_i$. The spline control points $\hat{\lambda}_i$ are the **design variables**. A transformation function $T(\mathbf{x})$ tiles the unit cell over the domain so that the geometry stays continuous from cell to cell.
- **Full-scale FEM in every iteration.** The whole lattice is meshed (FlexiCubes, then a freeform deformation, then tetrahedra) and linear elasticity is solved on that mesh. There is no homogenization and no periodicity assumption.
- **MMA updates the design.** The Method of Moving Asymptotes moves $\hat{\lambda}$ to minimize the compliance J under a volume constraint $V \leq V_{\text{target}}$.
- **Everything is differentiable.** The gradient $\partial J / \partial \hat{\lambda}$ is carried back through the whole chain $\hat{\lambda} \rightarrow \lambda \rightarrow f_\theta \rightarrow \Omega \rightarrow \bar{\Omega} \rightarrow J$, mesh extraction included (in the paper an adjoint sensitivity analysis gives the FEM link, automatic differentiation all the others). This is why every stage is written in PyTorch, and why code inside the loop must never call `.detach()` or `.item()` or take a detour through NumPy.

Chapter 2 explains each of these concepts and ties it to the code.

1.2 The pipeline in six steps

The paper splits the method into an *offline* part, done once per family of unit cells, and an *online* part, repeated in every optimization iteration (Figure 1). Steps 1 and 2 produce a trained decoder. Steps 3 to 6 use it as a geometry generator: evaluate the latent field, mesh the SDF, simulate, update the design variables.

The cost sits in step 5. In the paper's cantilever test case (about 200 000 tetrahedra) one iteration takes about 1 min 50 s, of which about 1 min 30 s is FEM plus sensitivity analysis. When you work on something unrelated to the physics, reduce the mesh resolution and the tiling first.

1.3 Where this repo stands

The library covers all six steps. This repo uses them unevenly:

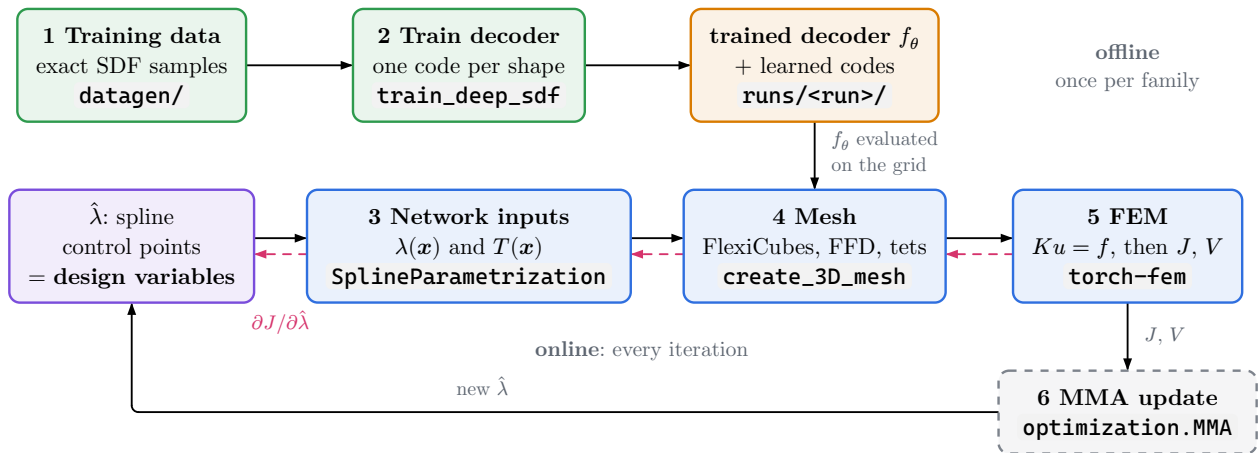


Figure 1: The method of the paper. Green: the main activity of this repo. Blue: library code that this repo calls or partly re-implements. Grey dashed: only in the library. Black arrows are the forward pass; the pink dashed arrows are the gradient $\partial J / \partial \hat{\lambda}$, which is carried back through steps 5, 4 and 3 to the control points, where MMA reads it.

Paper step	Where it lives	State in this repo
1 Training data	datagen/ (exact 2-D SDFs of plate families); the SDF maker window samples meshes with the library's SDFSampler (structsept/app/sdf_maker.py , datasets.py)	Main activity. Seven datasets, all 2-D plates, on the author's machine: data/ is gitignored, so a clone rebuilds them (Chapter 9)
2 Train decoder	library train_deep_sdf , wrapped by structsept.app.training.train (which fixes the seed order); called by the GUI Train tab and experiments/train_plate_*.py	Main activity. 13 run folders in runs/ (gitignored too; smoke tests and unfinished runs included) plus 6 untrained presets
3 Network inputs	library SplineParametrization , LatticeSDFStruct ; assembled by build_lattice in structsept/app/models.py	Moved by hand in the GUI Explore tab. It lists only 3-D decoders, so in practice the library's pretrained ones
4 Mesh	library create_3D_mesh , tetrahedralize_surface ; structsept/fem.py	Works: surfaces in the Explore tab, tetrahedra in experiment scripts; never inside a loop
5 FEM	torch-fem , driven by structsept/fem.py	Stops at the global stiffness matrix K. No supports, no loads, no solve
6 MMA update	library optimization.MMA	Not called anywhere in this repo. Only in the library tests; the full loop is test_structural_optimization.py

Two points in this table matter for a newcomer.

The decoders trained here are 2-D shapes, not 3-D unit cells. The current work is steps 1 and 2 on planar plate families: a plate with a circular hole and a plate with four triangular holes. These decoders take 2-D points (**geom_dimension 2** in **specs.json**, the recipe file of a training run, Chapter 6), and each one describes a *whole plate*, not a cell to be tiled. They cannot be plugged into **LatticeSDFStruct** and the MMA loop as they are. The GUI views them in a separate Explore 2-D tab, and the docstring of **structsept/app/models.py:1** says that they skip the lattice entirely.

There is no optimization loop in this repo yet. **structsept/fem.py** stops at K on purpose: **check_stiffness** verifies symmetry, a positive diagonal and the six rigid-body modes in the kernel, but nothing applies supports or loads or solves $Ku = f$. The only complete loop is the library test **DeepSDFStruct/tests/test_structural_optimization.py:21**, which is also the template for a first optimization run (Chapter 4, Chapter 9).

In progress Most of the newest work is on branch **gui-train-tab** (not merged into **main**) or not committed at all; Chapter 3 (Git state) lists it. Run **git status** before you build on any of it.

1.4 The current research question

The paper treats the latent space as a design space: MMA moves $\hat{\lambda}$ inside box bounds, and the library test uses $[0.15, 0.75]$ for every control point. But a DeepSDF decoder is an *auto-decoder*. It has no encoder and never sees the parameters that generated its training shapes. Each shape's code is a free variable, optimized together with the network weights, so the network invents its own coordinates. Before any optimization, this repo therefore asks: **does the latent space that an auto-decoder learns on its own recover the parameters that generated the shapes, and how many latent dimensions does it need for that?** If it does, box bounds on the latent axes have a physical meaning, and an optimized latent field can be read back as geometry.

The test bed is 2-D plates with exact signed distance functions, chosen because both their exact SDF and their generating parameters are known, which makes the comparison clean; how such whole-plate decoders lead back to the paper's 3-D unit cells is still open (Chapter 10). The two families are a hole with parameters (x_c, y_c, r) and four triangular holes with height h and base width w . After training, the learned codes are compared with the dataset's parameter table `params.csv` (row i belongs to code i). The answer so far: when the latent dimension d equals the number of parameters, they come back up to an arbitrary affine change of basis (fully with one or two parameters, partly with three); with fewer dimensions than parameters the network keeps what dominates the loss (the hole position) and drops the rest. Chapter 10 has the numbers, the rules of thumb and the open threads.

1.5 The code bases at a glance

Folder	What it holds	You go there to
<code>DeepSDFStruct/</code>	Git submodule, the upstream library: SDF classes, lattice, spline parametrization, FlexiCubes meshing, FFD, the DeepSDF trainer, MMA, pretrained decoders	use any step of the method; edit only for a change to the library itself
<code>structsept/</code>	This project's library: <code>plate_with_hole.py</code> (test geometry), <code>fem.py</code> (mesh to K), <code>pointcloud_sdf.py</code> (SDF of a point cloud)	reuse geometry, FEM or point-cloud code
<code>structsept/app/</code>	The Tkinter GUI (explorer window <code>main.py</code> with Explore, Explore 2-D and Train tabs; the separate <code>sdf_maker.py</code>) and the headless helpers <code>training.py</code> , <code>unattended.py</code>	train and inspect decoders, by hand or overnight
<code>datagen/</code>	Generators of the 2-D plate datasets with exact SDFs; writes to <code>data/</code>	build or extend a training set
<code>experiments/</code>	Runnable one-off scripts (never imported) and <code>IDEIAS.md</code> , the lab notebook in Portuguese	run an experiment; read why things are the way they are
<code>tests/</code>	pytest suite; <code>test_deepsdfstruct_env.py</code> is the offline smoke test	check the environment and your changes
<code>docs/</code>	This handbook, <code>paper_context.md</code> , <code>structure.md</code> , <code>gui.md</code> , <code>stiffness_theory/</code>	read the background
<code>data/</code> , <code>runs/</code> , <code>outputs/</code>	Generated datasets, training runs, logs; all gitignored	find results (they live only on disk)

One rule keeps this legible: `experiments/` imports from `structsept/`, which imports from `DeepSDFStruct`, never the other way round. `datagen/` imports nothing from the others; it talks to the rest only through the files it writes into `data/`. Because results in `data/`, `runs/` and `outputs/` are not versioned, findings are written down in `experiments/IDEIAS.md`. Chapter 3 has the annotated tree, the editable installs and where a new file goes.

1.6 How to read this handbook

The appendix is for lookup: glossary (paper symbol to code name), file formats, commands, gotchas. You do not need to read in order; pick a path:

Your goal	Suggested path
Understand the method	Ch. 2, Ch. 4, Ch. 8; then <code>docs/paper_context.md</code> and the paper itself (doi:10.1016/j.gmod.2025.101307; the PDF is not in git because the repo is public)
Train a decoder on a new family	Ch. 5, Ch. 6, the recipes in Ch. 9, then Ch. 10 for what earlier runs taught; formats in Appendix B
Run an optimization	Ch. 2, the template walkthrough in Ch. 4, what is missing in Ch. 8, the recipe in Ch. 9
Work on the GUI	Ch. 3, Ch. 7, the Train tab internals in Ch. 6, testing conventions in Ch. 9
Get going on day one	the setup recipe in Ch. 9, Appendix C and Appendix D

The companion board. `docs/handbook/board.html` shows the same material as a zoomable map, like a Miro board. It is a single file: open it in a browser, no server needed. It is organized in numbered frames, read in order, with cards inside them and arrows between the cards. Click a card for its explanation, the files behind it and the cards it connects to; when the board is opened from its place in the repo, each file has a button that opens VS Code at the line. A **Tour** button walks through it step by step.

Conventions. A file reference such as `structsept/fem.py:197` is relative to the repo root and points at the definition; library files start with `DeepSDFStruct/DeepSDFStruct/`. Boxes mark four kinds of side notes: *In the code* ties theory to a file, *Watch out* is a trap that has bitten someone, *Tip* is a shortcut, and *In progress* is work that is unfinished or not committed; in Chapter 4 most sections open with a small *Code:* line naming the class and file instead of an In-the-code box. `CLAUDE.md` in the repo root is the one-page briefing that the AI coding assistant (Claude Code) loads in every session; several chapters quote it, and correct it where it has fallen behind (Chapter 3). Every Python command runs through `uv`, never bare `python`. To see the project working before you read on:

```
# offline smoke test of the environment (downloads nothing)
uv run pytest tests/test_deepsdfstruct_env.py -v
# the explorer window: Explore, Explore 2-D and Train tabs
uv run python -m structsept.app.main
```

Chapter 2

The method, concept by concept

The paper turns a vector of design variables into a compliance value through one long chain of operations, and every link of the chain is differentiable. This chapter takes the links one at a time, in the order the code runs them: the idea, the little math you need, why it is done this way, and an *In the code* box with the class and the file. The API around these classes is in Chapter 4, this project's FEM code in Chapter 8, training in Chapter 6, paper symbols next to code names in Appendix A. Keep three coordinate spaces apart while you read: the decoder's cube $[-1, 1]^3$ holding *one* unit cell, the parametric cube $[0, 1]^3$ holding the *whole* lattice, and the physical domain $\bar{\Omega}$ in metres.

2.1 Signed distance functions

A solid Ω with surface $\Gamma = \partial\Omega$ is described by its signed distance function (SDF)

$$\varphi(\mathbf{x}) = \begin{cases} -\min_{\mathbf{y} \in \Gamma} \|\mathbf{x} - \mathbf{y}\| & \text{if } \mathbf{x} \in \Omega \\ +\min_{\mathbf{y} \in \Gamma} \|\mathbf{x} - \mathbf{y}\| & \text{otherwise.} \end{cases} \quad (1)$$

Negative means inside, i.e. material; positive means void; zero is the surface. An exact SDF has $\|\nabla\varphi\| = 1$ almost everywhere. Unlike a mesh, a function can be evaluated anywhere, combined with other functions and differentiated: if φ depends on parameters, its zero level set moves continuously with them. Shapes combine with `min` and `max`:

$$\varphi_{A \cup B} = \min(\varphi_A, \varphi_B), \quad \varphi_{A \cap B} = \max(\varphi_A, \varphi_B), \quad \varphi_{A \setminus B} = \max(\varphi_A, -\varphi_B). \quad (2)$$

The sign is right everywhere and the zero level set is exact, but away from the surface $|\varphi|$ is in general only a *lower bound* of the true distance (inside the overlap of two shapes, or near a concave corner, the true distance is larger). The mesher needs only the sign and a roughly right magnitude near the surface, so this is enough. `min` and `max` are differentiable almost everywhere; the gradient goes to the active argument.

In the code Every SDF is a `torch.nn.Module` subclassing `SDFBase` (`DeepSDFStruct/DeepSDFStruct/SDF.py:194`): `sdf(points)` maps $(N, 3)$ points to $(N, 1)$ distances, and primitive sizes are `nn.Parameters`, so gradients reach them. `UnionSDF` (line 575) is the `min` above, `DifferenceSDF` (line 660) the `max` with the negated subtrahend. API and traps: Chapter 4.

2.2 DeepSDF: the unit cell as an auto-decoder

A DeepSDF decoder is a multilayer perceptron $f_\theta(\lambda, \mathbf{x})$: it takes a latent code $\lambda \in \mathbb{R}^d$ concatenated with a point $\mathbf{x} \in [-1, 1]^3$ and returns the signed distance to the shape that λ encodes. Trained once on a family of shapes, it lets λ move continuously through the family. That is the design space of the method: low-dimensional, continuous, and limited to shapes like the training shapes, whereas density-based topology optimization can produce anything.

The decoder is an *auto-decoder* (Park et al., 2019): **there is no encoder**. Each training shape j gets its own code λ_j , a free parameter in a table, optimized together with the weights:

$$\min_{\theta, \{\lambda_j\}} \sum_j \sum_i |c_\delta(f_\theta(\lambda_j, \mathbf{x}_{ij})) - c_\delta(\varphi_{ij})| + \lambda_{\text{reg}} \|\lambda_j\|, \quad c_\delta(s) = \max(-\delta, \min(\delta, s)). \quad (3)$$

Clamping to $\pm\delta$ spends the network's capacity near the surface, where the zero level set is decided. The norm penalty, weighted by λ_{reg} (`CodeRegularizationLambda` in the code; the paper calls it σ), keeps the codes compact around the origin. After training, row j of the table is shape j . Two consequences:

- **The codes are coordinates the network invents.** Nothing ties a latent axis to a physical parameter such as a radius. Whether the learned axes recover the generating parameters is this repo's research question (Chapter 10).
- **Reconstruction is optimization.** To represent a new shape, freeze θ and minimize the same loss over the code alone, or over the control points of a latent field. The structural optimization uses the same mechanism: gradients with respect to the latent input, weights fixed.

In the code `train_deep_sdf` (`DeepSDFStruct/DeepSDFStruct/deep_sdf/training.py:299`) minimizes this loss over the decoder and the code table. `get_model` (`DeepSDFStruct/DeepSDFStruct/pretrained_models.py:98`) loads a trained run and `SDFfromDeepSDF` (`DeepSDFStruct/DeepSDFStruct/SDF.py:1199`) wraps it as an SDF on $[-1, 1]^3$ whose only parameter is the latent: the weights stay frozen, as reconstruction and MMA need. API and traps: Chapter 4; training: Chapter 6.

Watch out `PretrainedModels.AnalyticRoundCross`, used by the library template and by `structsept/plate_with_hole.py`, is not a learned network. Its `forward` (`DeepSDFStruct/DeepSDFStruct/deep_sdf/networks/analytic_round_cross.py:90`) ignores the weights and evaluates three axis-aligned cylinders of radius $r = \lambda$ in closed form: the latent *is* the strut radius. The trained `RoundCross` has codes in $[-1, 1]$ and at the same $\lambda = 0.6$ fills 29 % of the cell instead of 54 %. Compare shapes, never λ values across decoders.

2.3 The transformation function $T(x)$

The decoder knows one cell in $[-1, 1]^3$; the lattice has $t_x \times t_y \times t_z$ cells in $[0, 1]^3$. The paper maps one onto the other with one global function per coordinate (its Eq. 18):

$$T(x) = 4 \left| \frac{t_x x}{2} - \left\lfloor \frac{t_x x}{2} + \frac{1}{2} \right\rfloor \right| - 1, \quad \varphi_{\text{lat}}(\xi) = f_{\theta}(\lambda(\xi), T(\xi_1), T(\xi_2), T(\xi_3)). \quad (4)$$

T is a triangle wave (Figure 2): it rises from -1 to 1 across one cell and falls back across the next. **Neighbouring cells are mirror images, not translated copies.** An asymmetric cell, the normal case for a network geometry, does not match a translated copy of itself on the shared face: the SDF would jump and the mesh would crack; a mirrored copy matches by construction. The lattice SDF is continuous if both T and $\lambda(\xi)$ are, and the range of T does not depend on the number of cells. (Deep Local Shapes, by Chabra et al., blends discontinuous per-cell transformations instead.)

In the code `transform` (`DeepSDFStruct/DeepSDFStruct/lattice_structure.py:262`) is Eq. 18, and `LatticeSDFStruct._compute` (line 172) is φ_{lat} : the latent field at the *raw* parametric coordinates, the cell at the transformed ones, and a positive distance to the box outside the bounds (default $[0, 1]^d$). The values are in cell units (a cell spans 2): the zero level set is exact, the magnitudes are not metric. API and traps: Chapter 4.

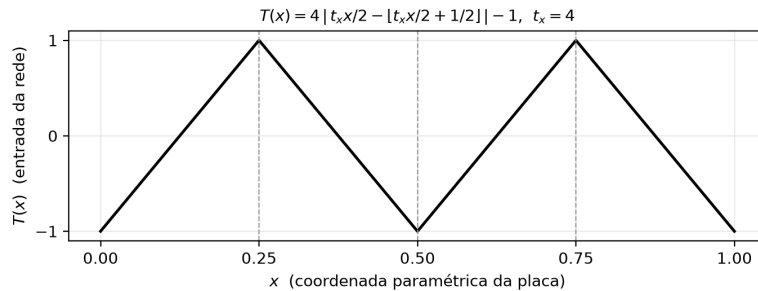


Figure 2: $T(x)$ for $t_x = 4$ (figure from `docs/stiffness_theory`, axis labels in Portuguese: *entrada da rede* = network input). Dashed lines are cell interfaces, where T mirrors instead of jumping.

2.4 The latent field $\lambda(x)$: the design variables

To grade the lattice, the latent code varies over space as a tensor-product B-spline (the paper's Eq. 21; here φ_i are basis functions, not the SDF):

$$\lambda(\xi) = \sum_i \varphi_i(\xi) \hat{\lambda}_i, \quad \hat{\lambda}_i \in \mathbb{R}^d. \quad (5)$$

The control points $\hat{\lambda}_i$ are the design variables. Four B-spline properties explain the choice:

- **Local support.** A degree- p basis function is non-zero on only $p + 1$ knot spans, so a control point changes the lattice only in its neighbourhood.
- **Partition of unity.** $\varphi_i \geq 0$ and $\sum_i \varphi_i = 1$: $\lambda(\xi)$ is a convex combination of nearby control points. Box bounds on $\hat{\lambda}$ therefore bound $\lambda(\xi)$ everywhere, and MMA's bounds keep every cell inside the decoder's trained range.
- **Independent resolution.** The number of control points does not depend on the number of cells: the template has $2 \times 1 \times 1$ cells and $5 \times 1 \times 3 = 15$ control points, the paper's cantilever 54 cells and $9 \times 2 \times 5 = 90$.
- **Continuity.** Degree $p \geq 1$ gives a continuous field, hence (with T) a continuous geometry.

In the code `SplineParametrization` (`DeepSDFStruct/DeepSDFStruct/parametrization.py:133`) wraps a `TorchSpline` (`DeepSDFStruct/DeepSDFStruct/torch_spline.py:275`), which copies the splinepy control points into an `nn.Parameter` of shape `(n_ctrl, d)` and evaluates the spline in torch. That parameter is $\hat{\lambda}$: `next(parametrization.parameters())`. `Constant` is the one-code baseline. API and traps: Chapter 4.

In progress The project's own geometry still uses `Constant([latent])`, i.e. identical cells, in `structsept/plate_with_hole.py` and the plate experiments. Switching to `SplineParametrization` is item 8 of `experiments/IDEIAS.md`, still open. Only the GUI's Explore tab and the point-cloud fit use a spline latent field today.

2.5 Closing the borders: `CappedBorderSDF`

Cutting a lattice at the domain faces leaves struts open, and an open surface has no inside: no volume, nothing for tetgen to fill, no FEM. `CappedBorderSDF` intersects the SDF with planes at the faces of the child's bounds. Each face key (`x0`, `x1`, ..., `z1`) maps to `{"cap":  $\pm 1$ , "measure":  $m$ }`; at the lower face $x = b_0$:

- **cap: -1** trims, $\varphi \leftarrow \max(\varphi, (b_0 + m) - x)$, a flat cut (at the face if $m = 0$);
- **cap: +1** adds a plate of thickness m , $\varphi \leftarrow \min(\varphi, x - (b_0 + m))$.

The default `UNIT_CUBE_CAPS_3D` trims all six faces flush. A plate (a `min`) also fills everything outside the domain, so plates are nested inside a trimming pass. This is how the template builds its clamping plate at `x0` and its loading plate at `z1`:

```
plates = {"x0": {"cap": 1, "measure": 0.05}, "z1": {"cap": 1, "measure": 0.1}}
sdf = CappedBorderSDF(CappedBorderSDF(lattice, plates)) # inner adds, outer trims
```

Measures are parametric. Because the caps are `min`/`max`, the gradient reaches $\hat{\lambda}$ only where the lattice is the active argument.

In the code `CappedBorderSDF` (`DeepSDFStruct/DeepSDFStruct/SDF.py:1450`), defaults at line 132, nesting at `DeepSDFStruct/tests/test_structural_optimization.py:54`. `LatticeSDFStruct` already returns a positive distance outside its bounds, so an uncapped lattice also closes somewhere near the face; the cap makes the closure an exact plane, and every lattice in this repo has one. API and traps: Chapter 4.

2.6 FlexiCubes: from the SDF to a surface mesh

The SDF is sampled on a regular grid; the surface crosses every grid cube whose corners differ in sign. Marching cubes puts a vertex on each sign-change edge (a, b) at $\mathbf{x}_e = (\varphi_b \mathbf{x}_a - \varphi_a \mathbf{x}_b) / (\varphi_b - \varphi_a)$ and connects them by a case table, which produces sliver triangles; the paper rejected it because slivers are bad for FEM. FlexiCubes (Shen et al., 2023) is a *dual* method: one vertex per cut cube (up to four where a cube holds several surface patches), at a weighted combination of its edge crossings, with neighbouring cubes joined by quads split into triangles. The elements are better shaped, and every vertex is a smooth function of the φ values at its cube's corners, hence of $\hat{\lambda}$. Only grid values at cut cubes receive a gradient, and the topology (which cubes are cut, how many vertices exist) is discrete: the gradient tells how the surface moves, not when a strut appears or vanishes.

Resolution. For an integer **N_base** the grid has $N_i = \lceil N_{\text{base}} t_i e_i / e_{\text{max}} \rceil + 1$ cubes along axis i (at least 4), over the bounds (extents e_i) extended by 5 % per side. On the parametric cube all e_i are equal, so **N_base** is roughly the number of cubes per unit cell along every axis (a little less, because the grid also covers the margins). The paper's "resolution 20" is **N_base = 20**; the template's 10 with tiling $[2, 1, 1]$ gives $21 \times 11 \times 11$ cubes. Cost grows with N_{base}^3 times the number of cells.

In the code `create_3D_mesh(sdf, N_base, mesh_type, ..., deformation_function=None)` (`DeepSDFStruct/DeepSDFStruct/mesh.py:669`); the resolution rule is `process_N_base_input` (line 614). `get_verts` (line 995) evaluates `sdf(samples)` with autograd on, runs **FlexiCubes**, applies the deformation. `mesh_type="surface"` gives triangles, `"volume"` FlexiCubes' interior tets; the vertices keep their `grad_fn`. API and traps: Chapter 4.

2.7 Freeform deformation: from the parametric cube to metres

The physical domain is reached through a trivariate B-spline, $\bar{\mathbf{x}} = \sum_i N_i(\boldsymbol{\xi}) \mathbf{P}_i$. For a box this is a stretch, $(L\xi_1, W\xi_2, H\xi_3)$; moving the $\mathbf{P}_i$ bends the part without touching anything upstream. It is the same **TorchSpline** class as the latent field, so it is differentiable, but its control points are **not** design variables (the paper keeps the deformation fixed). **The FFD moves mesh vertices after extraction; the SDF never sees metres.** Anything sized in metres, such as a hole, must be converted to parametric units before it enters the SDF (`ScaledSpaceSDF`, Chapter 8), cap measures are parametric, and a strongly stretching FFD gives stretched elements.

In the code `get_verts` (`DeepSDFStruct/DeepSDFStruct/mesh.py:995`) calls `deformation_function.forward(verts)` after FlexiCubes. The template maps $[0, 1]^3$ onto a $2 \times 1 \times 1$ beam with `TorchSpline(splinepy.helpme.create.box(2, 1, 1).bspline)`.

2.8 Tetrahedralization: the open trade-off

FEM needs a volume mesh. There are two routes and neither is satisfactory yet:

Route	Gain	Cost
FlexiCubes interior tets <code>mesh_type="volume"</code> (library template)	Differentiable end to end.	About 30 % of the volume missing. On a solid unit cube the tets sum to 0.68–0.70 at every resolution; their boundary has 13 034 triangles in 5 836 components, against 1 200 in one for the surface: internal cavities.
Surface, then tetgen <code>tetrahedralize_surface</code> (<code>structsept/fem.py</code>)	Volume matches the surface to all printed digits, with fewer elements.	No gradient. NumPy and tetgen end the autograd graph at the surface vertices.

The cavity mesh gives the compliance of a body with 30 % voids; the tetgen mesh gives the right compliance but no sensitivities. The two possible ways out are discussed in Chapter 8 (What is missing before an optimization loop). On either route torch-fem needs positively oriented tets (the element

Jacobian determinant is $6V_e$): the template and `fem.py` flip negative tets and drop tets with volume at most 10^{-12} .

In progress Unresolved. Until it is, compliance from the library template is that of the cavity mesh, not of the geometry the decoder describes.

2.9 FEM: linear elasticity, stiffness, compliance

On $\bar{\Omega}$ the structure obeys $\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0}$, $\boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$, $\boldsymbol{\sigma} = \mathbf{C} : \boldsymbol{\varepsilon}$. In the weak form, $\int \boldsymbol{\varepsilon}(\mathbf{v}) : \mathbf{C} : \boldsymbol{\varepsilon}(\mathbf{u}) \, dx = \ell(\mathbf{v})$, the left side becomes the stiffness matrix. It depends on geometry and material only, not on loads or supports, which is why `structsept/fem.py` can assemble $\mathbf{K}$ before choosing any boundary condition. For linear tetrahedra the shape functions are barycentric, the strain-displacement matrix $\mathbf{B}$ (6×12) is constant, and one integration point is exact:

$$\mathbf{k}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{C} \mathbf{B} \, dx = V_e \mathbf{B}^T \mathbf{C} \mathbf{B} \quad (12 \times 12), \quad \mathbf{K} = \sum_e \mathbf{L}_e^T \mathbf{k}_e \mathbf{L}_e. \quad (6)$$

The Boolean $\mathbf{L}_e$ are never formed: each entry of $\mathbf{k}_e$ is scatter-added at its pair of global degrees of freedom. Without supports $\mathbf{K}$ is singular, with the six rigid-body modes ($\mathbf{u} = \mathbf{c}$, $\mathbf{u} = \boldsymbol{\omega} \times \mathbf{x}$) as kernel. Checking $\mathbf{K} \mathbf{u}_{\text{rigid}} = \mathbf{0}$ tests $\mathbf{B}$ and the assembly, not the material: $\mathbf{B} \mathbf{u}_{\text{rigid}}$ is already zero element by element, so the product vanishes for any $\mathbf{C}$. Dirichlet constraints remove the kernel.

The optimization needs the compliance, the volume and the volume constraint:

$$J = \mathbf{f}^T \mathbf{u} = \mathbf{u}^T \mathbf{K} \mathbf{u}, \quad V = \sum_e V_e, \quad G = V - V_{\text{target}} \leq 0. \quad (7)$$

J is twice the strain energy (the paper writes $\int_{\Omega} \boldsymbol{\sigma}(\mathbf{u}) : \boldsymbol{\varepsilon}(\mathbf{u}) \, dx$); minimizing it maximizes stiffness under the given load. For design-independent loads compliance is self-adjoint, $dJ/d\mathbf{p} = -\mathbf{u}^T (\partial \mathbf{K} / \partial \mathbf{p}) \mathbf{u}$, with no second solve. The paper solved the FEM with MFEM through its Python wrapper and wrote this sensitivity out as a shape derivative (its Eqs. 36–37); the library and this repo use torch-fem, where autodiff does it: torch-fem's sparse solve is a custom autograd function whose backward solves $\mathbf{K}^T \mathbf{g} = \partial J / \partial \mathbf{u}$ (the adjoint solve) and returns $\partial J / \partial \mathbf{K}$; autograd continues from there into $\mathbf{k}_e$, V_e and the vertex positions.

In the code `structsept/fem.py` goes as far as $\mathbf{K}$: torch-fem's `solid.k0(C)` returns the $\mathbf{k}_e$ above, `solid.assemble_matrix` does the scatter-add, and `check_stiffness` (`structsept/fem.py:230`) runs the rigid-body test (Chapter 8). The library template adds supports, load, solve, J and G (Chapter 4).

2.10 MMA: updating the design

The Method of Moving Asymptotes (Svanberg, 1987) replaces, at every iteration, the objective and the constraints by convex, separable approximations around the current design $\mathbf{x}^{(k)}$,

$$\tilde{F}(\mathbf{x}) = r + \sum_j \left(\frac{p_j}{U_j - x_j} + \frac{q_j}{x_j - L_j} \right), \quad L_j < x_j < U_j, \quad (8)$$

with $p_j, q_j \geq 0$ set from the gradient so that $\tilde{F}$ matches F and its slope at $\mathbf{x}^{(k)}$. The asymptotes move closer to x_j when a variable oscillates (more conservative) and away when it moves steadily (bolder); the subproblem is solved cheaply (`mmapy` uses a primal-dual Newton method). MMA fits because each evaluation costs a full FEM solve plus a backward pass, there are many variables and few constraints, and only first derivatives exist.

In the code `MMA(parameters, bounds, max_step=0.1, n_constraints=1)` (`DeepSDFStruct/DeepSDFStruct/optimization.py:170`). `step(F, dF, G, dG)` (line 349) takes the

objective, the constraints and their gradients, solves one MMA subproblem with `mmapy.mmasub` and writes the new $\hat{\lambda}$ back into `parameters`. It normalizes F by its first value but not G , so scale the constraint yourself ($G = V/V_{\text{target}} - 1$), and it has no stopping rule. API and traps: Chapter 4.

Bounds. The template bounds every control point to $[0.15, 0.75]$: radii of the analytic round cross, inside the paper’s training range of radii 0.1 to 0.75. Outside its trained range a real decoder returns meaningless geometry. Trained codes have no physical scale (the bundled **RoundCross** codes span $[-1, 1]$, **ChiAndCross** $[-0.95, 0.70]$ and $[-0.98, 0.64]$ in its two components), so derive bounds from the per-component minimum and maximum of `model._trained_latent_vectors`. By the partition of unity they then hold for $\lambda(\xi)$ everywhere.

2.11 The gradient chain

One `create_3D_mesh` call plus one torch-fem solve form a single torch computation from $\hat{\lambda}$ to J ; `torch.autograd.grad(J, param)` walks it backwards (Figure 3) and MMA turns the result into the next design.

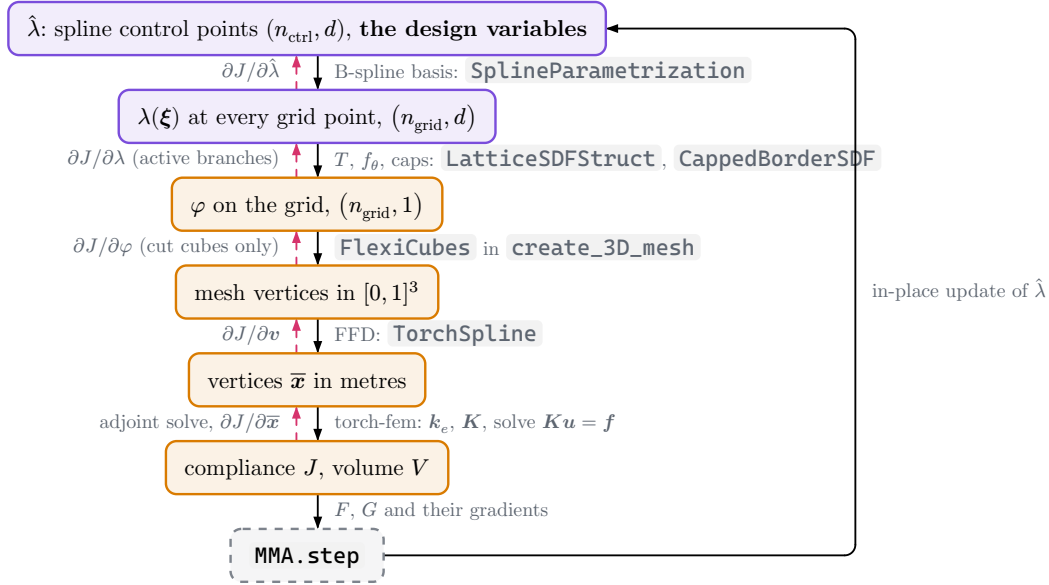


Figure 3: The gradient chain of one optimization iteration. Black: the forward pass and the class doing each step. Pink dashed: the backward pass of `torch.autograd.grad`. The FFD control points and the decoder weights sit on the chain but are not design variables.

Every link must be a torch operation recorded on the autograd graph. Hence the rule from **CLAUDE.md**, for anything between $\hat{\lambda}$ and J or V inside the loop: **no** `.detach()` (it returns a tensor cut off from the graph), **no** NumPy round trip (`.numpy()`, `np.*`, `torch.as_tensor(array)`: NumPy has no autograd, the rebuilt tensor is a constant), **no** `.item()` or `float()` (a Python number is a constant too), and **no** `torch.no_grad()` block. Breaking the rule raises no error: the gradient silently comes out zero, or misses part of the chain. Some things only look like violations: `.item()` for logging (the template does it), Boolean masks and index tensors, and a `torch.autograd.Function` with its own backward, such as torch-fem’s solve, which uses SciPy inside. The tetgen path of `fem.py` does break the rule, and the GUI breaks it on purpose (Chapter 7).

Tip After any change inside the loop, check that the gradient is non-zero (the template asserts it at the end) and that one component agrees with a finite difference: perturb one control point by $\pm h$, re-mesh, re-solve. Topology changes make finite differences noisy, so keep h small and `N_base` moderate.

Chapter 3

The repository

This chapter is the map: what lives where, the one rule that keeps the folders apart, how the packages are installed and why, where generated files land, and what state git is in.

3.1 The map

The repo root is the VS Code workspace. You edit code in four folders: `structsept/`, `datagen/`, `experiments/` and `tests/`. `DeepSDFStruct/` is the library, a git submodule. Everything in `data/`, `runs/` and `outputs/` is generated and not versioned.

code/	repo root = VS Code workspace			
— pyproject.toml	uv.lock	one distribution "struct-sept"; the locked environment		
— CLAUDE.md	briefing loaded into every Claude Code session			
— install_shortcuts.bat	fresh clone → uv sync → two desktop shortcuts			
— structsept/	LIBRARY: importable, nothing runs on import			
— plate_with_hole.py	recurring test geometry + ScaledSpaceSDF			
— fem.py	SDF → tetrahedral mesh → global stiffness K			
— pointcloud_sdf.py	SDF of a point cloud with normals			
— app/	the GUI and the training drivers			
— main.py	sdf_maker.py	the two windows		
— tab_explore.py	tab_explore2d.py	tab_train.py	the three tabs	
— hyperparams.py	hparam_window.py	xlsx.py	specs.json schema, sheets	
— training.py	unattended.py	trainer wrapper; headless runs		
— datasets.py	models.py	viz.py	dataset list, lattice, drawing	
— launcher.py	icons/	desktop shortcut start-up		
— runtime.py	widgets.py	theme.py	run_editor.py	GUI plumbing
— datagen/	TRAINING DATA: writes data/, imports nothing here			
— plate_hole_params.py	plate_hole_sdf.py	circular hole family		
— plate_tri_params.py	plate_tri_sdf.py	four triangles (untracked)		
— make_plate_hole.py	make_plate_tri.py	CLIs: parameters → data/		
— dataset.py	THE on-disk contract of a dataset			
— preview.py	preview.png of a written dataset			
— experiments/	RUNNABLE ONE-OFFS: never imported			
— plate_geometry.py	plate_with_hole_network.py	early studies of		
— plate_with_hole_stiffness.py	pointcloud_to_lattice.py	the library		
— make_plate_presets.py	untrained preset runs → runs/			
— train_plate_*.py	four unattended training scripts			
— IDEIAS.md	lab notebook (Portuguese): every finding			
— tests/	pytest; test_deepsdfstruct_env.py = smoke test			
— docs/	background documents + this handbook			
— DeepSDFStruct/	git submodule; the package is DeepSDFStruct/DeepSDFStruct/			
— data/	runs/	outputs/	generated, gitignored	
— .vscode/	.claude/	F5 configs + interpreter; the /update-context command		

Two things surprise people. `DeepSDFStruct/` is the submodule checkout and the Python package sits one level down, so library paths in this handbook start with `DeepSDFStruct/DeepSDFStruct/`. And `structsept/app/` is more than the GUI: `training.py` and `unattended.py` drive the trainer for the headless scripts too (Chapter 6). The gitignored files in the root (a copy of the paper and its text `.paper_text.txt`, `np.py`, `round_cross_00000.npz`, `struct_sept.egg-info/`) are not used by any code.

3.2 The one import rule

Imports point one way and never bend back (Figure 4):

- `experiments/` imports `structsept/`, which imports `DeepSDFStruct` (experiments may also use the library directly). `structsept/` never imports from `experiments/`, and experiment scripts never import each other, not even for a small helper.

- **datagen/** and **structsept/** never import each other. **datagen** writes **data/**, and the readers (Train tab, training scripts, library trainer) know only the file layout defined in **datagen/dataset.py** (Chapter 5). Any generator that writes this layout is a valid data source; the SDF maker window writes it from meshes.
- **tests/** may import anything except **experiments/**. The tests are where **datagen** and **structsept.app** meet: **test_datagen.py** and **test_app_explore2d.py** write a tiny dataset and train a decoder on it.

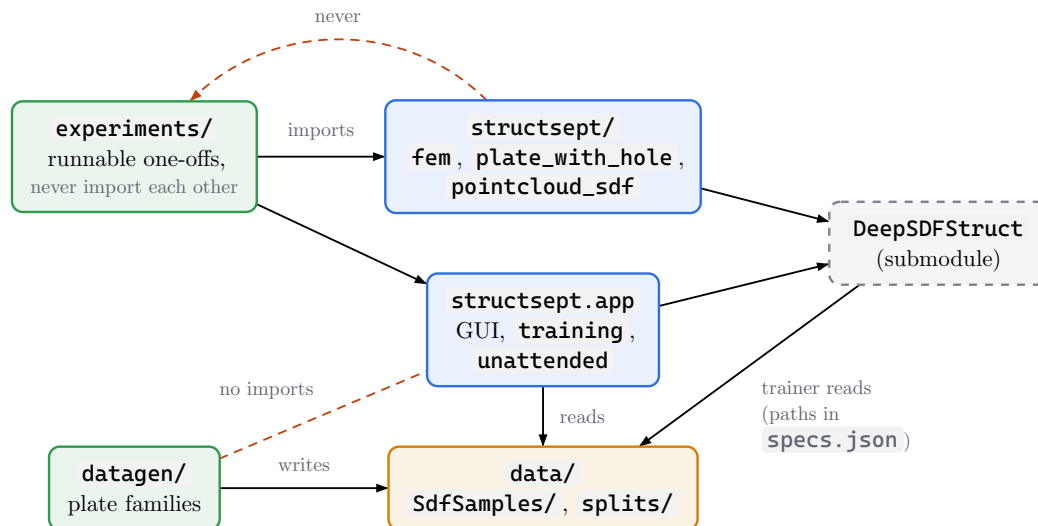


Figure 4: The import rule; an arrow means “depends on”, red dashed means forbidden. **datagen** reaches the rest only through the folder **data/** (which the SDF maker in **structsept.app** also writes). Inside **structsept/**, **app** and the three modules do not import each other.

Why. An experiment answers one question and is then finished. If library or GUI code depended on it, editing a one-off would break something unrelated. Keeping **datagen** behind a folder means the network side never cares how a dataset was made, and the generators stay free of torch.

Promote, never import sideways. When a second file needs a helper that an experiment script defined, the helper moves into **structsept/**. The precedent: when **train_plate_tri.py** needed the code-versus-parameter check of **train_plate_r_only_4h.py**, the function moved to **code_vs_parameter (structsept/app/unattended.py:525)** and the r-only script became a thin wrapper (not committed yet). Nothing enforces the rule automatically; today a search for imports in the forbidden directions finds none.

3.3 Where does my new file go?

If you are writing ...	... it belongs in
a new parametric shape family for training data	datagen/ : <family>_params.py, <family>_sdf.py, make_<family>.py (Chapter 5)
something a second file needs; a geometry, solver, transform, data structure	structsept/ , or structsept/app/ if it is about training or the GUI
a GUI tab or window	structsept/app/tab_<name>.py or <name>_window.py
a training run left alone for hours	experiments/train_<...>.py on top of structsept.app.unattended
a figure, mesh or number you look at once	experiments/
an assertion that must stay true	tests/test_<area>.py ; GUI tests are test_app_<part>.py
a result, a number, a decision	experiments/IDEIAS.md : data/ , runs/ , outputs/ are not versioned
an explanation of <i>why</i>	docs/
a fix or feature of the method itself	DeepSDFStruct/ , deliberately: it edits the upstream submodule

Unsure? Put it in `experiments/`. Promoting a script later costs a `git mv` and an import line; untangling a library module that grew around one experiment costs much more.

3.4 Packages and installs

`pyproject.toml` is 31 lines, and each setting fixes a specific problem:

Setting	What it does, and why
<code>name = "struct-sept"</code> <code>pyproject.toml:2</code>	One distribution. Its only declared dependency is <code>DeepSDFStruct</code> ; <code>torch</code> , <code>torch-fem</code> , <code>splipy</code> , <code>pyvista</code> , <code>mlflow</code> and even <code>pytest</code> come in through the library's own list.
<code>packages = [...]</code> <code>pyproject.toml:17</code>	The import packages <code>structsept</code> , <code>structsept.app</code> , <code>datagen</code> . Listed by hand, since auto-discovery would trip over the other top-level folders. <code>experiments/</code> and <code>tests/</code> are deliberately not packages.
<code>package = true</code> <code>pyproject.toml:22</code>	Installs this folder editable into <code>.venv</code> : <code>import structsept</code> and <code>import datagen</code> work from any directory, with no <code>sys.path</code> hacks.
<code>[tool.uv.sources]</code> <code>pyproject.toml:30</code>	<code>DeepSDFStruct</code> from the submodule folder, editable too: an edit there is live.
<code>editable_mode = "compat"</code> <code>pyproject.toml:25</code>	Installs <code>DeepSDFStruct</code> through a plain path <code>.pth</code> instead of <code>setuptools</code> ' import hook, so the real package beats the same-named checkout folder (below).
<code>link-mode = "copy"</code> <code>pyproject.toml:28</code>	The repo lives under OneDrive, where <code>uv</code> 's hardlinks fail (os error 396) when it rebuilds the submodule. Copying is slower but works.

Why `compat` mode. Without it, a script run from the repo root would import the checkout folder `DeepSDFStruct/` (it has no `__init__.py`) as a namespace package: imports of submodules still work, but the library's own `__init__.py` never runs. A test guards this case: `test_package_resolves_with_repo_root_on_path` (`tests/test_deepsdfstruct_env.py:64`).

In practice. Edits in `structsept/`, `datagen/` or `DeepSDFStruct/` are live at the next import, and so is a new module inside those packages; only a new *top-level* package must be added to `packages`. Run `uv sync` after any change to `pyproject.toml`, `uv.lock` or the submodule pointer (the desktop launcher re-syncs by itself when `pyproject.toml`, `uv.lock` or `DeepSDFStruct/pyproject.toml` changed, Chapter 7). Ask before adding a dependency: the environment is heavy (216 locked packages, `torch 2.13.0+cpu`, so training runs on the CPU) and resolution is slow. Always go through `uv run`; in VS Code, `.vscode/settings.json` pins `.venv/Scripts/python.exe` and every F5 configuration runs from the repo root.

The library takes its version from git metadata (`setuptools-scm`), so the submodule must be a real checkout: clone with `--recursive`, never download a zip. In short (the full recipe is in Chapter 9):

```
git clone --recursive -b gui-train-tab https://github.com/tomito6/struct_opt_nn.git code
cd code
uv sync
```

3.5 Where output lands

Path	Written by	Tracked
<code>data/</code>	<code>make_plate_hole</code> , <code>make_plate_tri</code> , the SDF maker window	no
<code>runs/</code>	Train tab, <code>train_plate_*.py</code> , <code>make_plate_presets.py</code>	no
<code>outputs/logs/</code>	unattended scripts (<code><run>.log</code> , <code><run>.pid</code>); the launcher (<code>explorer.log</code> , <code>sdf_maker.log</code>) for shortcut starts	no
<code>outputs/</code>	<code>plate_geometry.py</code> , the <code>datagen.*_params</code> CLIs (<code>--outdir</code> default)	no
<code>experiments/outputs/</code>	the other three early experiments and the <code>structsept.plate_with_hole</code> CLI	no
<code>docs/stiffness_theory/figures/</code>	<code>make_figures.py</code> plus renders; used by the PDF	yes
<code>docs/figures/</code>	GUI screenshots (<code>gui.md</code> uses two of the three)	yes

`data/` and `runs/` are located from the code's own path (`structsept/app/main.py:49`, `datagen/dataset.py:48`), so they always land in the repo root. All of it can be regenerated, which is also why a result only counts once it is written in `experiments/IDEIAS.md`.

Watch out The `--outdir` defaults (`outputs`, `experiments/outputs`) are relative to the *current directory*: run scripts from the repo root, or output lands wherever your shell happens to be. And every `runs/<run>/specs.json` stores *absolute* paths to `data/`, so `runs/` does not survive a moved folder or a new machine; rebuild the datasets and rerun `experiments/make_plate_presets.py` (Chapter 6).

3.6 Naming conventions

- **Language.** Folders, modules and GUI strings are English. `IDEIAS.md` has been written in Portuguese so far (Chapter 10 summarises its findings in English); agree with the team on the language of new entries.
- **Modules** are named for the object they describe (`plate_with_hole`, `fem`), not an action. Experiment scripts read as a sentence about their product: `plate_with_hole_stiffness.py` is that plate, taken as far as the stiffness.
- **Data families** come as a triple `<family>_params.py`, `<family>_sdf.py`, `make_<family>.py`, with `<family> = plate_hole` or `plate_tri`. Datasets are `<family>_2d_<variant>` (`plate_hole_2d_r_only`, `plate_hole_2d_xyr_n166`, `plate_tri_2d_hw`); shapes inside are named by their parameters, with `p` for the decimal point: `hole_x0p1200_y0p1200_r0p0700`.
- **Runs.** The GUI names a run `<dataset>_d<d>_<YYYYmmdd_HHMM>` (`structsept/app/tab_train.py:716`); scripts name it after the recipe, as in `plate2d_r_only_d1_8x256_4h` (latent dimension, layers $\times$ width, time budget). Untrained recipes start with `preset_plate2d_`; those of `make_plate_presets.py` are `preset_plate2d_<variant>_d<d>` (`r_only`, `xyr`, `tri_h`, `tri_hw`). The run name ties together `runs/<run>/`, `outputs/logs/<run>.log` and `docs/hyperparameters/NN_Training_Hyperparameters_<run>.xlsx`.

3.7 The docs/ folder

File	What it is
<code>paper_context.md</code>	The paper, distilled: notation-to-code table, transformation function, offline/online loop, test cases. Read it before touching an optimization loop.
<code>structure.md</code>	The short version of this chapter; partly outdated (see the last section).
<code>gui.md</code>	The GUI panel by panel, screenshots in <code>figures/</code> (Chapter 7).
<code>stiffness_theory/</code> <code>stiffness_theory.pdf</code>	Theory note in Portuguese (Typst): from the network to the stiffness matrix, tied to <code>structsept/fem.py</code> (Chapter 8).
<code>hyperparameters/</code>	The supervisor's xlsx template and filled sheets for the three <code>*_8x256_4h</code> runs; imported by the GUI via <code>from_sheet</code> (<code>structsept/app/hyperparams.py:1426</code>).
<code>context-maintenance.md</code> <code>context-log.md</code>	The procedure that keeps <code>CLAUDE.md</code> small and true, and the log of each pass.
<code>handbook/</code>	This handbook (<code>handbook.typ</code> , <code>chapters/</code> , <code>build.py</code>) and the companion <code>board.html</code> .

The root file `CLAUDE.md` is the briefing that Claude Code loads into every session, together with the library's `AGENT_INSTRUCTIONS.md`; it is also a fair one-page summary for humans. It has a budget of 150 lines and is never appended to directly: update it only through `/update-context` (procedure in `docs/context-maintenance.md`).

3.8 Git state

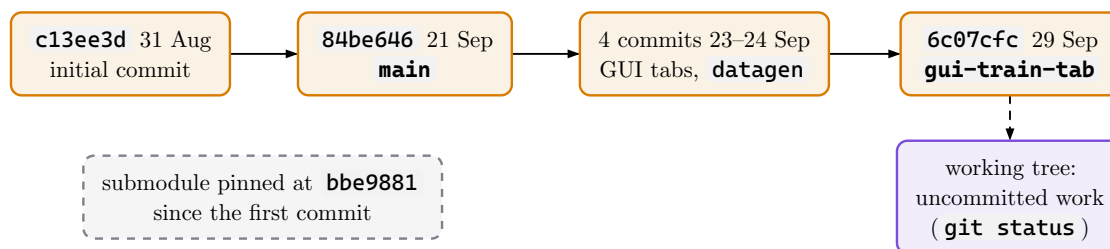


Figure 5: The history: `main` is five commits behind `gui-train-tab` (pushed, not merged), and the newest work is not committed at all.

The remote `origin` is `github.com/tomito6/struct_opt_nn`, a public repository (hence the ignored paper PDF). `gui-train-tab` holds everything from 23–29 September: GUI redesign, `datagen` and 2-D decoders, the radius-only family and presets, launcher and unattended training. The submodule points at `bbe9881` (“Update Readme (#77)” on upstream `main`) and has no local changes.

Watch out A plain `git clone` checks out `main`, which has no `datagen/`, no launcher and only six files in `structsept/app/`. Clone with `-b gui-train-tab`, or check it out first.

In progress Not committed (2 October 2026; run `git status` for the current list). *Modified*, among others: the `--large-holes` option in `datagen/make_plate_hole.py` and `plate_hole_params.py`; `prepare_run_like` and `code_vs_parameter` in `structsept/app/unattended.py` with the thinned `train_plate_r_only_4h.py`; two triangle presets in `make_plate_presets.py`; smaller edits to other `datagen/` and `structsept/app/` modules; `IDEIAS.md`, `docs/structure.md`, `docs/paper_context.md` and the tests. *Untracked*: the triangle family (`datagen/make_plate_tri.py`, `plate_tri_params.py`, `plate_tri_sdf.py`, `preview.py`), `experiments/train_plate_tri.py`, `experiments/train_plate_xyr_n166.py`, `docs/handbook/`. Commit before you build on any of it.

3.9 Docs that have drifted

`CLAUDE.md` predates most of `gui-train-tab`, and `docs/structure.md` is current only in part. Where they disagree with the code (the git history, the contents of `structsept/app/` and `datagen/`, the installed packages, where the paper PDF is), trust the code and this handbook. `CLAUDE.md` is also off on four method details, each explained where it comes up: the decoder lives on $[-1, 1]^3$ and only the lattice on the unit cube; an uncapped lattice does close at the borders, the caps just make the faces flat and exact, so keep them (both Chapter 2); this repo tetrahedralizes with `tetgen` rather than inside `create_3D_mesh`, and `PointCloudSDF` needs outward normals (both Chapter 8). And the `check_stiffness` docstring and the theory note credit the rigid-body test with checking the material $\mathbf{C}$ too; since $\mathbf{B}u_{\text{rigid}} = \mathbf{0}$, it tests only $\mathbf{B}$ and the assembly. Claude Code reads `CLAUDE.md` at every start, so it repeats these claims until the file is fixed.

Chapter 4

The DeepSDFStruct library

`DeepSDFStruct` implements the paper. It is a git submodule (`DeepSDFStruct/` at commit `bbe9881`, working tree clean; upstream `mkofler96/DeepSDFStruct` on GitHub) whose importable package is the inner folder `DeepSDFStruct/DeepSDFStruct/`. The library has two halves that meet in one object, the `DeepSDFModel`: a decoder network plus the table of latent codes it learned. The **learning half** turns SDF samples into such a model; the **geometry and optimisation half** uses it as a geometry generator inside the loop. Chapter 2 explains the concepts. This chapter is the API: which class does what, what a call looks like, and where it bites. Edit the submodule only when you mean to change the library itself (Chapter 3).

4.1 Module map

Module	What is in it	Used here by
<code>SDF.py</code>	<code>SDFBase</code> , booleans, <code>CappedBorderSDF</code> , <code>SDFfromDeepSDF</code>	everything
<code>sdf_primitives.py</code>	Analytic shapes: <code>SphereSDF</code> , <code>BoxSDF</code> , <code>CylinderSDF</code> , ...	<code>plate_with_hole</code>
<code>sdf_operations.py</code>	Space warps: <code>TwistSDF</code> , <code>ShellSDF</code> , <code>RepeatSDF</code> , ...	–
<code>lattice_structure.py</code>	<code>LatticeSDFStruct</code> , <code>transform = T(x)</code>	geometry, GUI
<code>parametrization.py</code>	<code>Constant</code> , <code>SplineParametrization</code>	geometry, GUI
<code>torch_spline.py</code>	<code>TorchSpline</code> (latent field <i>and</i> FFD), <code>TorchScaling</code>	geometry
<code>mesh.py</code> <code>flexicubes/</code> <code>flexisquares/</code>	<code>create_3D_mesh</code> , <code>create_2D_mesh</code> , <code>tetrahedralize_surface</code> ; adapted NVIDIA FlexiCubes and its 2-D twin	<code>fem.py</code> , GUI
<code>deep_sdf/models.py</code>	<code>DeepSDFModel</code> : decoder plus code table	via <code>get_model</code>
<code>deep_sdf/networks/</code>	Decoder classes, chosen by <code>NetworkArch</code>	<code>deep_sdf_decoder</code>
<code>deep_sdf/training.py</code>	<code>train_deep_sdf</code> ; DeepLS: <code>training_latent_field.py</code>	<code>app/training.py</code>
<code>deep_sdf/workspace.py</code>	Folder layout, <code>init_decoder</code> , <code>load_trained_model</code>	<code>app/training.py</code>
<code>deep_sdf/data.py</code>	<code>SDFSamples</code> : reads <code>SdfSamples/</code> via a split file	tests
<code>deep_sdf/reconstruction.py</code>	<code>reconstruct_from_samples</code>	indirectly
<code>sampling.py</code>	<code>SampledSDF</code> , the dataset writer <code>SDFSampler</code>	SDF maker
<code>pretrained_models.py</code>	<code>PretrainedModels</code> , <code>get_model</code> , <code>trained_models/</code>	everything
<code>geom_reconstruction.py</code>	<code>build_parameter_spline</code> , <code>LocalShapesReconstructor</code>	GUI, experiments
<code>optimization.py</code>	<code>MMA</code> , <code>tet_signed_vol</code> , <code>get_mesh_from_torchfem</code>	all but <code>MMA</code>

The remaining files (`local_shapes.py`, whose `LocalShapesSDF` is an older variant of `LatticeSDFStruct`; `design_of_experiments.py`; the `export_*` scripts; `splinepy_unitcells/`; `visualization/`) are not used here. Library docstrings often lag the code (wrong defaults, examples that call functions which no longer exist). Trust the code.

4.2 The geometry side

4.2.1 Everything is an SDF module

Code: `SDFBase` `DeepSDFStruct/DeepSDFStruct/SDF.py:194`, `UnionSDF` `DeepSDFStruct/DeepSDFStruct/SDF.py:575`, `DifferenceSDF` `DeepSDFStruct/DeepSDFStruct/SDF.py:660`

`SDFBase` is a `torch.nn.Module`. `sdf(points)` checks that `points` is `(N, 3)` or `(N, 2)`, calls the subclass's `_compute` and returns `(N, 1)` signed distances, negative inside; that, plus a check of the output's row count, is all `forward` does. A subclass implements `_compute` and `_get_domain_bounds`, a `(2, dim)` box the mesher uses by default. A geometry is therefore one module tree, and `parameters()` on its root finds every

differentiable number in it. The **leaves** are primitives with `nn.Parameter` sizes (plus `SDFfromMesh`, which goes through NumPy and breaks gradients) and the network wrapper `SDFfromDeepSDF`. The **combinators** are `UnionSDF` (min), `DifferenceSDF`, `Smooth*SDF`, `CappedBorderSDF`, `LatticeSDFStruct` and the warps. `a + b` returns `UnionSDF(a, b)`; no other operator is overloaded.

Watch out `UnionSDF`, `SmoothUnionSDF`, `SmoothIntersectionSDF` and the subtracted objects of `DifferenceSDF` and `SmoothDifferenceSDF` keep their children in a plain Python list, not as registered submodules, so `.to()`, `named_parameters()` and the lattice search in `create_3D_mesh` do not see them; a lattice inside a `UnionSDF` is meshed without scaling the resolution by its tiling. `DifferenceSDF.base_obj`, `CappedBorderSDF.sdf` and `LatticeSDFStruct.microtile` are registered.

4.2.2 From checkpoint to SDF

Code: `get_model` DeepSDFStruct/DeepSDFStruct/pretrained_models.py:98, `DeepSDFModel` DeepSDFStruct/DeepSDFStruct/deep_sdf/models.py:23, `SDFfromDeepSDF` DeepSDFStruct/DeepSDFStruct/SDF.py:1199, `set_latent_vec` DeepSDFStruct/DeepSDFStruct/SDF.py:1261

```
model = get_model(PretrainedModels.AnalyticRoundCross, device="cpu") # or a run folder
cell = SDFfromDeepSDF(model) # callable SDF on [-1, 1]^3, latent = code 0
cell.set_latent_vec(torch.tensor([0.4]))
phi = cell(torch.zeros(1, 3)) # (1, 1)
```

`get_model(model, checkpoint="latest", device=None)` accepts an enum entry or the path (as a `str`) of any experiment folder, so the project's `runs/<run>` load the same way. It reads `specs.json`, rebuilds the decoder named by `NetworkArch`, loads the `ModelParameters/` and `LatentCodes/` checkpoints, and returns a `DeepSDFModel`.

`DeepSDFModel` is a plain object, not an `nn.Module`. The good consequence: decoder weights never appear in `sdf.parameters()`, so an optimiser built on a geometry tree moves only codes or control points, which is exactly what reconstruction and MMA need. The bad one: `sdf.to(device)` does not move the decoder. `get_model` picks CUDA when it is available, while `create_3D_mesh` defaults to `"cpu"`; always pass `device=model.device` to the mesher.

`SDFfromDeepSDF(model, max_batch=32**3)` evaluates the decoder in batches. Its domain is $[-1, 1]^3$, the box the decoder was trained on, not the unit cube. It takes `geometric_dim` from the decoder, so a 2-D decoder gives a 2-D SDF. `set_latent_vec` accepts two shapes. A `(d,)` code becomes a `Constant` parametrization: one shape everywhere. An `(N, d)` tensor is stored as `latvec`, one code per query point; this is what a lattice writes, and `_compute` uses `latvec` whenever it is set.

4.2.3 The pretrained models

PretrainedModels.	NetworkArch	d	What it is
<code>AnalyticRoundCross</code>	<code>analytic_round_cross</code>	1	Three crossing cylinders in closed form; the latent <i>is</i> the radius
<code>RoundCross</code>	<code>deep_sdf_decoder</code>	1	Trained cross cell; 20 codes in $[-1, 1]$ (the analytic model ships a copy of this table)
<code>ChiAndCross</code>	<code>deep_sdf_decoder</code>	2	Chi-shaped cell combined with a diagonal cross
<code>Primitives</code> (= <code>PrimitivesCL32</code>), <code>PrimitivesCL16</code> , <code>PrimitivesCL08</code>	<code>deep_sdf_decoder</code>	32, 16, 8	DeepLS (Deep Local Shapes, Chapter 2) models: one latent <i>field</i> per training scene; the stored code table is zeros
<code>Primitives2D</code>	<code>deep_sdf_decoder</code>	16	DeepLS on 2-D primitives (still a 3-D decoder)

Watch out `AnalyticRoundCross` is a formula, not a network (Chapter 2): its values away from the struts are not exact distances, and the template's bounds `[0.15, 0.75]` are strut radii. For a trained decoder, take bounds from the range of its trained codes, `model._trained_latent_vectors`.

4.2.4 LatticeSDFStruct: tiling and the latent hand-off

Code: `LatticeSDFStruct` `DeepSDFStruct/DeepSDFStruct/lattice_structure.py:39`, `_compute` `DeepSDFStruct/DeepSDFStruct/lattice_structure.py:172`, `transform` `DeepSDFStruct/DeepSDFStruct/lattice_structure.py:262`

`LatticeSDFStruct(tiling, microtile, parametrization, bounds=None)` tiles a microtile over a box that defaults to the **parametric unit cube** $[0, 1]^d$. A `parametrization` module is required; `None` raises `TypeError`. `_compute` runs five steps:

1. Mask the query points inside the box.
2. Evaluate the latent field there, `(n_in, d)`, and push it into the microtile with `microtile._set_param`; for an `SDFfromDeepSDF` this sets `latvec`.
3. Map each coordinate into $[-1, 1]$ with `transform`, the paper's Eq. (18), for x normalised to $[0, 1]$ and t cells along that axis:

$$T(x) = 4 \left| \frac{tx}{2} - \left\lfloor \frac{tx}{2} + \frac{1}{2} \right\rfloor \right| - 1 \quad (9)$$

A triangle wave: cell 0 runs from -1 to 1 , cell 1 back from 1 to -1 . Neighbour cells are **mirror images**, not translated copies, so the field stays continuous even for an asymmetric cell.

4. Evaluate the microtile at the mapped points.
5. Give points outside the box their positive distance to the box, so the mesher's wider grid sees empty space there.

The values are in microtile units (a cell spans 2): the zero level set is right, the magnitudes are scaled by about $2t$ per axis.

Watch out Step 2 is shared, mutable state. Each evaluation overwrites `microtile.latvec` with one code per point of *that* query. Two evaluations at once (a worker meshing while a slider redraws) collide, and the loser fails with a shape mismatch. A stale `latvec` also breaks a later direct call on the microtile with another point count, even after a 1-D `set_latent_vec`, because `latvec` wins. The GUI (Chapter 7) locks the controls that touch the SDF while a worker runs (`run_worker`, `structsept/app/runtime.py:65`) and previews cells with a separate wrapper (`unit_cell_sdf`). Elsewhere: one wrapper per use, or set `latvec = None` first.

4.2.5 The latent field: SplineParametrization and TorchSpline

Code: `SplineParametrization` `DeepSDFStruct/DeepSDFStruct/parametrization.py:133`, `TorchSpline` `DeepSDFStruct/DeepSDFStruct/torch_spline.py:275`, `build_parameter_spline` `DeepSDFStruct/DeepSDFStruct/geom_reconstruction.py:85`

`SplineParametrization(spline, device)` wraps a `splinespy` B-spline in a `TorchSpline`, which copies the control points into an `nn.Parameter` of shape `(n_cp, d)` and evaluates the spline with a vectorised de Boor algorithm in torch. That parameter is $\hat{\lambda}$, the design vector; `next(parametrization.parameters())` returns it. The control net is flattened x-fastest, index $i + n_x(j + n_y k)$, which the GUI's `flat_index` relies on. The spline is evaluated at lattice coordinates, so its parametric box must equal the lattice box; `build_parameter_spline` makes a clamped spline with one knot span per cell. `set_param(values)` overwrites the control points under `no_grad`, without a rebuild. `Constant(value)` is the one-code-everywhere baseline that `plate_with_hole` still uses.

`TorchSpline` evaluates non-rational B-splines only (the NURBS in its docstring are not implemented). The same class is the FFD: in the template, a `TorchSpline` of the box spline `create.box(2, 1, 1)` maps $[0, 1]^3$ onto a $2 \times 1 \times 1$ beam. Its control points are not design variables.

4.2.6 CappedBorderSDF: closing the borders

Code: `CappedBorderSDF` `DeepSDFStruct/DeepSDFStruct/SDF.py:1450`, default dict `UNIT_CUBE_CAPS_3D` `DeepSDFStruct/DeepSDFStruct/SDF.py:132`

`CappedBorderSDF(sdf, cap_border_dict=None, scale=(1, 1, 1))` combines its child with planes at the child's box faces. Keys `x0` to `z1` name the faces, each mapped to `{"cap": c, "measure": m}`. With $c =$

`-1` the face *trims* (a `max`, cut m inside the face); with $c = +1$ it *adds* a solid plate of thickness m (a `min`). Measures are in parametric units; the default dict trims all six faces flush. This replaces the jump from strut (negative) to box distance (positive) at the lattice border by a clean plane. Without it the surface still closes, but between grid nodes at an uncontrolled place (Chapter 2); the template and every lattice the project builds itself are capped, so do the same. A plate fills the whole half-space beyond its face, because a `min` with a plane has no outer end, so nest: the inner wrapper adds plates, the outer one trims back to the box. The default dict is one shared module-level object; never mutate a wrapper's `cap_border_dict` in place.

```
plates = {"x0": {"cap": 1, "measure": 0.05}, "z1": {"cap": 1, "measure": 0.1}}
sdf = CappedBorderSDF(CappedBorderSDF(lattice, plates)) # add plates, then trim
```

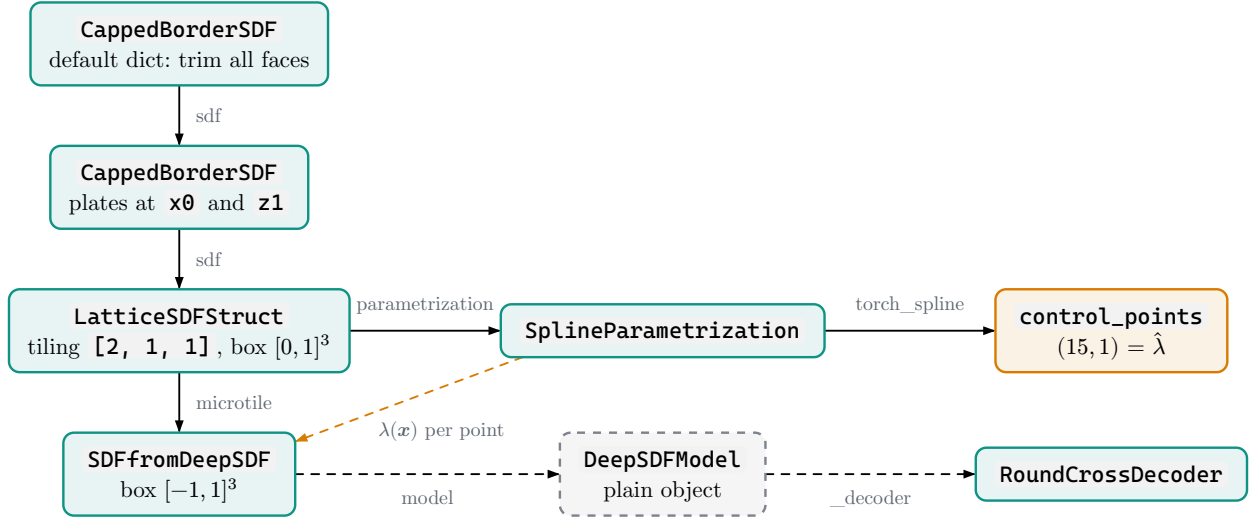


Figure 6: The geometry tree of the optimisation template. Solid arrows are registered submodules, seen by `parameters()` and `.to()`. Dashed arrows are plain attributes; the orange one is the latent hand-off (`_set_param`) at every evaluation.

4.2.7 Meshing

Code: `create_3D_mesh` DeepSDFStruct/DeepSDFStruct/mesh.py:669, `process_N_base_input` DeepSDFStruct/DeepSDFStruct/mesh.py:614, `tetrahedralize_surface` DeepSDFStruct/DeepSDFStruct/mesh.py:451, `create_2D_mesh` DeepSDFStruct/DeepSDFStruct/mesh.py:848

```
create_3D_mesh(sdf, N_base, mesh_type, differentiate=False, device="cpu", bounds=None,
               diffmode="fwd", deformation_function=None, use_tiling=True,
               extend_bounds=True) # → (torchSurfMesh | torchVolumeMesh, None)
```

The mesher takes the box from `sdf._get_domain_bounds()` (it must be a tensor) and widens it by 5 % on each side. An integer `N_base` becomes $\lceil N_{\text{base}} \cdot e_i / e_{\text{max}} \cdot t_i \rceil + 1$ cubes along axis i , at least 4, with e_i the box extents and t_i the tiling of the first lattice in the tree. For the usual cube-shaped parametric box that is $N_{\text{base}} t_i + 1$: **`N_base` is roughly the number of FlexiCubes cubes per unit cell and axis** (a little less, since the grid also spans the 5 % margins), whatever the physical shape. The template's 10 on `[2, 1, 1]` gives $21 \times 11 \times 11$ cubes; the paper's 20 on $6 \times 3 \times 3$ cells gives $121 \times 61 \times 61$.

The SDF is evaluated on that grid with autograd on. FlexiCubes extracts triangles ("`surface`") or its own tetrahedra ("`volume`", all reoriented positively), and `deformation_function` (a `TorchSpline` or `TorchScaling`) then moves the **vertices** into physical space. The SDF never sees physical coordinates, so anything sized in metres must be converted first (`ScaledSpaceSDF`, Chapter 8). The returned vertices carry `grad_fn`. `differentiate=True` does *not* switch gradients on; it adds an explicit Jacobian of the vertices with respect to all parameters, which is expensive and only serves derivative export.

`tetrahedralize_surface(surface_mesh)` fills a closed `gus.Faces` surface with tetgen (switches `pYq`) and returns the volume mesh plus the indices of the surface vertices in it. It works in NumPy: **the gradient stops there**. `create_2D_mesh` is the 2-D twin on FlexiSquares ("`line`", "`surface`", "`surface_triangle`");

it ignores the SDF's box and uses $[-0.05, 1.05]^2$ unless given `bounds`. Nothing in the project's own code calls it (only a library test does).

Watch out The template's `mesh_type="volume"` is differentiable but fills only about 70 % of the enclosed volume; surface plus tetgen is exact but not differentiable, and is the path `structsept/fem.py` takes. The trade-off is explained in Chapter 2.

4.3 The learning side

4.3.1 `train_deep_sdf`, step by step

Code: `train_deep_sdf` `DeepSDFStruct/DeepSDFStruct/deep_sdf/training.py:299`, whose only caller here is `train` `structsept/app/training.py:129`

```
train_deep_sdf(experiment_directory, data_source, continue_from=None, batch_split=1,
               device=None)           # device: the string "cpu" or "cuda", nothing else
```

The trainer fits a classic auto-decoder: one MLP plus one free latent code per training shape. The project always runs it on the CPU. In order:

1. Read `specs.json` from the experiment folder; build the decoder (line 409).
2. Seed Python, NumPy and torch with `seed` (default 42, line 415), *after* the decoder already drew its initial weights.
3. Load every sample file of the split `Path(data_source) / TrainSplit` into RAM (line 433); `datagen/` writes them (Chapter 5). The split order is the code index, saved as `latent_code_data_map.json`.
4. Create the codes, `nn.Embedding(n_scenes, CodeLength, max_norm=CodeBound)`, with initial spread `CodeInitStdDev /  $\sqrt{\text{CodeLength}}$` (line 471).
5. Build one Adam optimiser with two groups, decoder and codes, each driven by its own entry of `LearningRateSchedule` (line 492).
6. Per batch: `SamplesPerScene` points (half outside, half inside) from each of `ScenesPerBatch` shapes; input = code concatenated with coordinates; prediction and target clamped to $\pm$ `ClampingDistance`; loss = `LossFunction` (default `clampedL1`, which clamps again at a fixed 0.1) + `CodeRegularizationLambda` · $\min(1, \text{epoch} / 100)$ · $\text{mean } \|\lambda\|$. Decoder gradients are clipped to norm 1.0 (line 637).
7. Save `<epoch>.pth` at multiples of `SnapshotFrequency` and at `AdditionalSnapshots`, `latest.pth` every `LogFrequency` epochs (line 684), `training_summary.json` at the end.

4.3.2 The experiment folder and its recipe

An experiment is a folder with a `specs.json`; the trainer fills in the rest. The bundled `trained_models/<name>/` and the project's `runs/<run>/` share this layout (the project adds `metadata.json` and analysis files, Chapter 6):

<code>specs.json</code>	the recipe; the only file you write
<code>ModelParameters/{latest,<ep>}.pth</code>	<code>{"epoch", "model_state_dict"}</code>
<code>OptimizerParameters/{latest,<ep>}.pth</code>	Adam state (needed by <code>continue_from</code>)
<code>LatentCodes/{latest,<ep>}.pth</code>	<code>{"epoch", "latent_codes": {"weight": (n, d)}}</code>
<code>LatentCodes/latent_code_data_map.json</code>	code index $\rightarrow$.npz file
<code>Logs.pth, Logs.png</code>	loss per batch, learning rates, magnitudes
<code>training_summary.json</code>	final loss, epochs, duration, host, version

The keys that decide a run: `NetworkArch`, `NetworkSpecs` (hidden widths `dims`, skip layers `latent_in`, `geom_dimension` 2 or 3, ...), `CodeLength` (d), `NumEpochs`, `LearningRateSchedule` (decoder, then codes), `SamplesPerScene`, `ScenesPerBatch`, `ClampingDistance`, `CodeRegularizationLambda` (default 10^{-4} , always applied), `CodeBound`, `CodeInitStdDev`, `TrainSplit` and `seed`. The full schema, its validation and the spreadsheet import are in Chapter 6; formats in Appendix B.

4.3.3 Reconstruction

Code: `reconstruct_from_samples` DeepSDFStruct/DeepSDFStruct/deep_sdf/reconstruction.py:60,
`LocalShapesReconstructor` DeepSDFStruct/DeepSDFStruct/geom_reconstruction.py:337

`reconstruct_from_samples(sdf, sdfSample, ...)` runs Adam or L-BFGS (default `num_iterations=1000`) passes over the samples in batches of 512, learning rate $5 \cdot 10^{-4}$) on all `sdf.parameters()` against clamped-L1 targets from a `SampledSDF`; with the decoder unregistered, those are only codes or control points. `LocalShapesReconstructor` packages the “mesh to lattice of local shapes” fit: normalise the mesh, start a degree-1 latent spline at the mean trained code, sample the target, fit. `experiments/pointcloud_to_lattice.py` uses it to turn a point cloud into a starting design (Chapter 8).

4.4 The optimisation side: MMA

Code: `MMA` DeepSDFStruct/DeepSDFStruct/optimization.py:170, `MMA.step`
 DeepSDFStruct/DeepSDFStruct/optimization.py:349

`MMA(parameters, bounds, max_step=0.1, n_constraints=1)` runs Svanberg’s MMA (from `mmapy`) on a torch tensor:

```
x = next(lattice.parametrization.parameters()) # (n_cp, d); MMA updates it in place
opt = MMA(x, np.tile([0.15, 0.75], (x.numel(), 1))) # bounds: shape (numel, 2)
for it in range(n_iter):
    F, V = forward() # mesh → FEM, all in torch
    G = V / V_target - 1.0 # ≤ 0 means feasible
    dF = torch.autograd.grad(F, x, retain_graph=True)[0]
    dG = torch.autograd.grad(G, x)[0]
    opt.step(F, dF, G, dG)
```

`step` moves everything to NumPy, divides F and dF by $|F|$ of the first iteration, sets move limits $x \pm \text{max_step}$ (absolute, although the docstring says “fraction of the range”) clipped to the bounds, solves the subproblem, and copies the new design into `parameters` under `no_grad`; the next forward pass rebuilds the graph. All four arguments must be tensors. Constraints are not normalised, so scale G yourself. There is **no stopping criterion**: `opt.ch`, the relative change of the mean design, is only logged.

In progress MMA is not called anywhere in `structsept/` or `experiments/`. The project’s geometry code still uses a `Constant` latent; a `SplineParametrization` is built only by the GUI (`models.build_lattice`) and, inside the library’s `LocalShapesReconstructor`, by the point-cloud fit. The optimisation loop of this repo is still to be built, starting from the template below.

4.5 The template: test_structural_optimization.py

Code: `DeepSDFStruct/tests/test_structural_optimization.py:21`

This test is the only complete instance of paper steps 3–6 and the starting point for an optimisation run. It is a scaled-down paper test case 1 (which has $6 \times 3 \times 3$ cells, $9 \times 2 \times 5$ control points, resolution 20). Line by line:

1. **Decoder** (26–27): `get_model(AnalyticRoundCross)` with no device, so CUDA if available; then `SDFfromDeepSDF`.
2. **Plates** (29–32): `x0` 0.05 thick (clamped face), `z1` 0.1 thick (loaded face).
3. **Latent spline** (34–38): degrees `[1, 0, 1]`, knots giving $5 \times 1 \times 3 = 15$ control points, all 0.3, a strut radius since $d = 1$.
4. **Parametrization and FFD** (42–48): tiling `[2, 1, 1]`, `SplineParametrization`, and a `TorchSpline` box that maps $[0, 1]^3$ onto the $2 \times 1 \times 1$ beam.
5. **Geometry tree** (51–56): `LatticesSDFStruct` inside two `CappedBorderSDF` (Figure 6).
6. **Optimiser** (59–66): `param` is the control-point tensor; `target_vol = 0.5` (absolute: a quarter of the beam volume 2); bounds `[0.15, 0.75]`; then `MMA`.
7. **Mesh** (74–92): `create_3D_mesh` at `N_base` 10, `mesh_type="volume"`, with the FFD, gives tets in physical space; a second, surface mesh only checks watertightness.

8. **Clean the tets** (99–124): reorient negative tets by swapping two vertices, drop tets with float64 volume $\leq 10^{-12}$; `vol` is the differentiable sum of the rest.
9. **FEM** (128–150): default dtype float64; a `torchfem.solid.Solid` with $E = 1000$, $\nu = 0.3$; all DOFs fixed at $x < 10^{-5}$; a total force of -100 in z spread over the unclamped nodes with $z > 0.9$; `spsolve`.
10. **Update** (153–159): $F = f \cdot u$, $G = V - 0.5$, both gradients by `torch.autograd.grad` with respect to `param`, then `optimizer.step`.
11. **Wrap-up** (162–171): dtype back to float32, `sim_out.vtk` written to the working directory, both gradients asserted non-zero. One iteration (`num_iter=1`).

```
# from the repo root; writes sim_out.vtk into the current directory
uv run pytest DeepSDFStruct/tests/test_structural_optimization.py -v
```

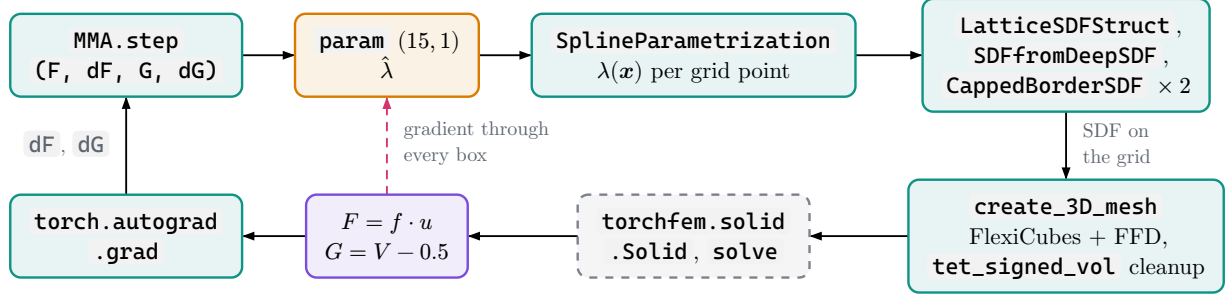


Figure 7: One iteration of the template with the real names. `MMA.step` writes the new design into `param` in place; the pink arrow is the autograd path from F and G back through FEM, FFD, FlexiCubes, the caps, the decoder and the spline basis.

For a real run, change at least: the iteration count and a stopping rule; a decoder with matching bounds (`np.tile` for $d > 1$; the test’s `np.zeros(param.shape) + [lo, hi]` works only because $d = 1$); a normalised G ; the volume path (box above); output per iteration. If anything raises between lines 128 and 162, float64 stays the default dtype for the whole process. The recipe is in Chapter 9.

4.6 Library-level traps

Besides the boxes above (volume cavities, analytic `AnalyticRoundCross`, decoder device, shared `latvec`, unregistered children), two more:

- **Silent mismatch.** Weights load with `strict=False`: a `specs.json` that does not fit them leaves layers random, without an error.
- **Where to run.** `uv` from the repo root only: `DeepSDFStruct/` has its own `pyproject.toml` and `uv.lock`, and `uv run` inside it builds a second environment.

The trainer’s own traps (seed order, hardcoded clipping, lost codes on resume, `latest.pth` lag, ...) are in the table at the end of Chapter 6.

Chapter 5

Training data: the `datagen` package

Step 1 of the paper’s offline stage samples points around the training geometries and computes their signed distance; the paper does this for 3-D unit cells. The current research in this repo uses 2-D plate families whose signed distance is known in closed form, so the samples are computed exactly and no mesh is involved. `datagen/` is the package that does this: it draws admissible shapes from a parametric family, evaluates their exact SDF at uniform and near-boundary points, and writes a DeepSDF training set into `data/`. Two families exist, a plate with one circular hole and a square plate with four triangular holes, and seven datasets built from them are on disk. What the trainer does with the files is Chapter 6; why the generating parameters are stored next to the samples is the research question of Chapter 10.

5.1 Why a separate package

`datagen` imports nothing from `structsept`, `experiments` or `DeepSDFstruct`, and no library or app code imports `datagen`. Only tests do: `tests/test_datagen.py`, and `tests/test_app_explore2d.py`, which builds a small fixture dataset with it. The boundary is a folder, as the package docstring (`datagen/__init__.py:1`) puts it:

```
datagen/  --writes→  data/  ←reads--  Train tab, experiments/train_*.py, DeepSDF trainer
```

This buys three things. The file layout in `datagen/dataset.py` is the whole interface, so any generator that writes it is a valid data source; the GUI’s SDF maker (Chapter 7) writes the same layout from meshes, without manifest and parameter table. `datagen` needs only NumPy, SciPy and Matplotlib, never torch, so generating data does not depend on the training stack. And the shape parameters visibly never reach the network: they leave `datagen` only through `params.csv` (and encoded in the instance names, which the trainer uses only as file names), and only analysis code reads them.

In progress The triangle family, `preview.py` and `--large-holes` are not committed (file list under Git state in Chapter 3); the datasets `plate_hole_2d_xyr_n166`, `plate_tri_2d_h` and `plate_tri_2d_hw` were built from that code. Commit before you build on it.

5.2 Parameters, field, glue

Every family is split into the same three layers (Figure 8). A `params` module (`plate_hole_params`, `plate_tri_params`) decides which parameter tuples are legal and draws a set of them; it is pure arithmetic in design units and has its own CLI to inspect the design space. An `sdf` module (`plate_hole_sdf`, `plate_tri_sdf`) computes the exact field of one tuple in the decoder’s $[-1, 1]^d$ frame and the sample points; the triangle module imports the frame, extrusion and sampling recipe from the hole module. A `make` script (`make_plate_hole`, `make_plate_tri`) is the CLI glue. Below them sits `dataset.py`, the on-disk contract, which knows nothing about plates, and `preview.py`, which draws the field of 6 shapes with the stored rows on top, read back from disk as a check.

The split pays off because admissibility is arithmetic on parameters: it can be checked, plotted and unit-tested without computing a single distance. The field module never decides whether a shape is legal; it assumes that it is, and that assumption is exactly what makes its formula exact.

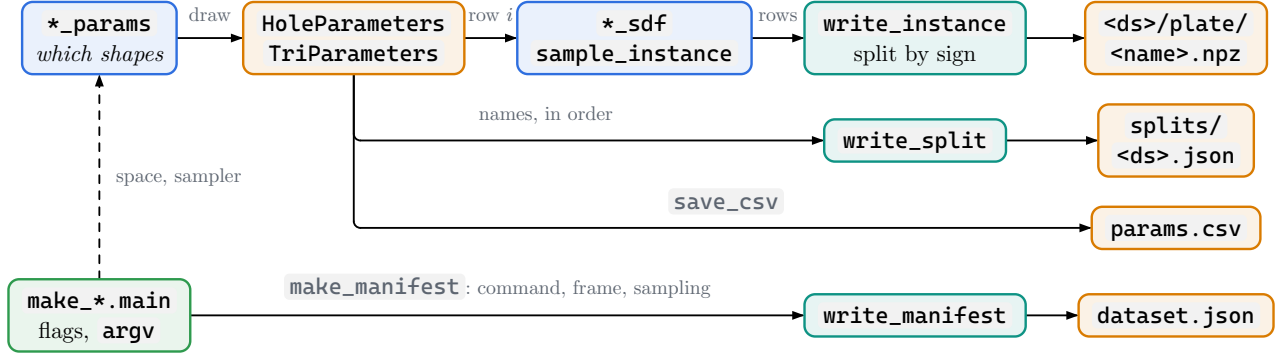


Figure 8: How one dataset is written. The `make_*` script drives every step. The parameter set fixes the order of the shapes, and that one order is reused for the `.npz` loop (shape i seeded with `[seed, i]`), the split and the parameter table.

5.3 The hole family

5.3.1 Which holes are legal

The plate is $[0, L] \times [0, W]$ in design units, default 1×1 , origin at the lower-left corner. A hole (x_c, y_c, r) must keep a ligament of at least m to every edge; otherwise it is a notch, not a hole. The four conditions $x_c - r \geq m$, $L - x_c - r \geq m$, $y_c - r \geq m$, $W - y_c - r \geq m$ collapse into one:

$$r_{\min} \leq r \leq d(x_c, y_c) = \min(x_c - m, L - x_c - m, y_c - m, W - y_c - m) \quad (10)$$

The margin may also be four values (left, right, bottom, top), for when a strip of one edge will carry a load or a support.

For a fixed r , the centre must lie in a rectangle that shrinks as r grows. In (x_c, y_c, r) space the feasible set is therefore a **pyramid**, not a box (Figure 9). Drawing the three parameters from their bounding box and rejecting the illegal ones would discard two thirds of the draws (`feasible_fraction()` gives 0.332, the volume of a pyramid over its box), and the survivors would be biased: large holes only fit near the centre.

The fix is a map from the unit cube $(u_x, u_y, t) \in [0, 1]^3$ onto the pyramid:

$$x_c = x_{lo} + u_x(x_{hi} - x_{lo}), \quad y_c = y_{lo} + u_y(y_{hi} - y_{lo}), \quad r = r_{\min} + t(d(x_c, y_c) - r_{\min}) \quad (11)$$

The centre box (`center_bounds`) is $[x_{lo}, x_{hi}] = [m + r_{\min}, L - m - r_{\min}]$ in x , and likewise in y . Every point of the cube is a legal hole, no draw is wasted, and the inverse map is exact. The unit coordinates are stored in `params.csv` next to (x_c, y_c, r) . They have a second purpose: a box is what MMA wants as bounds on design variables, so the unit coordinates are the natural *prescribed* latent codes (`latent_codes()`). Nothing calls that method yet; every training so far learns its codes (Chapter 10).

$r_{\min} = 0.07$ is strictly positive for three independent reasons: $r = 0$ is a change of topology; a hole must span several FlexiCubes cells to mesh as a circle (the smallest hole spans 4.48 cells at `N_base` = 32 cubes across the plate, the FlexiCubes resolution of Chapter 2; the rule asks for 4); and the decoder blurs small features.

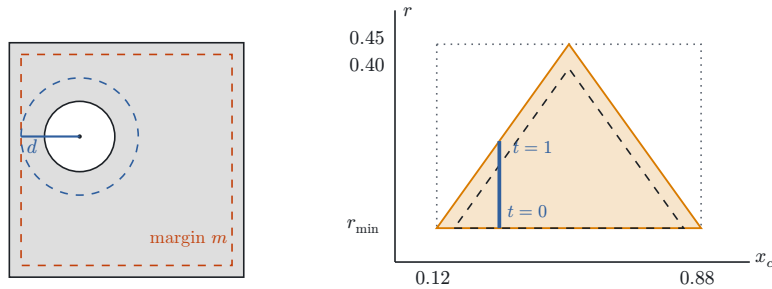


Figure 9: The hole family. Left: one hole of the training family (margin $m = 0.05$) and its centre. The largest hole at that centre (dashed) touches the margin line; its radius is $d(x_c, y_c)$. Right: section of the feasible set at $y_c = 0.5$. Filled: training margin 0.05; dashed: design margin 0.1; dotted: the box rejection sampling would draw from. The blue segment is one column of the reparametrisation, t running from $r_{\min}$ to $d(x_c, y_c)$.

5.3.2 Three samplers

Sampler	What it draws	Flag, dataset
<code>sample(n=64, method="sobol")</code>	All three parameters in the unit cube (Sobol, or <code>lhs</code> , <code>random</code> , <code>grid</code>), with 10 extreme points prepended (<code>_UNIT_EXTREMES</code> : the cube corners, the smallest and the largest centred hole)	default <code>xyr</code> , <code>n134</code> , <code>n25</code>
<code>sample_radius(n=40, centre=None)</code>	A fixed centre (the plate centre by default) and radii evenly spaced from $r_{\min}$ to <code>max_radius(centre)</code>	<code>--radius-only</code> <code>r_only</code>
<code>sample_large_holes(n, r_from, seed)</code>	r uniform in $[r_{\text{from}}, r_{\text{max}}]$, then the centre uniform in the section of the pyramid at that height	<code>--large-holes</code> <code>xyr_n166</code>

Why `--n 128` gives 134 shapes. At the corners of the centre box the radius span is zero, so the 8 cube corners give only 4 distinct holes, all of radius $r_{\min}$. The sampler removes duplicates on the physical triple (at 9 decimals), which leaves 6 extremes plus 128 Sobol points. The extremes come first in the order, so they hold latent indices 0 to 5. With `--n 19` the same rule gives the 25 shapes of `plate_hole_2d_n25`.

Why `sample_large_holes` exists. A large hole needs a centre near the middle *and* a t near 1, a small corner of the cube. The 134-shape Sobol set has only 8 holes with $r > 0.25$ and none between $r = 0.342$ and the extreme at 0.45. A decoder trained on it has a stretch of latent space without any shape (the gap in the GUI's latent-coverage view). The radius bias `t_power` cannot fix this, because it does not move the centres inwards. `make_plate_hole` appends the large holes with `HoleParameters.extended()` and seed + 1, so the first 134 shapes keep their index and, thanks to the per-shape seeding below, their files.

5.3.3 Training margin and design margin

`PlateHoleSpace` defaults to $m = 0.1$, the margin an optimizer would respect, but `make_plate_hole` trains with `TRAINING_MARGIN = 0.05`. The decoder thus sees a slightly larger family than the optimizer may explore ($r_{\max}$ 0.45 instead of 0.40, centre box $[0.12, 0.88]$ instead of $[0.17, 0.83]$), so the MMA bounds never sit on the edge of the training data. `PlateHoleSpace.contains()` checks this, and the parameter CLI prints the verdict. The triangle family has no such buffer: it uses margin 0.1 for both.

```
uv run python -m datagen.plate_hole_params --n 128 --margin 0.05 --design-margin 0.1
```

In the code `PlateHoleSpace` (`datagen/plate_hole_params.py:141`) holds the condition; `max_radius` (:231) is d ; `from_unit` (:308) and `to_unit` (:331) are the cube map; the samplers are `sample` (:347), `sample_radius` (:424) and `sample_large_holes` (:519). The result is a `HoleParameters` (:650) with `names`, `extended` (:711), `latent_codes` (:688) and `save_csv` (:764). `TRAINING_MARGIN` is `datagen/make_plate_hole.py:85`.

5.4 The exact signed distance

The frame. `PlateFrame` maps design units into the decoder's frame:

$$\mathbf{x}_{\text{norm}} = s(\mathbf{x}_{\text{design}} - \mathbf{x}_{\text{centre}}), \quad s = \frac{2(1 - \text{pad})}{\max(L, W)} \quad (12)$$

With $\text{pad} = 0.1$ the scale is $s = 1.8$, and the unit plate occupies $[-0.9, 0.9]^2$ inside the $[-1, 1]^2$ sampling box. The empty ring lets the decoder learn the outer edge from both sides instead of at the border of everything it ever sees. The scale is the same on every axis, so normalized distances are true distances; divide by s to get design units.

The field. With φ_{box} the exact distance to the rectangle and $\mathbf{c}$, R the hole in normalized units,

$$\varphi(\mathbf{p}) = \max(\varphi_{\text{box}}(\mathbf{p}), -(\|\mathbf{p} - \mathbf{c}\| - R)) \quad (13)$$

A boolean `max` is in general only a bound on the distance near the places where two surfaces meet. Here they never meet, because admissibility keeps a ligament of at least m : outside the plate the outer edge is the nearest boundary; in the material both terms are negative and the `max` picks the nearer surface; inside the hole its wall is nearer than the edge by at least the ligament. The formula is therefore the

exact Euclidean distance, **but only for admissible parameters**; the tests compare it with brute-force distances to a densely sampled boundary. Sign convention, as everywhere in DeepSDF: $\varphi < 0$ in the material, $\varphi \geq 0$ outside the plate and in the hole.

2-D or 3-D. `--dim` has no default, on purpose. With `--dim 3` the 2-D field is extruded exactly to the plate thickness (default 0.1) and the rows are (x, y, z, φ) . But a through-hole plate of constant thickness carries all its information in the plane; a 3-D decoder would spend most of its samples on two flat faces that are the same for every shape. `--dim 2` (rows (x, y, φ)) loses nothing, and the 2-D field can still be extruded later. Every dataset on disk is 2-D; the 3-D path is exercised only by the tests. A 2-D dataset trains a decoder with `geom_dimension 2`, which the GUI shows in the Explore 2-D tab (Chapter 7), not in the lattice pipeline.

5.5 Where the samples go

`sample_instance` returns all rows of one shape: `n_uniform + n_band` points with their exact φ , as `float32`.

- **Uniform points** in $[-1, 1]^d$, pad ring included. They teach the coarse field and the sign far from the surface.
- **Band points:** points on the boundary, moved along the normal by $\mathcal{N}(0, 1) \cdot \sigma$, with σ drawn per point from `stds = (0.05, 0.025)`, the two levels of DeepSDFStruct's mesh sampler. A share `hole_fraction = 0.5` of the band goes to the hole wall (spread by angle) whatever the hole's size, the rest by arc length to the outer outline: the hole is the only thing that changes between shapes. In 3-D the rest is split by area between the faces and the side walls.

The band carries half the budget because the trainer clamps target and prediction to $\pm\delta$, $\delta = 0.1$ by default (Chapter 6): only the band $|\varphi| < 0.1$ is learned accurately, and a uniform point far from the plate contributes little more than its sign. Since φ is evaluated exactly at every point, a band point pushed past a corner or across a thin ligament still carries the right label; band points can land slightly outside the box ($|x|$ up to about 1.16 in the datasets on disk).

Counts. The defaults are $10^4 + 10^4$ points for 2-D and $5 \cdot 10^4 + 5 \cdot 10^4$ for 3-D, but every dataset on disk was built with 25 000 + 25 000, i.e. 50 000 rows per shape. The trainer then draws `SamplesPerScene` rows per shape and step, half from `pos` and half from `neg`, whatever the ratio in the file. The inside share of the files (40 to 69 %) therefore never reaches the network.

Seeding. Shape i gets its own generator, `default_rng([seed, i])`, instead of sharing one with the whole loop. A file thus depends only on the seed, its own index, its parameters and the sampling settings. Appending shapes leaves the earlier files byte-identical: the first 134 files of `plate_hole_2d_xyr_n166` are bit for bit those of `plate_hole_2d_xyr`. Changing `--seed` or the counts changes every file; reordering the shapes changes the file of every shape that moves.

In the code In `datagen/plate_hole_sdf.py`: `PlateFrame (:100)`, `box_sdf_2d (:192)`, `plate_hole_sdf_2d (:211)`, `extrude (:236)`, `SamplingConfig (:285)`, `band_points (:377)`, `sample_instance (:434)`, `DEFAULT_COUNTS (:93)`. The per-shape seeding is in the loop of `make_plate_hole.main`, `datagen/make_plate_hole.py:407`. The trainer's half-and-half draw is `unpack_sdf_samples_from_ram` (the trainer always loads the split into RAM), `DeepSDFStruct/DeepSDFStruct/deep_sdf/data.py:169`.

5.6 The triangle family

The second family (all of it untracked, see the box at the top of this chapter) is a square plate of side S (default 1) with one isosceles triangle cut out of each quarter (Figure 10). The base is parallel to the nearest edge, the tip points at the plate centre, and the centre of each triangle (the midpoint of its height) is fixed a quarter of the side in from its edge. What remains is a frame plus an X. All four triangles share height h and base width w . Four ligaments depend on (h, w) and must all be at least $m = 0.1$:

- **wall**, base to plate edge: $S/4 - h/2$;

- **tip gap**, tip to plate centre: also $S/4 - h/2$, because the triangle centre sits halfway between edge and plate centre;
- **arm** of the X, between neighbouring triangles: $\sqrt{2} \min(S/4 + (h - w)/2, S/4 - h/2)$;
- **side**, base corner to the side edges: $(S - w)/2$.

There are two families. In the **shared-height** family (default) the base is tied to the height, $w = 2h$: a 90° tip, so the slanted edges of neighbours are parallel and every arm has constant width. The wall binds, so $h \in [h_{\min}, S/2 - 2m] = [0.05, 0.3]$, and `PlateTriSpace.sample` draws an even grid of 40 heights (names like `tri_h0p0500`). In the **free-width** family (`--free-width`) $w \in [w_{\min}, w_{\max}(h)]$ with

$$w_{\max}(h) = \min(w_{\text{cap}}, h + S/2 - \sqrt{2}m, S - 2m), \quad w_{\min} = 0.1, \quad w_{\text{cap}} = 0.6 \quad (14)$$

The arm rule binds below $h \approx 0.24$ and the cap above it, so the feasible set is a rectangle with one corner cut off, not a box. As for the hole, it is sampled through a unit square (t_h, t_w) : `sample_hw` draws 128 Sobol points and prepends the 4 corners and the centre of the square, 133 shapes (names like `tri_h0p0500_w0p1000`). The shared-height family is the diagonal $w = 2h$ of this set.

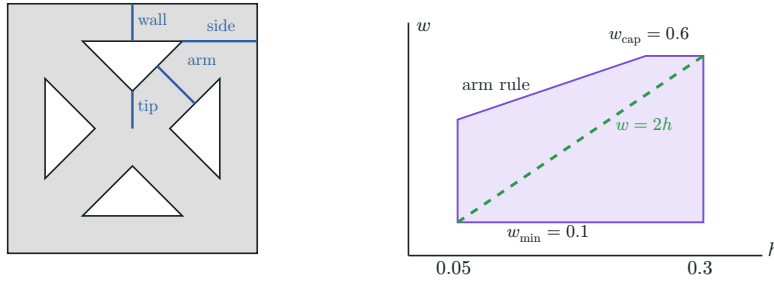


Figure 10: The triangle family. Left: $h = 0.2$, $w = 2h$, with the four ligaments. Right: the feasible set of the free-width family in (h, w) . The arm rule $w \leq h + 0.5 - \sqrt{2} \cdot 0.1$ cuts the corner; the shared-height family is the dashed diagonal.

The field is $\max(\varphi_{\text{box}}, -\min_i \varphi_{\text{tri},i})$, with `triangle_sdf_2d` the exact distance to a convex polygon; it is exact for the same reason as the hole. The band's `hole_fraction` is spread by arc length over the twelve triangle edges. The field functions also accept four separate heights (one per triangle), but no sampler or CLI flag produces them yet.

In the code `triangle_vertices` `datagen/plate_tri_params.py:160`, `PlateTriSpace` (:192) with `max_width` (:268), `sample` (:365) and `sample_hw` (:424); `triangle_sdf_2d` `datagen/plate_tri_sdf.py:115` and `plate_tri_sdf` (:175); the CLI is `make_plate_tri.main`, `datagen/make_plate_tri.py:274`.

Unlike $r_{\min}$, $h_{\min} = 0.05$ was set by hand and spans only 1.6 FlexiCubes cells at `N_base` = 32. The dataset does not care, since the field is exact, but meshing the smallest triangles for FEM later needs `N_base` ≥ 80 .

5.7 The on-disk contract

`datagen/dataset.py` defines the layout; the trainer fixes the name `SdfSamples/`, the GUI's dataset list the name `splits/`:

```
data/
├── SdfSamples/<dataset>/
│   ├── plate/<instance>.npz    arrays pos (phi ≥ 0) and neg (phi < 0), float32,
│   │                           rows (x, y, [z,] phi) in the normalized frame
│   ├── params.csv              generating parameters, one row per instance
│   ├── dataset.json            manifest: exact command, geom_dimension, frame, sampling
│   ├── preview.png, parameters.png (with --plot)
│   └── splits/<dataset>.json   {"<dataset>": {"plate": [instance names, in order]}}
```

The invariant: `split order` = `latent index` = `params.csv` row. The trainer gives latent code i to the i -th name of the split, and row i of `params.csv` describes the same shape. The manifest repeats the rule in its `order` key. Analysis code still joins codes to parameters *by name* through the split (`split_names`, `code_vs_parameter`) instead of trusting the row order.

File	Written by	Read by
<code><instance>.npz</code>	<code>write_instance</code>	the DeepSDF loader (half <code>pos</code> , half <code>neg</code>); the SDF maker's audit <code>validate_dataset</code> ; <code>preview.py</code>
<code>splits/<ds>.json</code>	<code>write_split</code>	the trainer, through <code>TrainSplit</code> in <code>specs.json</code> ; <code>split_names</code> ; <code>list_datasets</code> (only whether it exists)
<code>params.csv</code>	<code>save_csv</code>	<code>code_vs_parameter</code> ; the scripts <code>train_plate_xyr_4h.py</code> and <code>train_plate_xyr_n166.py</code> ; the tests
<code>dataset.json</code>	<code>write_manifest</code>	only the key <code>geom_dimension</code> , by <code>structsept/app/datasets.py</code> ; everything else is for humans

The manifest's most useful key is `command`, the exact CLI that built the dataset. `data/` is gitignored, so this is how a dataset is reproduced on another machine. The `frame` block (scale 1.8, pad 0.1) is informational: no code converts decoder coordinates back to design units for you. Without a manifest (the SDF maker's datasets) the GUI falls back to the row width of the first `.npz`. The columns of `params.csv` (one schema for holes, two for triangles) and every manifest key are listed in Appendix B.

`prepare()` refuses to write into an existing dataset (any `.npz` or the split file present) unless `--overwrite` is given. Even then it deletes only the files `datagen` owns in the `plate` class (`.npz` / `.stl`, manifest, parameter table, previews, split), and it refuses if another class folder holds samples. How a run's `specs.json` points at the split and at `data/` is in Chapter 6.

In the code Writers in `datagen/dataset.py`: `prepare` (:81), `write_instance` (:138), `write_split` (:145), `write_manifest` (:153). Readers: `list_datasets`, `geom_dimension` and `validate_dataset` in `structsept/app/datasets.py` (:44, :82, :271); `get_instance_filenames` and `unpack_sdf_samples_from_ram` in `DeepSDFStruct/DeepSDFStruct/deep_sdf/data.py` (:57, :169); `split_names` and `code_vs_parameter` in `structsept/app/unattended.py` (:403, :525); `to_specs`, `structsept/app/hyperparams.py`:1118.

5.8 The datasets on disk

All seven datasets are 2-D, have the single class `plate`, 25 000 + 25 000 rows per shape, pad 0.1 and scale 1.8; the hole sets use margin 0.05, the triangle sets 0.1. They take 16 to 101 MB each. Each one is rebuilt by the common command of its family, plus the flags in the last column, plus `--name <dataset>`; these are the flags that the `command` key of its manifest records.

```
uv run python -m datagen.make_plate_hole --dim 2 --n-uniform 25000 --n-band 25000 --plot
uv run python -m datagen.make_plate_tri --dim 2 --n-uniform 25000 --n-band 25000 --plot
# add --dry-run to print the space, the instances and the output size, writing nothing
```

Dataset	Family and draw	Shapes	Extra flags
<code>plate_hole_2d_n25</code>	hole, Sobol 19 + 6 extremes (first pilot set)	25	<code>--n 19</code>
<code>plate_hole_2d_n134</code>	hole, Sobol 128 + 6 extremes	134	<code>--n 128</code>
<code>plate_hole_2d_xyr</code>	the same draw; <code>.npz</code> files byte-identical to <code>n134</code>	134	none
<code>plate_hole_2d_xyr_n166</code>	<code>xyr</code> + 32 large holes, $r \in [0.25, 0.45]$	166	<code>--large-holes 32</code> <code>--large-holes-from 0.25</code>
<code>plate_hole_2d_r_only</code>	centred hole, 40 radii from 0.07 to 0.45	40	<code>--radius-only</code>
<code>plate_tri_2d_h</code>	triangles, 40 heights from 0.05 to 0.3, $w = 2h$	40	none
<code>plate_tri_2d_hw</code>	triangles, Sobol (h, w) 128 + 5 extremes	133	<code>--free-width</code>

The training presets written by `experiments/make_plate_presets.py` expect `r_only`, `xyr`, `tri_h` and `tri_hw`, and the unattended training scripts print the build command of their dataset when it is missing (Chapter 6). A third family means a new `*_params` / `*_sdf` / `make_*` triple against the same contract; the recipe is in Chapter 9.

5.9 Gotchas

Watch out

- **Without `--name` a hole dataset gets a default name** (`plate_hole_2d`, `plate_hole_2d_r`) that the presets do not know; they look for `plate_hole_2d_xyr` and `plate_hole_2d_r_only`. (The triangle defaults `plate_tri_2d_h` and `plate_tri_2d_hw` happen to match.) Without the two count flags you get $10^4 + 10^4$ rows and different files. To reproduce a dataset, copy the `command` from its `dataset.json`.
- **`n134` and `xyr` are twins**: identical `.npz` files and parameter tables, and splits that differ only in their dataset key. `n134` and `n25` predate the manifest keys `parameters.varied`, `parameters.fixed_centre` and `parameter_sampling.family`; code that reads them must tolerate their absence.
- **The margin default depends on the entry point**: 0.1 in `PlateHoleSpace` and in the `plate_hole_params` CLI, 0.05 in `make_plate_hole`. Inspect the training set with `--margin 0.05`.
- **The parameter CLIs plot into `outputs/` under the current directory**, not under the repo root (`--outdir`, default `outputs`). The `make_*` scripts put their plots into the dataset folder.
- **A leftover split file blocks a rebuild**: `prepare()` raises `FileExistsError`, so delete `splits/<ds>.json` too when you clean up by hand. The other way round, `get_instance_filenames` only logs a warning for a split entry without its `.npz`, and the trainer's RAM load right after it raises, before the first epoch.
- **Flag combinations**. `--large-holes` needs `--large-holes-from` and excludes `--radius-only`; `--centre` needs `--radius-only`. A Sobol draw whose n is not a power of two (`--n 19`) prints a harmless SciPy warning.

Chapter 6

Training a decoder

Before any optimization can happen, the project needs a trained DeepSDF decoder $f_\theta(\lambda, \mathbf{x})$: a network that maps a latent code λ of length d and a query point $\mathbf{x}$ to a signed distance. Chapter 2 explains the auto-decoder idea. This chapter shows how a decoder gets trained in this repository, what ends up on disk, and how you judge the result.

The training loop is not ours. It is the library function `train_deep_sdf`, and apart from the data folder it takes nothing but a directory: every setting comes from the `specs.json` inside it. So the code in this chapter does three things: write a correct `specs.json`, refuse one that would crash hours later, and record and analyse what the trainer leaves behind. There are two front ends, and both call the same three functions of `structsept/app/training.py` (`write_specs`, `train`, `write_metadata`; the two `--like` scripts copy a `specs.json` instead of calling `write_specs`): the GUI's **Train** tab for runs you watch, and the headless scripts `experiments/train_plate_*.py` for runs of hours.

6.1 What a run is

A **run** is a folder `runs/<name>/` that holds a `specs.json`; that is the whole definition (`training.list_runs`). The app or a script writes `specs.json` and a `metadata.json`, the trainer fills the folder with checkpoints and logs, and an analysis step may add result files. A run counts as *trained* once `ModelParameters/latest.pth` exists. A *preset* is a run folder with only the two JSON files: a recipe waiting to be used.

What the trainer optimises: every training shape i gets its own code λ_i , one row of a `torch.nn.Embedding` of shape (N_{shapes}, d) . The codes start as $\mathcal{N}(0, \sigma^2/d)$ per component ($\sigma = \text{CodeInitStdDev}$) and their norm is capped at `CodeBound`. Adam optimises the decoder weights θ and all codes together, with one learning-rate schedule per parameter group. For a batch B of (shape, point) samples at epoch e the loss is

$$\mathcal{L} = \frac{1}{|B|} \sum_{(i,j) \in B} |\text{cl}(f_\theta(\lambda_i, \mathbf{x}_j)) - \text{cl}(s_{ij})| + \lambda_{\text{reg}} \min\left(1, \frac{e}{100}\right) \cdot \frac{1}{|B|} \sum_{(i,j) \in B} \|\lambda_i\| \quad (15)$$

with s_{ij} the true signed distance and `cl` a clamp to $[-\delta, \delta]$, $\delta = \text{ClampingDistance}$. The network never sees the parameters that generated its shapes, only points and distances.

6.2 The hyperparameter schema

`structsept/app/hyperparams.py` describes every `specs.json` key the app lets you change. It contains no Tk code, so tests and scripts import it freely. Each key is one frozen `Field` (`structsept/app/hyperparams.py:86`) with a key, label, group, kind (`int`, `float`, `bool`, `choice`, `int_list`, `layers`, `opt_int`, `opt_float`, `text`), default, range, help text and the `specs.json` key it lands in. `FIELDS` (`structsept/app/hyperparams.py:292`) holds 36 of them in six `GROUPS`. The hyperparameter window builds its form from this table, the Train tab its one-line summary, and the `specs.json` writer and reader their key lists. Adding a hyperparameter is one `Field` entry plus one line each in `to_specs` and `from_specs`.

Group	Fields and defaults
Architecture (9)	<code>n_layers</code> 6, <code>width</code> 128, <code>latent_in</code> [2] (skip connection), <code>weight_norm</code> on, <code>norm_layers</code> all, <code>dropout_layers</code> all, <code>dropout_prob</code> 0.2, <code>xyz_in_all</code> off, <code>use_tanh</code> off
Latent codes (5)	<code>latent_dim</code> 1, <code>code_init_std</code> 1.0, <code>code_bound</code> 1.0, <code>code_reg_lambda</code> 1e-4, <code>latent_dropout</code> off
Loss (2)	<code>loss_function</code> clampedL1, <code>clamping_distance</code> 0.1

Learning rate (12) six per parameter group (decoder, codes): type **Step**, initial $5e-4 / 1e-3$, interval 500, factor 0.5, **Warmup** target, length 100

Sampling (2) **samples_per_scene** 8000 (half inside, half outside), **scenes_per_batch** 10

Budget (6) **num_epochs** 200, **seed** 42, **log_frequency** auto, **snapshot_frequency** auto, **additional_snapshots** [1], **description**

The defaults copy the shipped **RoundCross** and **ChiAndCross** decoders on a smaller CPU budget: 8000 samples per shape instead of 16 000, and 200 epochs. So with the defaults the 500-epoch learning-rate step never fires. And **code_init_std** 1.0 gives each code component a variance of $1/d$, while the paper uses 0.01; the runs trained here set $\sigma = \sqrt{0.01 \cdot d}$, except one early $d = 2$ run (**plate_hole_2d_n134_d2_20260928_1338**, $\sigma = 0.1$). The Train tab card has spinboxes for four fields: **latent_dim**, **n_layers**, **width**, **num_epochs**. The rest is edited in the *All hyperparameters...* window.

6.2.1 Validation: error, warning, note

validate takes a set, the dataset's shape count and its geometry dimension, and returns its findings as **Issue** objects, worst first (**structsept/app/hyperparams.py:862**). It runs while you type in the window, when you press *Train*, inside **write_specs**, and in **unattended.prepare_run** before a script writes a run. It does not run for the two **--like** scripts: **prepare_run_like** copies another run's **specs.json** key for key and checks only that the recipe is unchanged and the geometry dimension matches, not that the recipe suits the new dataset.

Level	Meaning	Real examples
error	The library crashes, or the run ends with nothing to load. Blocks <i>Apply</i> , <i>Train</i> and write_specs .	skip connection at layer 0 or at n_layers + 1 ; a skip with width $\leq d$ + geometry dimension; odd samples_per_scene ; log_frequency longer than the run
warning	Legal, but almost certainly not meant.	batch larger than the dataset; epochs not a multiple of the save interval, so latest.pth holds an earlier epoch
note	An effect that is easy to misread.	"Decoder learning rate steps every 500 epochs, so it stays at 0.0005 for the whole 200-epoch run"; clampedL1 with $\delta > 0.1$; "2 of the 134 shapes sit out every epoch"

The errors are not guesses: **tests/test_app_hyperparams.py** builds a working **DeepSDFDecoder** from a list of sets the validator accepts, checks over a grid of architectures that every decoder the library rejects is refused, and shows that an odd sample count really crashes the trainer.

to_specs (**structsept/app/hyperparams.py:1118**) turns a set into the **specs.json** dict. It adds what is not a hyperparameter: **NetworkArch** **deep_sdf_decoder**, **geom_dimension** (always taken from the dataset, never typed), absolute **DataSource** and **TrainSplit** paths, and **DataLoaderThreads** 0. It resolves **auto**: **LogFrequency** is the first of 10, 5, 2, 1 that divides **NumEpochs**, and **SnapshotFrequency** is **NumEpochs // 4**. **from_specs** (**structsept/app/hyperparams.py:1193**) reads a run's or a shipped decoder's specs back key by key and adds a note for every value it drops. It does not carry over **Description** or **geom_dimension**. What cannot be changed at all is listed in **FIXED** (**structsept/app/hyperparams.py:624**) and shown read-only in the window: the architecture, the geometry dimension (from the dataset), Adam, gradient clipping and the regularisation ramp (both hardcoded in the trainer), the CPU, and **DataLoaderThreads** 0 (a loader process re-imports the whole app on Windows).

6.3 The supervisor's sheet

The folder **docs/hyperparameters/** holds the supervisor's Excel template for reporting training settings (one hyperparameter per row, the value in column B) and one filled copy for each of the three 8×256 "4h" runs (**r_only_d1**, **r_only_d2**, **xyr_d3**). The window's *Import sheet...* button reads such a file. **xlsx.read_rows** (**structsept/app/xlsx.py:30**) parses the workbook with the standard library (zip and XML), because **openpyxl** is not worth adding to a slow-resolving environment for one column of values. **from_sheet** (**structsept/app/hyperparams.py:1426**) maps ten rows directly (geometries per batch, latent dimension, hidden layers, neurons per layer, dropout, epochs, both learning rates, clamp value,

initial latent regularization; the trainer ramps that weight up over 100 epochs, while the paper's Eq. 9 prints a decay, Chapter 10). Three rows need arithmetic, because the sheet counts differently from the trainer. With the sheet of `plate2d_r_only_d1_8x256_4h`:

- **Points per training step** counts the whole batch. Samples per shape = $16\,384 / 4$ geometries per batch = 4096, rounded down to an even number. The paper's 16 000 (and the shipped decoders' `SamplesPerScene`) count per shape, so the two numbers are not comparable (open question, Chapter 10).
- **Latent initialization variance** is per component, and the trainer draws from $\mathcal{N}(0, \sigma^2/d)$, so $\sigma = \sqrt{0.01 \cdot 1} = 0.1$.
- **Decay factor** 0.5 and **interval** 150 go into both schedules, set to `Step`. (The `xyr_d3` sheet has 75 instead; see the box below.)

Rows about the data (number of geometries, samples per geometry, sampling strategy) and rows the app cannot change (activation, optimizer, mean) are checked and reported, never applied.

Watch out

- The template has no row for the skip connection, so an imported set keeps `latent_in [2]`; the 8×256 runs used layer 4. To reproduce a run exactly, use *Start from* > *run*: ... in the window, which reads the run's `specs.json`.
- The `xyr_d3` sheet says 600 epochs with the decay every 75; the run it documents trained 300 epochs with the same decay. Chapter 10 lists this, and the test it breaks, under known issues.

6.4 What pressing *Train* does

The Train tab keeps the full set in its state dict (`st["tr_hparams"]`; the dict is explained in Chapter 7) and lays the four card spinboxes over it. Chapter 7 explains the worker thread and the queue it relies on. Pressing *Train* calls `_start_training` (`structsept/app/tab_train.py:692`):

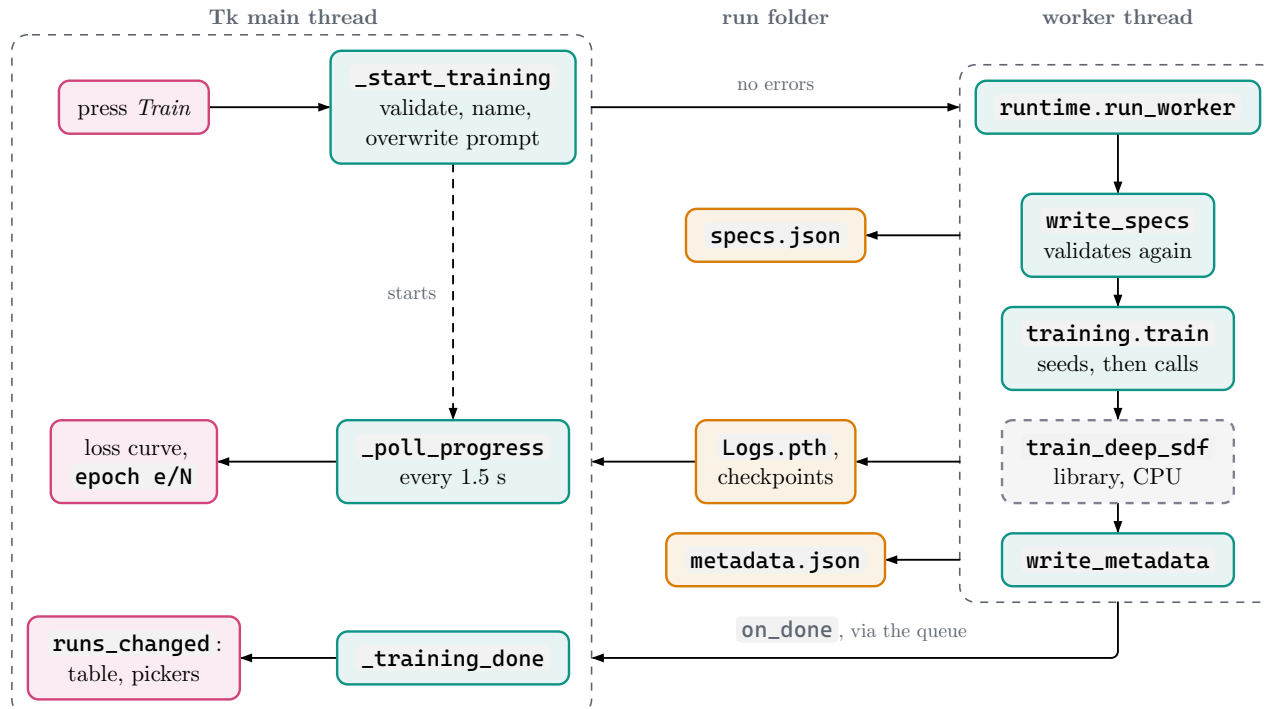


Figure 11: Pressing *Train*. The worker is inside the library trainer for the whole run, so the Tk thread reads progress back from `Logs.pth`, which the trainer rewrites every `LogFrequency` epochs.

1. **Check.** The dataset needs a split file. The set is validated against the dataset's shape count and geometry dimension (`datasets.geom_dimension` reads the `dataset.json` manifest or the width of the npz rows). On errors a dialog lists them and nothing is written.

2. **Name.** An empty name becomes `<dataset>_d<d>_<YYYYmmdd_HHMM>`. An existing `specs.json` under that name triggers an overwrite prompt.
3. **Worker.** `runtime.run_worker` starts a daemon thread that makes three calls. `write_specs` validates once more and writes the file; the non-default values and all issues go to the log. The worker calls `training.train` inside `runtime.signals_off()`, because the trainer installs a Ctrl-C handler and that raises off the main thread. `write_metadata` records dataset, epochs and final loss. The three are at lines 28, 129 and 185 of `structsept/app/training.py`.
4. **Progress.** The trainer reports progress only through a tqdm bar, and the worker is stuck inside it. So `_poll_progress` (`structsept/app/tab_train.py:770`) loads `Logs.pth` every 1.5 s; the curve moves in steps of `LogFrequency` epochs. It follows only the run this window started; a headless run is followed in its log file. In the committed code a run that stopped early also looks finished in the runs table (the `80/800` display is uncommitted, see the box under Unattended training).
5. **Done.** `_training_done` calls `run_editor.runs_changed(st)`, which refreshes the runs table and both explorer pickers.

`training.train` also fixes a library bug: it seeds Python, NumPy and torch *before* it calls the trainer. The library builds the decoder, which draws its initial weights from the global torch RNG, a few lines before it seeds (lines 409 and 415 of `deep_sdf/training.py`). In a long-lived GUI process that RNG has been advanced by every earlier run and every decoder the Explore tab built. Without the fix, two runs with identical settings start from different weights, and an A/B test of one hyperparameter silently compares two initialisations too. The fix is `_seed_everything` (`structsept/app/training.py:172`); `test_same_seed_gives_the_same_network` pins it. Any new code should therefore call `training.train`, never `train_deep_sdf` directly.

Watch out Overwriting a run name rewrites only `specs.json`. The trainer deletes nothing: old snapshots, `Logs.pth` and `metadata.json` stay until replaced, and if the new run crashes the table still shows the old final loss. Use a new name.

6.5 Anatomy of a run folder

`runs/plate2d_r_only_d1_8x256_4h` is a complete run: 40 plates with a centred hole of radius 0.07 to 0.45, $d = 1$, trained by `experiments/train_plate_r_only_4h.py`. Appendix B lists every key of every file.

File	Written by	In this run
<code>specs.json</code>	<code>write_specs</code>	8×256, skip at 4, dropout 0.2, <code>geom_dimension</code> 2, $d = 1$, 600 epochs, <code>Step</code> every 150, 4096 samples per shape, 4 shapes per batch, $\sigma = 0.1$; absolute data paths
<code>metadata.json</code>	<code>write_metadata</code>	dataset, d , dims, epochs, <code>final_loss</code> 0.00431; from the script also <code>preset</code> , <code>stopped_at_deadline</code> , <code>last_epoch</code> 600, <code>wall_seconds</code> 5764
<code>training_summary.json</code>	trainer	only after the last epoch: loss, duration 1:35:59, host, device
<code>Logs.pth</code> , <code>Logs.png</code>	trainer	per-batch loss (6000 = 600 epochs × 10 steps), learning rates per epoch, timings, code magnitude; the PNG at snapshots
<code>ModelParameters/</code>	trainer	decoder weights θ at epochs 1, 100, ..., 600 and <code>latest.pth</code> (1.86 MB each)
<code>LatentCodes/</code>	trainer	the 40 × 1 codes at the same epochs, and <code>latent_code_data_map.json</code> (code index → npz file)
<code>OptimizerParameters/</code>	trainer	Adam state (3.7 MB each); only for resuming, which nothing here does
<code>code_vs_r.csv</code> , <code>.png</code>	the script	code +0.216 at $r = 0.07$ falling monotonically to −0.173 at $r = 0.45$

Two rules follow from how the trainer saves. `latest.pth`, the file the explorers load, is written only when the epoch is a multiple of `LogFrequency`; numbered snapshots come from `SnapshotFrequency` and `AdditionalSnapshots`. And the latent index is the order of the split file: code i belongs to the i -th name

in `data/splits/<dataset>.json`. The absolute paths in `specs.json` and `latent_code_data_map.json` tie a run folder to the machine it was trained on.

6.6 Presets

`experiments/make_plate_presets.py` writes four untrained run folders, one per dataset variant (`r_only`, `xyr`, `tri_h`, `tri_hw`), each validated against its dataset. They share one recipe, `COMMON` (`experiments/make_plate_presets.py:56`): 4×64 ReLU, skip at layer 2, no dropout, 1500 epochs, 4096 samples per shape, 5 shapes per batch, clampedL1 at 0.1, seed 42, and the default learning rates (halved at epochs 500 and 1000). Only d and $\sigma = \sqrt{0.01 \cdot d}$ differ: `preset_plate2d_r_only_d1`, `_xyr_d3`, `_tri_h_d1`, `_tri_hw_d2`. In the GUI you load a preset with *Start from > run: ...* or with *Load hyperparameters* on the runs table, then pick the dataset and train. Headless, `train_plate_tri.py` copies a preset's `specs.json` as its recipe (`--like`). The specs hold absolute paths and `runs/` is gitignored, so regenerate the presets on every new machine once the datasets exist:

```
uv run python experiments/make_plate_presets.py
```

The two other presets in `runs/` (`preset_plate2d_30min`, `preset_plate2d_sheet_quick`) are older and not written by the script.

6.7 Unattended training

A long run takes hours on this laptop's CPU (torch is installed CPU-only). The GUI is the wrong tool: closing it kills the run, Windows may go to sleep, and nobody is there to start the next one. The scripts `experiments/train_plate_*.py` use the same `training.train` (CPU, same seeding) and share their machinery in `structsept/app/unattended.py`. All four follow one lifecycle:

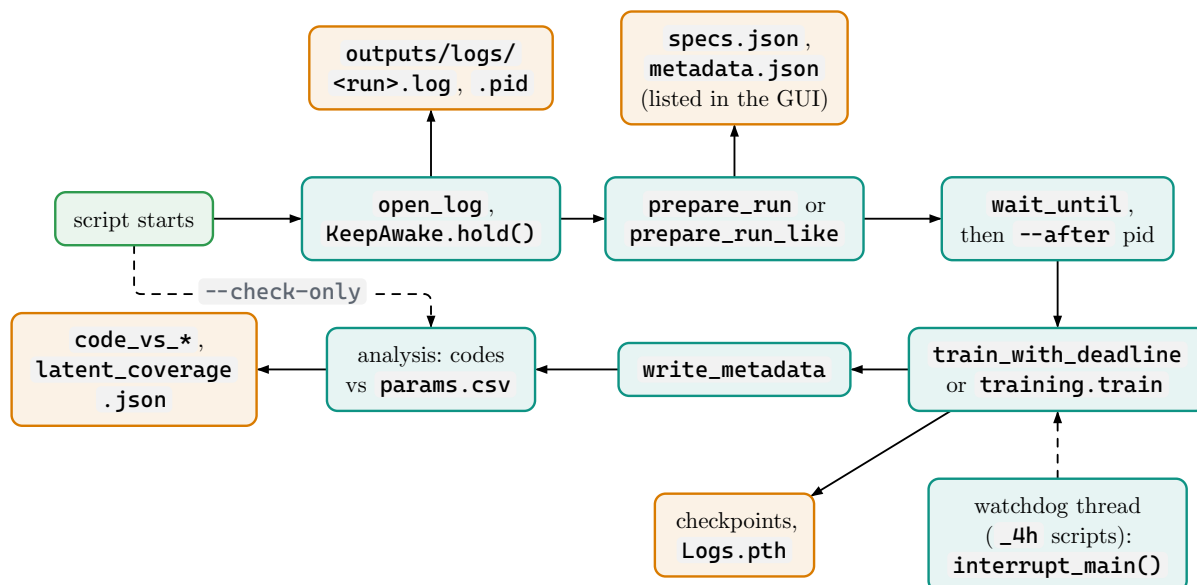


Figure 12: Lifecycle of an unattended run. The run folder is written first, so the GUI lists the run while it waits. `--check-only` skips straight to the analysis.

1. **Before any import** each script sets `TQDM_DISABLE=1` and `MPLBACKEND=Agg`: `tqdm` reads its switch at import time, and the trainer plots with `pyplot`.
2. **Log and keep awake.** `open_log` (`structsept/app/unattended.py:73`) tees stdout and stderr into `outputs/logs/<run>.log` and writes the process id to `<run>.pid` beside it. `KeepAwake.hold()` calls `SetThreadExecutionState`, a per-process request that changes no power setting.
3. **Write the run.** `prepare_run` (`structsept/app/unattended.py:247`) takes `hyperparams.defaults()` updated with the script's `HPARAMS`, clamps the log and snapshot intervals to the epoch count (so a 2-epoch smoke test still ends with a `latest.pth`), validates against the dataset and writes both JSON files. If the dataset is missing it prints the `datagen` command that builds it. `prepare_run_like`

(`structsept/app/unattended.py:422`) instead copies another run's `specs.json` key for key. Only `DataSource`, `TrainSplit`, `TestSplit` and `Description` may differ, otherwise it exits: “same recipe” is checked, not promised. Both refuse a folder that already holds `ModelParameters/latest.pth` unless `--force` is given.

4. **Wait.** `--start-at` takes `now`, `HH:MM` or `YYYY-MM-DD HH:MM`. `--after` takes a pid or a `.pid` file and polls every 30 s until that process is gone.
5. **Train.** In the `_4h` scripts `train_with_deadline` (`structsept/app/unattended.py:319`) starts a watchdog thread. At `--max-hours` it calls `_thread.interrupt_main()`, which fires the library's own Ctrl-C handler (`sys.exit(0)`). The loop stops, and `latest.pth` (at most `LogFrequency` epochs old) and the snapshots remain. The other two scripts train without a limit.
6. **Record and analyse.** `write_metadata` adds `last_epoch`, `wall_seconds` and, in the `_4h` scripts, `stopped_at_deadline`. Then the codes are compared with the dataset's `params.csv`, or, in `train_plate_xyr_n166.py`, measured for latent coverage (next section). An analysis error is only logged.

Script and dataset	Recipe	Measured
<code>train_plate_r_only_4h.py</code> <code>plate_hole_2d_r_only</code> (40)	HPARAMS in the script: 8×256 , skip at 4, dropout 0.2, 600 epochs, LR halved every 150, 4 shapes per batch; $d = 1$ (<code>--latent-dim</code>)	$d = 1$: 1.60 h, code vs r Pearson -1.000 . $d = 2$: 1.95 h, 84.8 % of the variance on one axis
<code>train_plate_xyr_4h.py</code> <code>plate_hole_2d_xyr</code> (134)	same, but 300 epochs and LR halved every <code>epochs // 4 = 75</code> (<code>--decay-interval</code>); $d = 3$	2.79 h; affine R^2 of (x_c, y_c, r) : 0.988, 0.741, 0.857
<code>train_plate_tri.py --family h hw</code> <code>plate_tri_2d_h</code> (40), <code>_hw</code> (133)	the family preset's <code>specs.json</code> (<code>--like</code>): 4×64 , 1500 epochs; $d = 1$ or 2	h : 20.6 min, Pearson -1.000 . hw : 49.9 min, linear R^2 of h 1.000, of w 0.996
<code>train_plate_xyr_n166.py</code> <code>plate_hole_2d_xyr_n166</code> (166)	<code>plate_hole_2d_xyr_d3_ep800</code> copied (<code>--like</code>): 8×256 , 800 epochs, LR halved every 75; $d = 3$	died at epoch 80 of 800 after 83 min (62 s per epoch: plan 14 h); no result

All four take `--start-at`, `--after`, `--run`, `--epochs`, `--force`, `--data-root`, `--runs-dir` and `--log-dir`; only the `_4h` scripts take `--max-hours` (default 4). `--check-only` re-runs just the analysis on an existing run; `train_plate_r_only_4h.py` lacks it. In the two `--like` scripts `--epochs` is a smoke-test override that also skips the recipe check; in the `_4h` scripts it sets the real epoch count.

```
# 1. smoke-test first, into a throwaway run (delete runs/smoke afterwards)
uv run python experiments/train_plate_tri.py --family hw --epochs 2 --run smoke
# 2. the real run: writes its specs now, trains at 23:30
uv run python experiments/train_plate_tri.py --family hw --start-at 23:30
# 3. queue another run behind the first one's process
uv run python experiments/train_plate_xyr_4h.py --after outputs/logs/<run>.pid
# 4. only the analysis, on a run that exists
uv run python experiments/train_plate_tri.py --family hw --check-only
# 5. stop a run; its pid is in outputs/logs/<run>.pid
taskkill /PID <pid> /F
```

The long runs recorded in `experiments/IDEIAS.md` were launched hidden, with PowerShell's `Start-Process -WindowStyle Hidden`, so closing the terminal does not kill them. Follow a run in its `.log` file or in the GUI (press *Refresh*: run lists update only on the app's own events). Chain long runs with `--after` instead of running them side by side: torch uses every core, and two runs at once slow each other down.

Watch out

- Give a smoke test `--run smoke`. Without it, `--epochs 2` writes into the real run's folder, and the real launch then refuses to start without `--force`.
- `--force` retraines **from scratch**; there is no resume. `training.train` never passes the library's `continue_from`, and that path would not help anyway: it restores the decoder and Adam but drops the saved codes (see the trap table).

- A hard kill or a crash leaves `metadata.json` as `prepare_run` wrote it (`final_loss` null, no `last_epoch`) and no `training_summary.json`, yet a loadable `latest.pth`. Check the epoch it holds with `unattended.last_epoch`. `runs/plate_hole_2d_xyr_n166_d3_ep800` is such a run: its checkpoint holds epoch 80.

In progress `unattended.py` (with `prepare_run_like` and `code_vs_parameter`), `make_plate_presets.py` (the two triangle presets) and `train_plate_r_only_4h.py` are modified and uncommitted; `train_plate_tri.py` and `train_plate_xyr_n166.py` are untracked. Edits from 2 October, also uncommitted, make the runs table show a run that stopped short as `80/800` (`training.checkpoint_epoch`) and make `from_sheet` note the missing skip-connection row.

6.8 Analysing a run

A low final loss says the decoder reproduces its training shapes. It says nothing about the question this project asks of a run (Chapter 10): did the latent space learn the parameters that generated the shapes? Three functions answer that. All of them match codes to the rows of `params.csv` by shape name in split order (`unattended.split_names`), because the latent index is the split order.

Function	What it computes and writes
<code>code_vs_parameter</code> <code>structsept/app/unattended.py:525</code>	For one parameter column: per component Pearson, Spearman and whether the code is monotonic. For $d > 1$ also the variance share on the first principal axis (PC1), the correlation along it, and the R^2 of a linear fit (with intercept) of the parameter on all components. Writes <code>code_vs_<p>.csv</code> and <code>.png</code> .
<code>compare_codes_with_params</code> <code>experiments/train_plate_xyr_4h.py:261</code>	Correlation tables code $\times$ parameter, an affine fit of (x_c, y_c, r) on the codes (R^2 , RMSE, coefficients) and the reverse fit, and the variance share of every principal axis. Writes <code>code_vs_params.csv</code> , <code>.json</code> , <code>.png</code> .
<code>latent_coverage</code> <code>experiments/train_plate_xyr_n166.py:160</code>	Per latent axis, the widest gap between neighbouring trained codes as a share of the axis range, against <code>viz.GAP_FRACTION = 0.12</code> , the share at which the GUI shades a gap as untrained. Writes <code>latent_coverage.json</code> .

Judge with an affine fit over all components, never component by component. The loss does not care how the latent axes are oriented: the decoder's first layer is linear in λ and absorbs a rotation, and the regulariser $\|\lambda\|$ is the same in any rotated basis. So no single component has to track a parameter. In `plate2d_r_only_d2_8x256_4h` the two components correlate with r at $+0.904$ and -0.933 and neither is monotonic, yet 84.8 % of the variance lies on one axis and the position along it has Spearman $+1.000$ with r . In `plate_tri_2d_hw_d2_4x64` the components correlate with h at only 0.675 and 0.744, but the linear fit recovers h with $R^2 = 1.000$ and w with 0.996.

The range of the trained codes matters later too: it is the box the design variables may move in. For `plate2d_r_only_d1_8x256_4h` it is -0.17 to $+0.22$, not the $[0.15, 0.75]$ the library's tests use for the shipped decoders.

6.9 Trainer traps

`DeepSDFstruct/DeepSDFstruct/deep_sdf/training.py:299` is where `train_deep_sdf` starts. Reading it line by line (23 September) turned up the traps below. Each row was checked against the library code, with lines relative to `DeepSDFstruct/DeepSDFstruct/deep_sdf/`; the right column says what this repository does about it.

Trap	Library line	What the repo does
------	--------------	--------------------

The decoder is built before the seed is set, so its initial weights depend on whatever used the torch RNG before.	<code>training.py</code> 409, 415	<code>training.train</code> seeds first.
Gradient clipping is hardcoded to max-norm 1.0; <code>GradientClipNorm</code> is read, then overwritten.	<code>training.py</code> 341, 637	not offered; listed in <code>FIXED</code>
<code>clampedL1</code> clamps at its own ± 0.1 , and the trainer clamps both sides at $\pm \delta$: the band learned is $\min(\delta, 0.1)$.	<code>nn_utils.py</code> 18, <code>training.py</code> 581, 607	<code>validate</code> note; use <code>L1</code> for a wider band
Each shape draws <code>SamplesPerScene / 2</code> rows inside and outside; an odd count leaves the batch one sample short of its codes and crashes.	<code>data.py</code> 176, <code>training.py</code> 585	<code>validate</code> error
<code>latest.pth</code> is written only when <code>epoch % LogFrequency == 0</code> ; the default 10 leaves a 5-epoch run with nothing to load.	<code>training.py</code> 684	<code>auto</code> picks 10, 5, 2 or 1; <code>prepare_run</code> clamps it
<code>drop_last=True</code> whenever the batch fits: <code>n_shapes % ScenesPerBatch</code> shapes sit out each epoch (134 at 4: two).	<code>training.py</code> 450	<code>validate</code> note
<code>CodeRegularization</code> (bool) is never read. The penalty is λ_{reg} times the mean code norm (not squared), ramped in over 100 epochs.	<code>training.py</code> 622	ramp listed in <code>FIXED</code> ; the bool is written but inert
A Ctrl-C handler calling <code>sys.exit(0)</code> is installed; installing it off the main thread raises.	<code>training.py</code> 371, 391	GUI: <code>signals_off()</code> ; scripts: the deadline uses it
The tqdm bar writes to stderr, which is <code>None</code> under <code>pythonw</code> .	<code>training.py</code> 555	<code>TQDM_DISABLE</code> ; the launcher redirects stderr (Chapter 7)
<code>training_summary.json</code> is written only after the last epoch, so a stopped run keeps <code>final_loss</code> null.	<code>training.py</code> 696	check <code>last_epoch</code> instead
<code>continue_from</code> reloads decoder and optimizer, but assigns the loaded codes to <code>_</code> : they restart from random under a trained decoder.	<code>training.py</code> 517	not exposed; a dead run is retrained with <code>--force</code>

`experiments/IDEIAS.md` (23 September) marks the hardcoded clipping as worth reporting upstream; the seed order is arguably another candidate. Appendix D collects these traps with the rest of the project's.

Chapter 7

The GUI

In the paper the optimizer never looks at a picture: MMA moves the latent control points $\hat{\lambda}$, the code meshes and simulates, and the loop repeats. A latent value of 0.6 is not a radius or a thickness, though; it is a coordinate in a learned space. The GUI makes it visible. You load a trained decoder f_θ , set the control points by hand and watch the geometry change while you drag. It also puts the offline half of the pipeline (dataset, training) behind buttons. The GUI is **not** part of the optimization loop: the slice and field evaluations run under `torch.no_grad`, and the extracted surface goes to trimesh (NumPy), so the “no `.detach()`” rule of `CLAUDE.md` does not apply here. All code lives in `structsept/app/`.

7.1 Two windows and how to start them

Window	Module	What it is for
Lattice explorer	<code>structsept.app.main</code>	Tabs Explore (a 3-D unit-cell decoder tiled into a lattice), Explore 2-D (planar decoders) and Train (train a decoder, manage <code>runs/</code>).
SDF maker	<code>structsept.app.sdf_maker</code>	A folder of meshes becomes an <code>SdfSamples</code> dataset; also validates datasets.

```
uv run python -m structsept.app.main           # explorer
uv run python -m structsept.app.sdf_maker      # SDF maker
uv run python -m structsept.app.launcher explorer # what the desktop shortcut runs
```

The SDF maker used to be the explorer’s first tab. It became its own window because a dataset is built once in a while, and the explorer assumes that valid datasets already exist. On Windows, `install_shortcuts.bat` puts both windows as shortcuts into `Desktop\NN\` (section on the desktop launcher below).

In progress The screenshot predates the Explore 2-D tab and the *Edit...* button of the decoder picker. The GUI is committed on branch `gui-train-tab` (`6c07cfc`), not yet merged into `main`. Uncommitted at the time of writing: the pickers and the runs table are being taught to flag a run whose checkpoint stops short of its planned epochs (“trained here, epoch 80 of 800” via `models.entry_tag`).

7.2 Architecture: one dict, one queue, no classes

`build_app(root=None)` (`structsept/app/main.py:53`) builds the window without starting `mainloop`, so both `main()` and the desktop launcher can call it. The order of its steps matters:

1. **Agg backend first** (`structsept/app/main.py:31`). `DeepSDFStruct` calls `pyplot` from worker threads (the trainer’s loss curve, the mesher’s debug figures). An interactive backend would then create Tk widgets off the main thread and kill the app. The app’s own plots are hand-built `FigureCanvasTkAgg` canvases, which work with any backend.
2. **Theme before widgets**. `theme.apply_theme` works through `option_add`, which only reaches widgets created after the call.
3. **One shared state dict**. `st = runtime.new_state(root)` starts as `{"root", "queue", "busy"}`. Each tab keeps its widgets, Tk variables and data in it under its own key prefix: `ex_` (Explore), `e2_` (Explore 2-D), `tr_` (Train). The tabs are modules, not classes, and every function takes `st` first. `root.app_state = st` lets tests and a debugger reach all of it.
4. **Tabs**. Each tab module’s `build(st, frame, ...)` fills one `ttk.Notebook` page and registers a refresh callback in `st["run_listeners"]`.

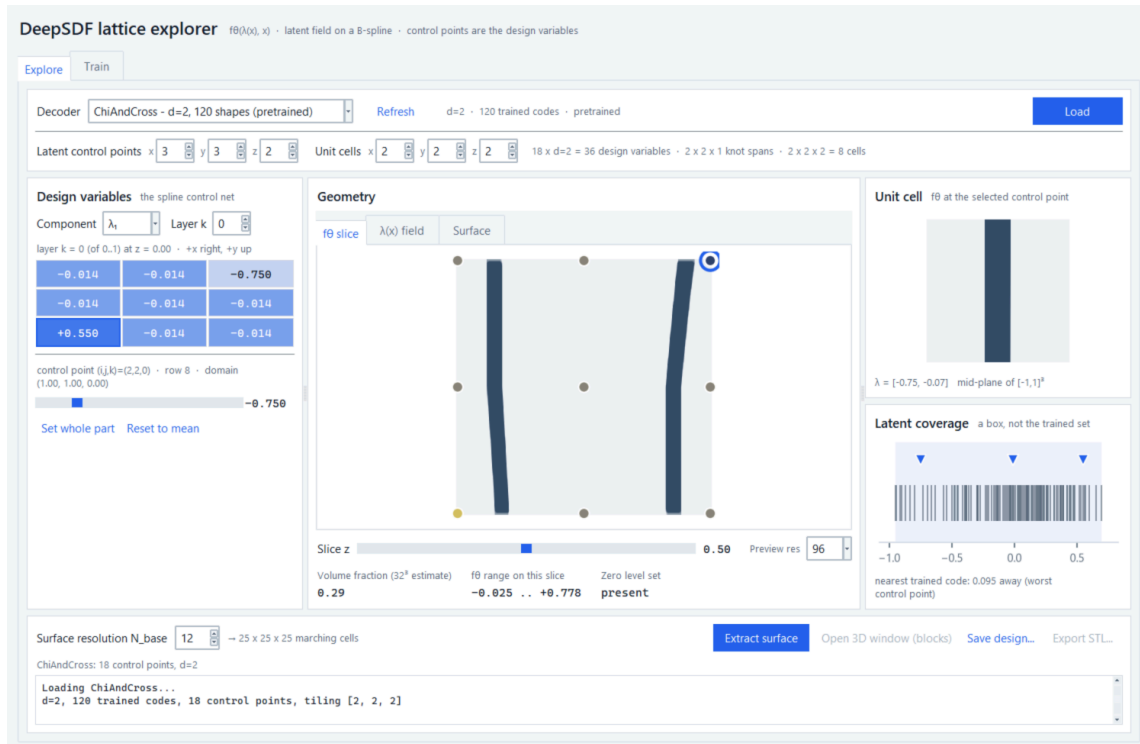


Figure 13: The Explore tab with the shipped **ChiAndCross** decoder ($d = 2$, 120 codes): a $3 \times 3 \times 2$ control net on $2 \times 2 \times 2$ cells, i.e. 36 design variables. Left: one layer of the control net, two values set by hand. Centre: the f_θ slice as material/void, control points as dots. Right: the unit cell of the selected control point and the latent coverage strip.

5. **Pump and shutdown.** `root.after(100, ...)` starts the queue pump. Closing the window runs `_shutdown` (`structsept/app/main.py:124`), which cancels `poll_job` and every pending `after` whose key starts with `ex_job_`, `e2_job_` or `tr_job_`; otherwise Tk fires them into a dead interpreter. A new debounced job only needs a key with one of these prefixes.

7.2.1 The threading model

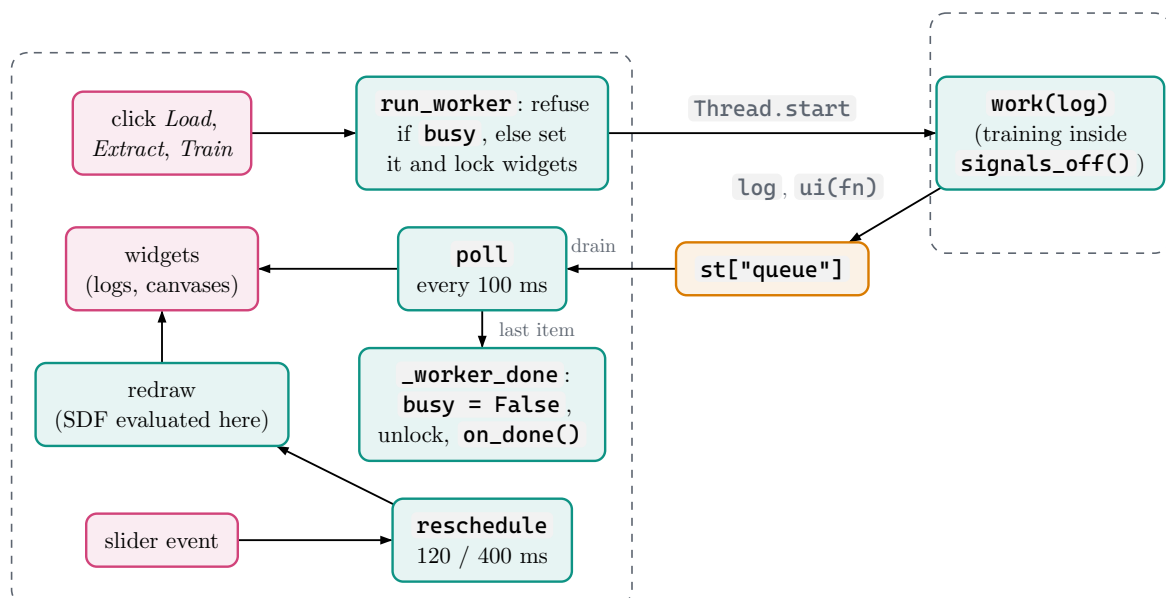


Figure 14: The threading model of `structsept/app/runtime.py`. Left box: the Tk main thread, the only one that touches widgets. Right box: a daemon worker. Slow one-shot jobs run on the worker and reach the interface only through the queue; interactive redraws stay on the Tk thread, throttled by a debounce.

There is one rule: **only the Tk main thread touches widgets**. Tk is not thread-safe, and a widget changed from another thread crashes the app or corrupts it silently. `runtime.py` exists to keep that rule:

- **Workers** run the slow one-shot jobs: loading a decoder, extracting a surface, training, and the SDF maker's *Generate* and *Validate*. `run_worker` (`structsept/app/runtime.py:65`) refuses with a “Busy” dialog if `st["busy"]` is set. Otherwise it sets the flag, disables the button and the widget subtrees passed as `lock`, and runs `work(log)` on a daemon thread. In a `finally`, the worker queues `_worker_done`, which clears `busy`, re-enables the widgets and calls `on_done`: the place where a job releases what it held.
- **The queue is the only way back.** `log(line)` and `runtime.ui(st, fn)` put items on `st["queue"]`. A worker hands over its result as a closure: the Explore loader queues `_install_model`, which then runs on the Tk thread. `poll` (`structsept/app/runtime.py:39`) drains the queue every 100 ms. A callback that raises does not stop the pump; the error is printed as **INTERNAL ERROR** in the Explore tab's log, whichever tab raised it.
- **Redraws are debounced, not threaded.** `reschedule` (`structsept/app/runtime.py:126`) cancels the pending `after` stored under a key and arms a new one, so during a drag only the last event does any work.
- **signals_off()** (`structsept/app/runtime.py:112`) swaps `signal.signal` for a no-op while training runs: the DeepSDF trainer installs a SIGINT handler, and installing one off the main thread raises.

Watch out

- Each window has **one busy** flag. While the Train tab trains (possibly for hours), *Load* in both explorers and *Extract surface* only answer “Busy”. Sliders on a decoder that is already loaded keep working. To explore while training, open a second window.
- The docstrings of `main.py` and `runtime.py` say “the Tk loop only draws”. Not quite: the slice, the latent field, the unit cell and the 32^3 volume grid are evaluated on the Tk thread, the volume grid alone in about 0.2 s. Keep new per-drag work cheap, or put it in the slow redraw tier.

7.3 Explore: one optimizer iteration by hand

The Explore tab runs the online steps of the paper with you in place of MMA. Steps 1–2 (sampling, training) are the Train tab, the SDF maker and **datagen** (Chapter 6, Chapter 5).

Paper step	In an optimization run	In the Explore tab
3 network inputs	spline field $\lambda(\mathbf{x})$ from $\hat{\lambda}$, tiling $T(\mathbf{x})$	<code>models.build_lattice</code> , same classes
4 mesh	FlexiCubes, FFD, tetrahedra	<i>Extract surface</i> : the FlexiCubes surface only
5 FEM	compliance J	absent (see Chapter 8)
6 update	MMA moves $\hat{\lambda}$	you move $\hat{\lambda}$; a tile shows the volume fraction $V(\hat{\lambda})$

7.3.1 What each panel answers

The layout follows the questions you ask while exploring, in that order (the docstring of `tab_explore.py` writes the order down):

Panel	Question it answers	Fed by
Header: picker	Which decoder, and is it usable? Shows d and the number of codes; after <i>Load</i> , warns when all stored codes are identical.	<code>list_models</code>
Header: readout	How many design variables, how many cells? The two are independent: $18 \times d=2 = 36$ design variables $\cdot 2 \times 2 \times 1$ knot spans $\cdot 2 \times 2 \times 2 = 8$ cells.	<code>_update_derived</code>
Left: control net	Which knob moves which region? One $n_x \times n_y$ layer laid out in space, $j = 0$ at the bottom; red outline = outside the trained range.	<code>ControlPointGrid</code>
Centre: f_θ slice	What does the shape look like now? A z -slice of the capped lattice, only the sign of φ , control points as markers.	<code>eval_sdf_slice</code>

Centre: $\lambda(\mathbf{x})$	How does the latent field vary? Up to three components, on a fixed colour scale (the trained range).	<code>eval_latent_slice</code>
Centre: tiles	How much material is there? Does this slice have a zero level set at all?	<code>volume_fraction</code>
Right: unit cell	What does this latent value mean? f_θ at the selected control point on the bare cube $[-1, 1]^3$.	<code>eval_cell_slice</code>
Right: coverage	Is the design still supported by training data? Codes as ticks (a histogram past 400 codes), the widest gap shaded, the distance to the nearest code.	<code>LatentNeighbors</code>

7.3.2 The data model and Load

The design vector is a numpy array `st["ex_cps"]` of shape (n_{cp}, d) with $n_{cp} = n_x n_y n_z$. Control point (i, j, k) is row $i + n_x(j + n_y k)$ (`models.flat_index`, `structsept/app/models.py:307`), because splinepy stores the net with x running fastest; a test checks this against splinepy. The latent spline is clamped and of degree 1, so control point (i, j, k) sits at $(i/(n_x - 1), j/(n_y - 1), k/(n_z - 1))$ in the unit cube, where its marker is drawn. The spinboxes allow 2–6 control points and 1–8 cells per axis.

Changing the decoder, the net or the tiling only shows “settings changed - press Load to apply”. `Load` (`_load_model`, `structsept/app/tab_explore.py:532`) runs on a worker: it loads the weights with `get_model(ref, checkpoint="latest")`, puts the mean trained code on every control point, builds the lattice and runs a small warm-up evaluation (the first one builds lazy kernels and would otherwise freeze the window). The slider spans the trained min/max of the active component plus 10 % of the span on either side. `build_lattice` (`structsept/app/models.py:201`) is paper step 3 almost word for word (classes in Chapter 4; no bounds are passed, so the domain is $[0, 1]^3$):

```
spline = build_parameter_spline([1, 1, 1], [n - 1 for n in n_ctrl], d)
spline.control_points = control_points          # lambda-hat, shape (n_cp, d)
parametrization = SplineParametrization(spline, device=model.device)
lattice = LatticeSDFStruct(
    tiling=tiling,
    microtile=SDFfromDeepSDF(model),
    parametrization=parametrization,
)
sdf = CappedBorderSDF(lattice, None)           # flat, closed faces at the domain border
```

7.3.3 From slider to picture

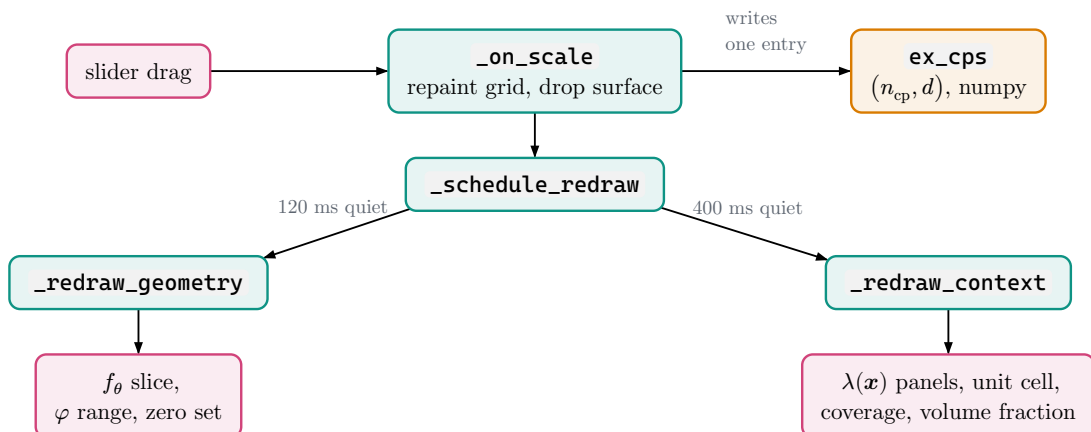


Figure 15: A slider event in the Explore tab: it writes one entry of the design vector, then two debounced tiers redraw. All of it runs on the Tk thread.

`_on_scale` (`structsept/app/tab_explore.py:768`) writes the value into `ex_cps[row, component]`, repaints the grid and drops any extracted surface, which no longer matches the design. `_schedule_redraw` then arms two timers. **120 ms** after the last event, `_redraw_geometry` pushes `ex_cps` into the live torch spline (`set_param`, no rebuild) and evaluates the capped lattice on the slice plane. **400 ms** after it,

`_redraw_context` redraws the latent panels, the unit cell, the coverage strip and the volume fraction, plus the status line (values outside the trained range) and the coverage note (distance from the worst control point to its nearest trained code, amber above 0.15). The geometry is what you cannot predict, so it follows the drag; the context costs more and does not change meaning mid-drag.

Two numbers deserve a note. The **volume fraction** samples 32^3 cell centres, because `CappedBorderSDF` forces $\varphi \geq 0$ on the domain faces and a node grid puts many samples there (27 % at resolution 20). The **nearest-code distance** exists because a per-component min/max box cannot see holes: the 20 codes of `RoundCross` span $[-1, 1]$ with a 0.4-wide empty gap, and their mean, the starting design, falls right into it. The box test passes; the distance shows the problem.

7.3.4 Extract surface, Save design and the shared-SDF lock

Extract surface (`structsept/app/tab_explore.py:1038`) runs `models.surface_mesh` on a worker: `create_3D_mesh` with `mesh_type="surface"` and `differentiate=False`, converted to `trimesh`. This is paper step 4 **without** FFD and **without** tetrahedra, so no FEM and no MMA can follow. The marching resolution is `N_base` $\times$ tiling +1 per axis (`N_base` defaults to 12). *Export STL...* writes `.stl`, `.obj` or `.ply` in unit-cube coordinates; *Open 3D window* opens a pyvista view that blocks the interface until closed.

Save design... (`structsept/app/tab_explore.py:1139`) writes what the tab actually produces: $\hat{\lambda}$ and its grid, as JSON with the keys `decoder`, `source`, `n_ctrl`, `tiling`, `latent_dim`, `control_points` and `control_point_order`. That is the starting design of an optimization run; the STL is only a picture of one evaluation of it.

Watch out Never let two things evaluate the same lattice SDF at once. Each call of `LatticeSDFStruct` writes one latent vector per query point into its microtile, so overlapping calls fail with “Latent vector shape mismatch”. `_extract_surface` therefore sets `st["ex_sdf_busy"]` (both redraw functions return at once while it is set), cancels the pending redraw timers and locks the design and view panels via `run_worker(lock=...)`. When the mesher is done, `_worker_done` unlocks the panels and `_sdf_released` (its `on_done`) clears the flag and re-arms the redraw. Disabling widgets alone is not enough: an already scheduled redraw would still fire. For the same reason the unit-cell preview has its own `SDFfromDeepSDF` wrapper (`models.unit_cell_sdf`) around the same weights.

7.4 Explore 2-D

A planar decoder $f_\theta(\lambda, x, y)$, such as those trained on the `datagen` plate families (Chapter 5), describes one **whole shape** per latent vector. There is no lattice, no control net and no tiling: the design variables are the components of λ itself, one slider each, starting at the mean code with the same range rule (`structsept/app/tab_explore2d.py:345`). *Load* refuses a decoder whose `geom_dimension` is not 2. The field is a node grid over $[-1, 1]^2$ evaluated by `decode_2d` (`structsept/app/models.py:484`), which builds the decoder input by hand (the code repeated for every point, then (x, y)) and calls `model._decoder` directly. The same 120/400 ms tiers drive the material/void field with its area fraction, one coverage strip per component and the nearest-code distance. There is no surface extraction and no *Save design*.

7.5 How a run reaches the explorers

`models.list_models` (`structsept/app/models.py:128`) lists the 7 shipped `PretrainedModels` decoders plus every `runs/<name>/` that holds both `specs.json` and `ModelParameters/latest.pth`, and keeps those whose `geom_dimension` (in the `NetworkSpecs` of `specs.json`, default 3) matches: 3 for Explore, 2 for Explore 2-D. A run without a checkpoint (an untrained preset, a run stopped before its first save) does not appear at all. A run that stopped early (deadline, crash) is listed like a finished one; only the uncommitted `models.entry_tag` marks it (box at the top of this chapter).

The pickers re-read the disk only on the app’s own events: when the Train tab finishes, renames or deletes a run, `run_editor.runs_changed` calls every function in `st["run_listeners"]` (both pickers and the runs

table). Runs written by another process, such as a headless `experiments/train_*.py` script, appear after *Refresh*. Two windows can also collide on the automatic run name `<dataset>_d<d>_<YYYYmmdd_HHMM>`, which has minute resolution: two trainings of one dataset and d started in the same minute get the same folder (the second sees an overwrite prompt only if the first has already written its `specs.json`). Double-clicking a row of the Train tab's runs table (or *Open*) calls `_open_in_explore` (`structsept/app/tab_train.py:486`): it selects the run in whichever explorer lists it and switches tab, but does **not** load it.

The Train tab, including how it trains on a worker and polls `Logs.pth`, is covered in Chapter 6. That poll follows only the run started in this window; a headless run shows its progress only in its log.

Tip Today all 11 loadable runs in `runs/` are planar, so the Explore picker lists only the shipped decoders and every local run opens in Explore 2-D. `Primitives2D` is a 3-D decoder despite its name and appears on Explore.

7.6 The SDF maker

The SDF maker is step 1 of the offline pipeline for arbitrary meshes. You pick a mesh folder, an extension (default `stl`), dataset and class names, the samples per geometry (default 50 000) and whether to add surface samples.

- *Generate dataset* (`structsept/app/sdf_maker.py:171`) reads the meshes and asks before going on if any is not watertight: the SDF sign comes from a winding number, so an open mesh gives a silently wrong dataset. On a worker, `datasets.make_dataset` (`structsept/app/datasets.py:168`) then removes stale files and runs the library's `SDFSampler`, which scales each mesh into $[-1, 1]^3$ and writes `data/SdfSamples/<ds>/<class>/*.npz` and `data/splits/<ds>.json`.
- *Validate* checks an existing dataset for missing inside samples, NaNs, points outside the $[-1, 1]^d$ box (tolerance 0.2), an unnormalized φ , a skewed inside/outside ratio and files that mix 2-D and 3-D rows, and plots a φ histogram.

It uses the same `runtime` plumbing with its own `st` and Tk root. Every dataset in `data/` today was made by `datagen`, not by the SDF maker (Chapter 5); formats are in Appendix B.

7.7 The desktop launcher

A shortcut runs `.venv\Scripts\pythonw.exe -m structsept.app.launcher explorer` (or `sdf_maker`) in the repository root. `structsept/app/launcher.py` exists only so the app can be double-clicked, and three facts shape it:

- **pythonw has no console:** `sys.stdout` and `sys.stderr` are `None`, but `tqdm` and the mesh loader write to `stderr`, so the Train tab and the SDF maker would crash as soon as they start. `attach_log` (`structsept/app/launcher.py:158`) sends both streams to `outputs/logs/<target>.log` (append mode, rotated past 2 MB), sets `TQDM_DISABLE=1` and points `faulthandler` there, all **before** `DeepSDFStruct` is imported, because a `logging.StreamHandler` captures `sys.stderr` when created.
- **The imports take 15–30 s**, and a shortcut that shows nothing gets clicked again. `run` (`structsept/app/launcher.py:529`) shows a splash within a second and imports the app on a worker thread that reports through a queue polled every 100 ms (the app's own pattern), then calls `build_app(root=root)` to build the window into the root the splash already owns.
- **Shortcuts hold absolute paths**, and editable installs keep only the code current. `env_hash` (`structsept/app/launcher.py:185`) hashes `pyproject.toml`, `uv.lock` and `DeepSDFStruct/pyproject.toml`; if the hash differs from `.venv/.structsept-env-hash`, the launcher runs `uv sync`. It skips the sync while another `structsept` window is open, because Windows cannot replace a loaded `.pyd` and a half-finished sync breaks the environment. `InstanceLock` (`structsept/app/launcher.py:292`) detects open windows: each holds an OS lock on one of 16 bytes of `.venv/.structsept-instances`.

`install_shortcuts.bat` sets this up on a fresh clone: it checks for `uv`, fetches the submodule if it is empty, runs `uv sync` and then `launcher --install`, which writes the `.lnk` files through PowerShell's `WScript.Shell` (no `pywin32` needed) and records the hash. Run it again after moving the folder or re-creating `.venv`; the full setup is in Chapter 9.

Watch out A window started from a shortcut shows its output nowhere: tracebacks and the trainer output of GUI training runs go to `outputs/logs/explorerer.log`. A terminal launch redirects nothing. If a dialog at start-up reports an `ImportError`, close every structsept window and run `install_shortcuts.bat` (or `uv sync`).

7.8 Known limits and tests

- `Primitives`, `PrimitivesCL16` and `PrimitivesCL08` store all-zero latent codes: the slider range falls back to ± 1 and the coverage panel says nothing.
- The SDF maker reads the mesh folder on the Tk thread and freezes on large folders.
- The 3-D preview is static, Explore 2-D has no surface or export, and nothing in the repository reads a saved design JSON yet.

The GUI tests drive the real widgets. `_hide` (`tests/test_app_explore.py:26`) makes the window transparent and frameless and parks it at $(-6000, -6000)$ instead of withdrawing it, because a withdrawn window has no device context and the matplotlib canvases would never get a size. The tests need a display, but nothing appears on it.

File	What it covers
<code>test_app_explore.py</code>	One test per defect that shipped once, on <code>ChiAndCross</code> with a $4 \times 2 \times 3$ net: flat index vs splinepy, layer clamp, surface dropped after an edit, mesher holding the SDF alone, unbiased volume fraction.
<code>test_app_explore2d.py</code>	Planar path end to end: a 6-shape <code>datagen</code> set, a 2-epoch run through the Train tab's code, the run landing on Explore 2-D.
<code>test_app_runs.py</code>	Rename, notes, delete, the lock on the run being trained.
<code>test_app_launcher.py</code>	22 tests; every launch scenario runs in a fresh interpreter, one under a real <code>pythonw.exe</code> without standard handles.

```
uv run pytest tests/test_app_explore.py tests/test_app_explore2d.py -v
uv run pytest tests/test_app_runs.py tests/test_app_launcher.py -v
```

Tip Four rules when you change the app. (1) Never touch a widget from a worker; queue a closure with `runtime.ui`. (2) Keep state under your tab's prefix and name debounced jobs `<prefix>_job_<name>`, so that `_shutdown` cancels them. (3) Code that evaluates the lattice SDF must respect `ex_sdf_busy`. (4) Keep torch out of `viz.py` (field evaluation belongs in `models.py`); it imports no torch and is testable without the heavy stack.

Chapter 8

From geometry to stiffness

Before an optimization loop can exist, three things are needed: a **test geometry** to try ideas on, a way to turn it into a **finite element model**, and (the supervisor asked for this on 13/09) a way to bring in a **real part given as a point cloud**. The project’s own library `structsept/` has one small module for each:

Module	What it does	Driven by
<code>plate_with_hole.py</code>	The recurring test geometry, a lattice plate with a circular hole, plus <code>ScaledSpaceSDF</code> , the bridge between metres and the parametric cube.	stiffness and network scripts
<code>fem.py</code>	Watertight SDF $\rightarrow$ tetrahedra $\rightarrow$ torch-fem <code>Solid</code> $\rightarrow$ global stiffness matrix K , plus sanity checks. Writes no files, parses no arguments.	stiffness script
<code>pointcloud_sdf.py</code>	Signed distance field of an oriented point cloud, used as ground truth when a latent field is fitted to a real part.	<code>pointcloud_to_lattice.py</code>

The scripts that drive them live in `experiments/` and follow the import rule of Chapter 3. Neither the GUI nor `datagen` uses these modules, and no test under `tests/` covers them; the smoke test only covers the `DeepSDFStruct` primitives underneath.

Watch out There are two “plates with a hole”. `datagen` builds plate-with-hole *shapes* as training data for a decoder (Chapter 5). `structsept.plate_with_hole` builds a *lattice* plate with a hole cut through it, as a test case for the FE model. They share a name and nothing else.

8.1 Three coordinate spaces

Most of the confusion in this code comes from mixing up coordinate systems. Keep three apart (Figure 16). The **network cube** $[-1, 1]^3$ is where the decoder $f_\theta(\lambda, \mathbf{x})$ lives; it holds one unit cell. The **parametric cube** $[0, 1]^3$ holds the whole lattice: `LatticeSDFStruct` maps each cell back into the network cube with the mirrored triangle wave $T(\mathbf{x})$ (Chapter 2). Every SDF in this chapter, including the hole and the border caps, is a function on this cube. **Physical metres** appear only when the `TorchSpline` box moves the *mesh vertices*, inside `create_3D_mesh(..., deformation_function=...)`. The SDF never sees a metre.

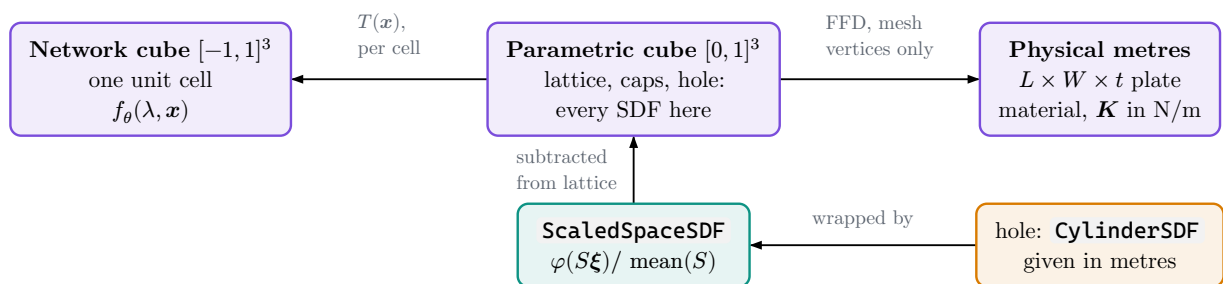


Figure 16: The three spaces. A shape given in metres has to be pulled back into the parametric cube through `ScaledSpaceSDF` before it can meet the lattice.

`ScaledSpaceSDF` (`structsept/plate_with_hole.py:43`) evaluates the wrapped shape at $S\xi$, with $S = \text{diag}(L, W, t)$, and divides the result by the mean of S (0.7 for the default $1 \times 1 \times 0.1$ m plate). The zero level set is exact, and that is all a mesher looks for. Away from the surface the values are only approximate distances on a non-cubic plate, because one scalar cannot undo a stretch that differs per axis. The division keeps them at the right order of magnitude, which is what FlexiCubes needs to place vertices.

Watch out The mesh resolution N (`N_base`, Chapter 2) counts FlexiCubes cubes per unit cell *in parametric space* ($N \cdot \text{tiling} + 1$ cubes per axis, spread over the domain plus a 5 % margin on each side). On the $4 \times 4 \times 1$ plate the 0.1 m thickness gets as many intervals as a 0.25 m cell in the plane, and the solid slab (no tiling) gets 11^3 cubes in total at $N = 10$.

8.2 The test geometry: `plate_with_hole()`

The function (`structsept/plate_with_hole.py:83`) takes the hole radius and centre and the plate size in metres, plus `tiling`, `latent` and `solid`. The defaults are a $1 \times 1 \times 0.1$ m plate with a centred hole of radius 0.25 m, tiling $5 \times 5 \times 1$ and latent 0.4. It returns `(plate, deformation)` and builds the SDF in a fixed order (Figure 17), where each step has a reason:

1. **Microtile:** `get_model(PretrainedModels.AnalyticRoundCross)` in `SDFfromDeepSDF`. This “decoder” ignores its weights: its `forward` computes three perpendicular cylinders of radius $r = \lambda$ in closed form. Zero reconstruction error is what a test case needs: anything wrong in the mesh or in K is then the fault of the mesher or the FE code, not of the network.
2. **Lattice:** `LatticeSDFStruct(tiling, microtile, Constant([latent]))`, so all cells are identical. Swapping `Constant` for `SplineParametrization` is the one change that turns this into a design problem. Nothing on the plate/FE path does it yet (the GUI’s `structsept/app/models.py` builds spline lattices, but has no FE model).
3. **Hole:** `DifferenceSDF` with a `CylinderSDF` from $z = -t$ to $z = 2t$, wrapped in `ScaledSpaceSDF`. The overshoot gives a clean through-hole, not a pocket whose end cap lands inside the material.
4. **Caps:** `CappedBorderSDF(body)` trims all six faces of the unit cube flush, so every strut the border cuts ends in a flat face on the plate surface. Without it the surface would still close (`LatticeSDFStruct` is positive outside its box and the mesher pads the grid by 5 %), but at an uncontrolled place between grid nodes (Chapter 2). Hole and caps both only remove material, so their order does not matter.
5. **Map to metres:** a `TorchSpline` of `splinepy.helpme.create.box(L, W, t)`, returned separately for `create_3D_mesh` and `plot_slice`.

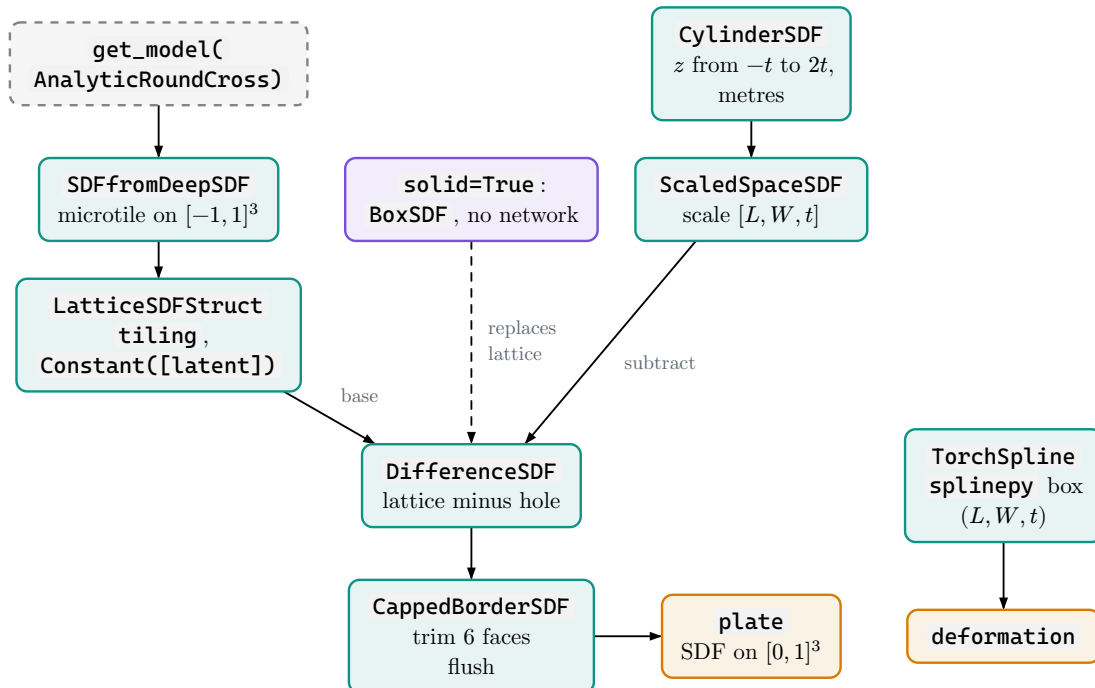


Figure 17: The SDF tree built by `plate_with_hole()`, read top to bottom. The two outputs on the bottom row are returned as a pair.

`solid=True` replaces the lattice with `BoxSDF` and loads no network: same hole, same mesher, same FE code, no lattice. It is the comparison case. `mesh_and_export(plate, deformation, resolution=14)`

(`structsept/plate_with_hole.py:170`) is the geometry-only consumer: it writes two SDF slices, a FlexiCubes surface in metres (VTK and STL) and a render, and logs watertightness, volume and bounding box. The module CLI calls these two functions.

8.3 `fem.py`: from an SDF to K

`structsept/fem.py` is step 5 of the pipeline, cut short right after the global stiffness matrix exists: no supports, no loads, no solve (Figure 18). Without the export, the whole driver is six calls:

```
plate, deformation = plate_with_hole(tiling=(4, 4, 1), device="cpu")
vertices, tets = tetrahedral_mesh(plate, deformation, resolution=10) # float32 side
with default_dtype(torch.float64): # torch-fem side
    solid = build_solid(vertices, tets) # steel: E = 210e9 Pa, nu = 0.3
    k, K = assemble_stiffness(solid) # k: (n_e, 12, 12) dense, K: sparse COO
    check_stiffness(K, solid) # logs the checks, warns if one fails
K_csr = to_scipy(K) # SciPy CSR, for saving or a sparse solve
```

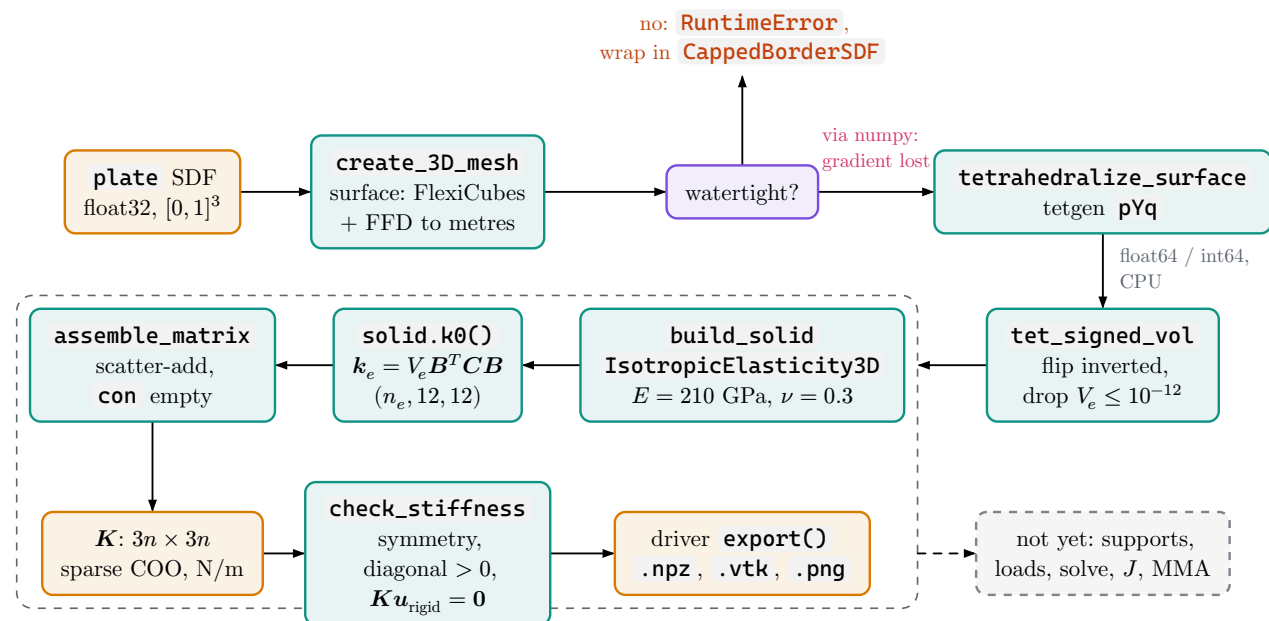


Figure 18: The chain in `structsept/fem.py`, read as a snake from the top left. Everything in the dashed frame runs under `default_dtype(torch.float64)`. The pink label marks where the autograd chain from the latent to K breaks.

Meshing. `tetrahedral_mesh` (`structsept/fem.py:98`) extracts a FlexiCubes surface already in metres and raises a `RuntimeError` if it is not watertight. The library’s `tetrahedralize_surface` then hands it to `tetgen` with the switches `pYq`: the surface triangles are kept as faces and quality refinement adds interior Steiner points. Inverted tets get two nodes swapped and tets of volume $\leq 10^{-12}$ m³ are dropped. Both passes do nothing on today’s `tetgen` output; they guard `torch-fem`, which needs positive orientation (the element Jacobian determinant is $6V_e$) and no zero-volume tets.

Why not the volume path? `create_3D_mesh(mesh_type="volume")`, FlexiCubes’ own tetrahedralization, leaves internal cavities. Measured on a plain unit cube, its tets add up to only 0.68–0.70 of the enclosed volume at every resolution tried, and the lattice plate loses about as much. `tetgen` reproduces the surface volume to every printed digit, so `fem.py` uses it. `CLAUDE.md` still describes `create_3D_mesh` as doing the tetrahedralization; this project’s FE path replaces that step.

Watch out `experiments/plate_geometry.py --volume` writes the cavity mesh while its help text calls it “needed for FEM later”. Do not feed it to an FE model.

`DeepSDFStruct/tests/test_structural_optimization.py` uses the same path, so its compliance values belong to a body with about 30 % voids.

Material and precision. `build_solid` ([structsept/fem.py:176](#)) wraps the nodes and tets in a `torchfem.solid.Solid` with an isotropic linear-elastic material, steel in SI units by default. With the mesh in metres, $\mathbf{K}$ is in N/m. The decoder and FlexiCubes run in float32, but torch-fem builds its tensors from torch’s *default* dtype and loses digits in float32 assembly. Hence `default_dtype` ([structsept/fem.py:48](#)), a context manager around everything from `build_solid` on.

Assembly. `assemble_stiffness` ([structsept/fem.py:197](#)) makes two calls. First `solid.k0()` gives one matrix $\mathbf{k}_e = \mathbf{V}_e \mathbf{B}^T \mathbf{C} \mathbf{B}$ of size 12×12 per tet; one integration point is exact for a linear tet. Then `solid.assemble_matrix(k, con)` scatter-adds them into $\mathbf{K}$. Its argument `con`, the list of constrained DOFs, is empty because nothing is pinned yet, so $\mathbf{K}$ is the free stiffness matrix. That is on purpose.

Checks. Without supports $\mathbf{K}$ **must** be singular: a free body can translate three ways and rotate three ways without straining, so $\mathbf{K}$ has exactly six zero eigenvalues. Never try to invert it. `check_stiffness` ([structsept/fem.py:230](#)) turns this into a test. It logs the relative asymmetry, the smallest diagonal entry, and $\max |\mathbf{K} \mathbf{U}_{\text{rigid}}| / \max |K_{ij}|$, where `rigid_body_modes` ([structsept/fem.py:66](#)) builds the $(3n, 6)$ matrix of three unit translations and three infinitesimal rotations $\boldsymbol{\omega} \times \mathbf{r}$. The rigid-body check is the strongest of the three. A rotation produces zero strain only if every element’s $\mathbf{B}$ is consistent with its node positions, and the zero survives assembly only if every $\mathbf{k}_e$ landed on the right DOFs. A wrong geometry can still give a symmetric $\mathbf{K}$ with a positive diagonal; it cannot pass the rotation test. It says nothing about the material: $\mathbf{B} \mathbf{u}_{\text{rigid}} = \mathbf{0}$ element by element, so $\mathbf{K} \mathbf{u}_{\text{rigid}}$ vanishes for any $\mathbf{C}$. (The docstring at [structsept/fem.py:237](#) and the Portuguese theory note claim the test also exercises $\mathbf{C}$; they are wrong there.)

In the code Thresholds at [structsept/fem.py:260](#): asymmetry or residual above 10^{-10} , or a diagonal entry ≤ 0 , logs the warning `K failed a sanity check -- do not trust it`. It never raises, so read the log.

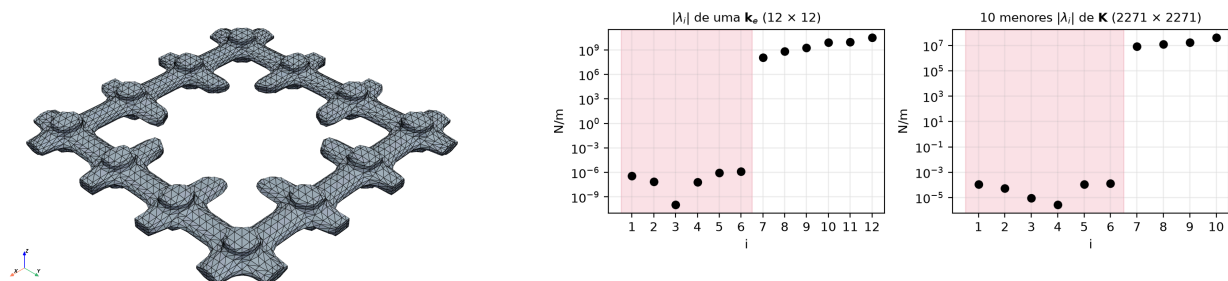


Figure 19: Left: tet mesh of the default run; the hole removes the four central cells of the $4 \times 4 \times 1$ lattice. Right (titles in Portuguese): eigenvalue magnitudes of one $\mathbf{k}_e$ and the ten smallest of the solid plate’s $\mathbf{K}$. The six rigid-body modes (shaded) sit at about 10^{-4} N/m or below, the seventh at 8.2×10^6 N/m.

The two runs below (resolution 10, steel) are documented in the theory PDF. The matrices saved in `experiments/outputs/` and `experiments/outputs/solid/` match in shape and nonzero count. Of the 4,667 lattice nodes, 571 are Steiner points; the tets add up to 0.020241 m^3 , the volume the surface encloses.

Quantity	Lattice plate (default)	Solid plate (<code>--solid</code>)
Surface triangles	8,192	1,456
Nodes / tets	4,667 / 16,015	757 / 2,478
$\mathbf{K}$: size, nonzeros	$14,001^2$, 488,007 (0.25 %)	$2,271^2$, 78,147 (1.5 %)
Asymmetry / rigid residual	2.2×10^{-16} / 3.7×10^{-16}	3.0×10^{-16} / 2.9×10^{-16}
Wall time	about 7 s	about 5 s

8.4 The experiment ladder

Four scripts walk from “a lattice exists” to “ $\mathbf{K}$ exists”. They are a **conceptual progression, not a data pipeline**: no script reads another one’s output. Three of them build the plate themselves; only

`ScaledSpaceSDF` and `plate_with_hole()` are shared. Scripts 2–4 write into `experiments/outputs/`, script 1 into `outputs/` at the repo root.

Script (in <code>experiments/</code>)	What it adds	Writes
1 <code>plate_geometry.py</code>	Lattice plate, no hole. Does not import <code>structsept</code> . Tiling $4 \times 4 \times 1$. <code>--skins</code> adds face sheets; <code>--volume</code> writes the cavity mesh.	<code>plate_slices.png</code> , <code>plate_surface.vtk/.stl</code> , render
2 <code>-m structsept.plate_with_hole</code>	Adds the hole. Tiling $5 \times 5 \times 1$. <code>--solid</code> gives the slab; the hole and plate flags mirror the function arguments.	<code>hole_slices.png</code> , <code>plate_with_hole.vtk/.stl</code> , render
3 <code>plate_with_hole_network.py</code>	Swaps in the <i>trained</i> <code>RoundCross</code> decoder at latent 0.6. Figures of the domain Ω , the cell f_θ and both combined, plus a 5-value latent sweep.	<code>sdf_stages.png</code> , <code>latent_sweep.png</code> , <code>plate_network.stl</code> , render
4 <code>plate_with_hole_stiffness.py</code>	Thin driver: <code>plate_with_hole()</code> , the <code>fem</code> chain, then <code>export()</code> . Tiling $4 \times 4 \times 1$, resolution 10, about 7 s.	<code>stiffness_matrix.npz</code> (CSR), <code>plate_with_hole_tets.vtk</code> , <code>stiffness_sparsity.png</code>

```
uv run python experiments/plate_geometry.py
uv run python -m structsept.plate_with_hole
uv run python experiments/plate_with_hole_network.py
uv run python experiments/plate_with_hole_stiffness.py
# K back in Python: scipy.sparse.load_npz("experiments/outputs/stiffness_matrix.npz")
```

For the slab, add `--solid --outdir experiments/outputs/solid` to step 2 or 4, or it overwrites the lattice results. In VS Code, `.vscode/launch.json` has steps 2 and 4 and the solid slab ready on F5.

Tip Open the `.vtk` / `.stl` outputs in ParaView, or with `pyvista`, which is already in the environment (start `uv run python` from the repo root):

```
import pyvista as pv
pv.read("experiments/outputs/plate_with_hole_tets.vtk").plot()
```

Watch out

- Step 4 computes $\mathbf{K}$ for the **analytic** cell of step 2, not for the trained network of step 3. There is no flag to switch.
- The two cells read the latent differently: at $\lambda = 0.6$ the analytic cell fills 54 % of the unit cube, the trained one 29 %. Compare shapes, not λ .
- Imports work from anywhere, but outputs land relative to the current directory. Run the scripts from the repo root.

8.5 Point cloud in, lattice out

The library already fits a latent field to a *watertight mesh* (`LocalShapesReconstructor`, Chapter 4), taking its signed ground truth from `SDFfromMesh`. A point cloud has no faces, hence no inside and no sign. `PointCloudSDF` (`structsept/pointcloud_sdf.py:216`) fills that gap as a drop-in `SDFBase`:

- **Magnitude:** distance to the nearest point from a SciPy `ckdtree`; within 3 median point spacings, the distance to that point's tangent plane instead. Point-to-point distance is biased outwards by about half a spacing, right at the zero level set.
- **Sign:** the generalized winding number (Barill et al. 2018). Each point is a dipole of strength $a_i \mathbf{n}_i$:

$$w(\mathbf{q}) = \frac{1}{4\pi} \sum_i a_i \frac{(\mathbf{p}_i - \mathbf{q}) \cdot \mathbf{n}_i}{\|\mathbf{p}_i - \mathbf{q}\|^3} \quad (16)$$

is about 1 inside and 0 outside, and $w > 0.5$ means inside. It is a dense $O(n_{\text{query}} \times n_{\text{cloud}})$ sum, computed in chunks of 2048 queries over at most 20,000 dipoles with rescaled areas.

Watch out The cloud **must carry outward normals**, despite `CLAUDE.md` calling it “unoriented”. `estimate_normals` (PCA, pointed away from the centroid) is only right for roughly star-shaped parts. Winding values near 0.5 mean inconsistent normals; do not tune the threshold. `PointCloudSDF` is not differentiable and its KD-tree runs on the CPU: ground truth, computed once, never inside an optimization loop.

`sample_cloud_sdf` (`structsept/pointcloud_sdf.py:395`) constructs the near-surface band instead of querying it: a cloud point moved by t along its normal has signed distance t . The band holds 85 % of the samples with the script’s defaults, so the expensive winding number runs only on the uniform ones.

`experiments/pointcloud_to_lattice.py` runs the whole path (Figure 20). By default it simulates a 20,000-point scan of `DeepSDFStruct/tests/data/cone.stl` and fits a $3 \times 3 \times 3$ lattice of the `Primitives` decoder (latent dimension 32) with a degree-1 latent spline started at the mean trained code. This is **auto-decoder inference, not an encoder**: the decoder stays frozen and Adam moves the control points. Logged result: IoU from 15.4 % to 99.5 % in 170 steps (18 s). The whole run takes 146 s at the default `--resolution 32` and 71 s at 16; the difference is all mesh export. The saved `latent_control_points.npz` holds a $(64, 32)$ array, 4^3 control points of 32 components: exactly the design variables MMA would start from.

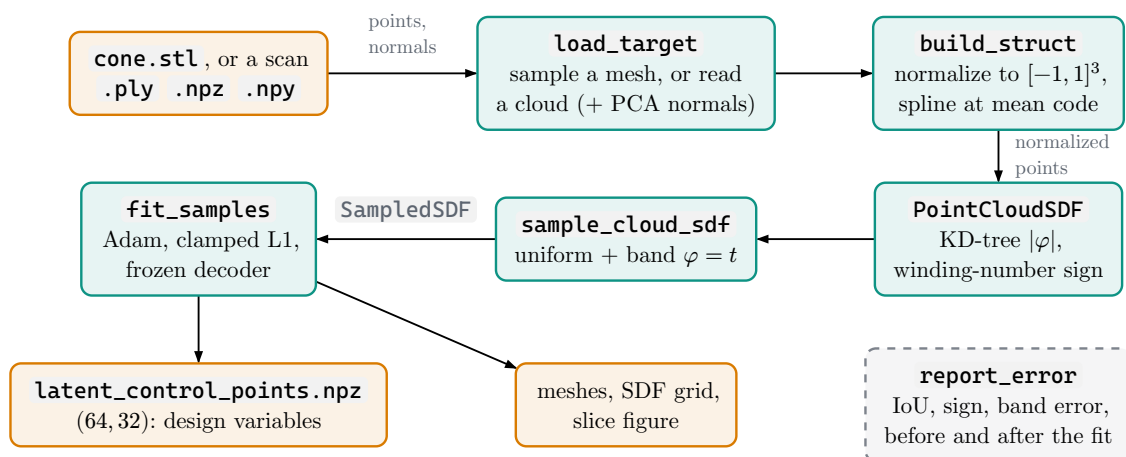


Figure 20: `experiments/pointcloud_to_lattice.py`. Only `PointCloudSDF` and `sample_cloud_sdf` are new; structure, fit and export are the library’s. Outputs go to `experiments/outputs/pointcloud/`.

```

uv run python experiments/pointcloud_to_lattice.py # cone.stl
uv run python experiments/pointcloud_to_lattice.py --noise 0.01 --validate
uv run python experiments/pointcloud_to_lattice.py --input scan.ply # real cloud

```

`report_error` (`experiments/pointcloud_to_lattice.py:151`) scores with occupancy IoU as the headline. Comparing φ deep inside the part is meaningless, because inside a lattice φ is the distance to the nearest strut, not to the part’s boundary; the third number is therefore restricted to the target’s $|\varphi| < 0.05$ band. Note `--output-dir` (not `--outdir`) and `--stds` in normalized $[-1, 1]$ units.

In progress Open (IDEIAS, point-cloud section): if the target part is solid, “imitate the cloud” pushes every cell towards its most solid form, and perhaps the lattice should only fit *inside* the part; this needs the supervisor’s answer. Real clouds without normals need minimum-spanning-tree orientation, not implemented.

8.6 The theory document

`docs/stiffness_theory.pdf` (Portuguese; source `docs/stiffness_theory/stiffness_theory.typ`) walks through the stiffness run function by function with the theory behind each step and the measured

numbers: SDFs and min/max Booleans, the hole and `ScaledSpaceSDF`, DeepSDF and the analytic round cross, tiling and the latent spline, caps, FlexiCubes, FFD, tetgen and tet orientation, linear elasticity and the Voigt matrix $\mathbf{C}$, the weak form, shape functions and $\mathbf{k}_e = \mathbf{V}_e \mathbf{B}^T \mathbf{C} \mathbf{B}$, assembly and sparsity, the three properties of $\mathbf{K}$ with the spectra, and what is still missing. Read it next to `fem.py`. Only three of its seven images are scripted: `make_figures.py` writes `transformation.png`, `sdf_1d.png` and `spectra.png` (plus `numbers_solid.json`) into the directory given as its only argument. The compile command is in the header of the `.typ` file.

```
uv run python docs/stiffness_theory/make_figures.py docs/stiffness_theory/figures
```

8.7 What is missing before an optimization loop

$\mathbf{K}$ exists and passes its checks. The path from there to the paper’s loop (theory in Chapter 2, template in Chapter 4, recipe in Chapter 9):

1. **Supports:** `solid.constraints[mask, :] = True` on one face removes the six rigid-body modes and makes $\mathbf{K}$ invertible.
2. **Loads:** `solid.forces[mask, d] = ...` on the opposite face.
3. **Solve:** `solid.solve(...)` re-assembles with the constraints and solves $\mathbf{K}\mathbf{u} = \mathbf{f}$, the cost bottleneck (in the paper, FEM plus sensitivities take about 1 min 30 s of the 1 min 50 s per iteration).
4. **Objective and constraint:** compliance $J = \mathbf{f}^T \mathbf{u}$ and volume $V = \sum_e V_e$ against a target.
5. **Update:** `MMA` moves the `SplineParametrization` control points within bounds (`[0.15, 0.75]`) for the analytic cell; for a trained decoder the per-component range of its codes, Chapter 4), using $\partial J / \partial \hat{\lambda}$ and $\partial V / \partial \hat{\lambda}$ from autodiff.

A comment at `experiments/plate_with_hole_stiffness.py:167` sketches the first three steps, and `DeepSDFStruct/tests/test_structural_optimization.py` does all five on a cantilever. The catch is the gradient. In `fem.py` the surface goes through `gustaf` and `numpy` into `tetgen` and comes back via `torch.as_tensor`, which cuts the autograd chain from the latent to $\mathbf{K}$. The library test keeps its gradient by staying in `torch`, but on the volume path with its cavities. One way out is to accept that path. Another is to keep `tetgen` for the connectivity and carry the surface-vertex positions, which do carry gradients, to the tet nodes through the `surface_mesh_indices` map that `tetrahedralize_surface` returns and `fem.py` discards; the interior Steiner points would then need a rule of their own.

In progress None of the five steps exists in `structsept` yet: IDEIAS queue item 9 (“tet mesh + torch-fem”) stops at $\mathbf{K}$, and the graded lattice (item 8) is still open. A mesh that is both differentiable and free of cavities is the open design question of this part. The modules themselves are committed and unchanged in the working tree.

Chapter 9

Recipes

Seven tasks, each as numbered steps with the exact commands. The other chapters explain why the steps are what they are; here they are only referenced. Two rules hold for every command: run it from the repo root `code/` (the folder with `pyproject.toml`), and run Python only through `uv run`. Every flag shown was checked against the script's `argparse` code; if `--help` ever disagrees, `--help` wins.

9.1 Set up a fresh clone

`data/`, `runs/` and `outputs/` are gitignored, so a clone has the code but no datasets, no trained decoders and no presets. Set up the environment first, then the data.

1. **Environment.** You need `git` and `uv`; `uv` fetches its own Python (other install options: docs.astral.sh/uv). The project has been developed and run only on Windows 11 with CPython 3.12.7; there is no `.python-version`; `uv python pin 3.12` keeps `uv` on that minor version. The desktop shortcuts, `KeepAwake` and the PowerShell commands of this chapter are Windows-only: elsewhere `launcher --install` refuses and `KeepAwake` does nothing.

```
winget install --id=astral-sh.uv -e # Windows; once per machine
git clone --recursive https://github.com/tomito6/struct_opt_nn.git code
cd code
git checkout gui-train-tab # work since 23/09, not in main
git submodule update --init --recursive
uv sync # .venv, editable installs, CPU torch
uv run pytest tests/test_deepsdfstruct_env.py -v # offline smoke test
```

The submodule needs its git history (it reads its version from it), so a zip copy does not install. Run `uv` only from `code/`: inside `DeepSDFStruct/` it would build a second, unrelated environment.

2. **Desktop shortcuts (optional).** Double-click `install_shortcuts.bat`. It checks for `uv` and the submodule, runs `uv sync`, then writes *Lattice explorer* and *SDF maker* into `Desktop\NN\` (Chapter 7). Run it again after moving the folder: the shortcuts hold absolute paths.
3. **Data.** Rebuild the datasets, then the presets. The presets come last because their `specs.json` store the absolute path of the dataset split (which is also why a `runs/` folder does not work on another machine).

```
uv run python -m datagen.make_plate_hole --dim 2 --radius-only --n-uniform 25000 --n-band 25000 --name plate_hole_2d_r_only --plot
uv run python -m datagen.make_plate_hole --dim 2 --n-uniform 25000 --n-band 25000 --name plate_hole_2d_xyr --plot
uv run python -m datagen.make_plate_tri --dim 2 --n-uniform 25000 --n-band 25000 --name plate_tri_2d_h --plot
uv run python -m datagen.make_plate_tri --dim 2 --free-width --n-uniform 25000 --n-band 25000 --name plate_tri_2d_hw --plot
uv run python experiments/make_plate_presets.py
```

For the 166-shape set, add `--large-holes 32 --large-holes-from 0.25` to the second command and name it `plate_hole_2d_xyr_n166`. Each dataset keeps its exact command under `"command"` in its `dataset.json`. Always pass `--name` and both sample counts: with the hole family's default names (`plate_hole_2d`, `plate_hole_2d_r`) the presets do not find the dataset, and the default 10 000 + 10 000 samples differ from every existing dataset.

In progress A clone of `gui-train-tab` lacks the triangle family (`datagen/make_plate_tri.py` and its modules), `--large-holes`, `unattended.prepare_run_like` and `code_vs_parameter`, `experiments/train_plate_tri.py`, `experiments/train_plate_xyr_n166.py` and the two triangle presets: all uncommitted on the author's machine (2 October 2026). Until they are committed, a clone can rebuild only the two hole datasets and their presets.

9.2 The daily research loop

The research question (Chapter 10) is answered by one loop, run again and again: make shapes whose generating parameters are known, train an auto-decoder that never sees them, check whether its codes recover them (Figure 21).

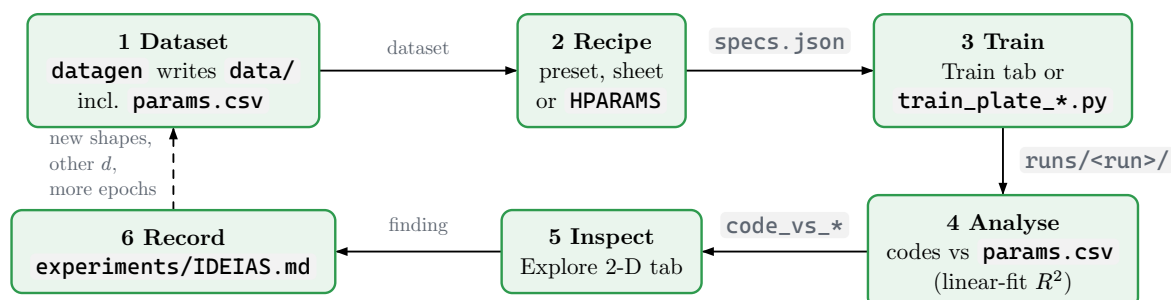


Figure 21: The loop behind every result so far. The dashed arrow is the decision a finding leads to; the coverage gap on the radius axis, for example, led to the 166-shape dataset.

1. **Dataset** in `data/SdfSamples/<dataset>/` (Chapter 5). Everything below relies on one invariant: split order = latent code index = `params.csv` row.
2. **Recipe** from a preset `runs/preset_*` or a supervisor sheet in `docs/hyperparameters/` (both loaded in the Train tab's *All hyperparameters...* window), or a script's `HPARAMS` dict or `--like` preset (Chapter 6). The presets set d to the number of generating parameters.
3. **Train**: runs of minutes in the GUI Train tab, runs of hours headless (recipe below), which survives a closed window and keeps Windows awake.
4. **Analyse**. The training scripts end by writing `code_vs_<param>.csv/.png`, `code_vs_params.*` or `latent_coverage.json` into the run folder. For a run trained in the GUI, rerun only that step: add `--check-only --run <run>` to the triangle, free-hole or n166 script. Otherwise call `unattended.code_vs_parameter` (`structsept/app/unattended.py:525`) with the run folder, the dataset folder, the split and a `params.csv` column; it matches codes to rows by shape name. Judge a parameter by the linear-fit R^2 over *all* components, since the learned basis is arbitrary: in `plate_tri_2d_hw` no single component is monotonic in h , yet the fit gives $R^2 = 1.000$.
5. **Inspect**. Double-click the run in the Train tab's run table. A planar run opens in Explore 2-D: one slider per latent component, gaps wider than 12 % of an axis shaded (Chapter 7).
6. **Record** a section `## <title> (dd/mm)` in `experiments/IDEIAS.md`, the lab notebook (Portuguese so far): the request, what was built, the exact command, launch time and pid, what you expect, later a `**Resultado**` paragraph with the numbers. It is the only versioned record of a result; run folders are gitignored.

9.3 Add a new parametric family

A new family (four independent triangle heights, an elliptic hole, ...) gets the same three `datagen/` modules as the existing two, then a preset and a training script. Copy the triangle family: it is the newer one and imports the hole family's frame, extrusion and sampling settings instead of repeating them.

File	What it must provide
<code>datagen/<family>_params.py</code>	The admissible parameter set, a map from the unit cube onto it (<code>from_unit</code> , <code>to_unit</code>), samplers (grid for one parameter, Sobol plus extremes for several), and a parameters class with <code>names</code> , <code>rows()</code> , <code>save_csv()</code> . Template: <code>datagen/plate_tri_params.py</code> .
<code>datagen/<family>_sdf.py</code>	The exact signed distance in the $[-1, 1]^d$ frame and <code>sample_instance(rng, dim, frame, ..., config)</code> returning float32 rows <code>(x, y, [z,] phi)</code> . Reuse <code>PlateFrame</code> , <code>extrude</code> , <code>SamplingConfig</code> from <code>plate_hole_sdf.py</code> .
<code>datagen/make_<family>.py</code>	The CLI (<code>--dim</code> required, <code>--name</code> , <code>--overwrite</code> , <code>--plot</code> , <code>--dry-run</code> , sample counts); instance i seeded with <code>default_rng([seed, i])</code> ; all writing through <code>datagen/dataset.py</code>

	(<code>prepare</code> , <code>write_instance</code> , <code>write_split</code> , <code>write_manifest</code> with <code>geom_dimension</code> and <code>command</code>) plus <code>save_csv</code> . Template: <code>main</code> in <code>datagen/make_plate_tri.py</code> .
<code>tests/test_datagen.py</code>	Exactness against a brute-force distance to a densely sampled boundary; the file layout written through the CLI into <code>tmp_path</code> .
<code>experiments/make_plate_presets.py</code>	One more <code>PRESETS</code> entry (<code>experiments/make_plate_presets.py:70</code>): run name, dataset, <code>latent_dim</code> , description.
<code>experiments/train_<family>.py</code>	A copy of <code>train_plate_tri.py</code> with its own <code>FAMILIES</code> entry (dataset, preset, run name, the <code>params.csv</code> columns to check) and a <code>LABELS</code> entry for each of those columns. <code>prepare_run_like</code> and <code>code_vs_parameter</code> do the rest.

Three rules carry over from the existing families (Chapter 5). First, the field is exact only for *admissible* parameters: the boolean `max(plate, -hole)` is a true distance only while the ligament rule holds. Second, latent indices must stay stable: prepend the extremes and append extra shapes at the end, as `HoleParameters.extended` does. Third, `datagen` imports nothing from the rest of the repo. Then run the CLI with `--dry-run`, then for real with `--plot` (look at `preview.png`), rerun `make_plate_presets.py`, smoke-test the training script, and list the new modules in `datagen/__init__.py` and `docs/structure.md`.

9.4 Train overnight without the GUI

Torch runs on the CPU here, so the 8×256 decoder trains for hours. Long runs go through the headless scripts in `experiments/`, which run the Train tab's trainer with the helpers in `structsept/app/unattended.py` (lifecycle in Chapter 6).

Which script trains which dataset, with which recipe and how long it took: table in Chapter 6; every flag: Appendix C.

1. **Smoke-test first.** Two epochs run the whole lifecycle (specs, training, metadata, analysis); the log must end with the analysis numbers and `done`.

```
uv run python experiments/train_plate_tri.py --family hw --epochs 2 --run smoke
```

In the triangle and n166 scripts `--epochs` exists only for this; in the `_4h` scripts it is the real epoch count, so never forget `--run smoke` there. A missing dataset or preset stops the script with the command that builds it; the n166 script also needs its `--like` run, which exists only where it was trained. Delete `runs/smoke/` and `outputs/logs/smoke.*` afterwards.

2. **Launch hidden** (PowerShell), so that closing the terminal does not kill the run. `specs.json` is written at once, so the run is in the GUI table (after *Refresh*) while it waits.

```
$cmd = "run python experiments/train_plate_tri.py --family hw"
Start-Process uv -ArgumentList $cmd -WindowStyle Hidden
```

3. **Schedule or chain.** `--start-at` takes `now`, `HH:MM` (next occurrence) or `"YYYY-MM-DD HH:MM"`; `--after` takes a pid or `.pid` file and waits until that process has exited.

```
uv run python experiments/train_plate_xyr_4h.py --start-at 23:30
uv run python experiments/train_plate_tri.py --family hw --after outputs/logs/<run>.pid
```

`<run>` is the name of the first run. Start it before the second: `--after` reads the pid file once, and a leftover file from an older run holds a dead pid, so the second would start at once.

4. **Follow and stop** (PowerShell). The Train tab's loss curve follows only a run started there, so follow a headless run in its log.

```
Get-Content outputs/logs/<run>.log -Wait -Tail 30      # follow the log
taskkill /PID <pid> /F                                # pid: outputs/logs/<run>.pid
```

The two `_4h` scripts also stop by themselves, cleanly, once `--max-hours` have passed, and say so in `metadata.json` (`stopped_at_deadline`). At the end every script adds `last_epoch` and `wall_seconds` there and writes its analysis files.

Watch out **One training at a time:** torch uses every core, so two trainings (two scripts, or a script and the Train tab) slow each other down; chain them with `--after`. **No resume:** `structsept.app.training.train` never passes `continue_from`, and `--force` starts from scratch. `plate_hole_2d_xyr_n166_d3_ep800` died silently at epoch 80 of 800, and its `latest.pth` loads like a finished one. The run table shows it as `80/800` and the explorer pickers as *epoch 80 of 800* (uncommitted code); in a script, `unattended.last_epoch(run_dir)` gives the epoch.

9.5 Compute the stiffness matrix of a geometry

The FEM of this repo stops at the unconstrained global stiffness matrix K on purpose (Chapter 8).

1. **The demonstration:** a $4 \times 4 \times 1$ lattice of the analytic round cross (latent 0.4) with a 0.25 m hole, resolution 10 (about 10 FlexiCubes cubes per cell), steel. About 7 s; writes `stiffness_matrix.npz`, `plate_with_hole_tets.vtk` and `stiffness_sparsity.png` into `experiments/outputs/`.

```
uv run python experiments/plate_with_hole_stiffness.py
uv run python experiments/plate_with_hole_stiffness.py --solid --outdir experiments/
outputs/solid
```

Further flags: `--tiling`, `--latent`, `--resolution`, `--hole-radius`, `--hole-center`, `--length`, `--width`, `--thickness`, `--youngs-modulus`, `--poisson-ratio`, `--no-plot`. The log reports asymmetry, smallest diagonal entry and rigid-body residual (`check_stiffness`).

2. **Your own geometry.** The functions of `structsept/fem.py` take any SDF on the parametric cube $[0, 1]^3$ together with a `TorchSpline` that maps the cube to metres. Wrap the SDF in `CappedBorderSDF`: it closes the surface with flat faces at the domain border. A lattice alone also closes (it is positive outside its box), but an SDF still negative at the edge of the meshing grid gives an open surface, and `tetrahedral_mesh` raises. `plate_with_hole` (`structsept/plate_with_hole.py:83`) returns such a pair:

```
import torch
from DeepSDFStruct.utils import configure_logging
from structsept.fem import (assemble_stiffness, build_solid, check_stiffness,
                           default_dtype, tetrahedral_mesh, to_scipy)
from structsept.plate_with_hole import plate_with_hole

configure_logging() # otherwise the checks are silent
plate, deformation = plate_with_hole(hole_radius=0.15, tiling=(4, 4, 1), device="cpu")
vertices, tets = tetrahedral_mesh(plate, deformation, resolution=10)
with default_dtype(torch.float64): # torch-fem needs float64
    solid = build_solid(vertices, tets) # steel, E = 210e9 Pa, nu = 0.3
    k, K = assemble_stiffness(solid) # k_e (n_el, 12, 12), K sparse
    check_stiffness(K, solid)
K_csr = to_scipy(K)
```

Watch out K is singular by design (six rigid-body modes): pin nodes before solving. The demo cell is the *analytic* round cross, not a trained decoder. And the surface-to-tetgen path goes through NumPy: no gradient reaches K from the latent.

9.6 Start a real optimization run

No optimization loop exists in `structsept/` or `experiments/` yet. The only one is the library test `DeepSDFStruct/tests/test_structural_optimization.py:21`, walked through in Chapter 4. Copy it into `experiments/`, so the submodule stays untouched, and run the copy from `code/`:

```
cp DeepSDFStruct/tests/test_structural_optimization.py experiments/optimize_cantilever.py
uv run python experiments/optimize_cantilever.py
```

Its `__main__` block turns every warning into an error; drop that line, and send `sim_out.vtk` (written to the current directory) to `experiments/outputs/`. Then change what defines the problem (line numbers of the original):

What	Line	Change, and why
Decoder	26	<code>AnalyticRoundCross</code> is a formula (latent = radius). For a learned cell use <code>RoundCross</code> ($d = 1$), <code>ChiAndCross</code> ($d = 2$) or <code>get_model("runs/<run>")</code> for a 3-D decoder trained here.
Domain	42, 47	<code>tiling = [2, 1, 1]</code> and the FFD <code>box(2, 1, 1)</code> that sets the physical size.
Control net	34–38	Degrees <code>[1, 0, 1]</code> and the knots give $5 \times 1 \times 3 = 15$ control points of length d , starting at 0.3: the design variables.
Bounds	65	<code>[0.15, 0.75]</code> are radii of the analytic cell. For a trained decoder use the range of <code>model._trained_latent_vectors</code> , as an array of shape <code>(numel, 2)</code> (<code>np.tile([lo, hi], (param.numel(), 1))</code> ; the template's broadcast only works for $d = 1$).
Supports, loads	29–32, 133–145	Caps at the clamped (<code>x0</code>) and loaded (<code>z1</code>) faces; $E = 1000$, $\nu = 0.3$; nodes at $x \approx 0$ fixed; total force -100 in z on nodes with $z > 0.9$.
Volume	63, 156	$G = V - V_{\text{target}}$ with an absolute $V_{\text{target}} = 0.5$ (domain volume 2). MMA normalizes only the objective, so scale G yourself, e.g. $V/V_{\text{target}} - 1$.
Resolution	76	$N = 10$ per cell; the paper's cantilever uses 20 (about 200 000 tets, 1 min 50 s per iteration).
Iterations	21	<code>num_iter=1</code> ; MMA has no stopping rule, so add one.

Watch out Between control points and F , G everything stays in torch: no `.detach()`, no `.item()`, no NumPy, or the gradient silently vanishes. The template meshes with `mesh_type="volume"`, which loses about 30 % of the volume to cavities; the surface-plus-tetgen path of `fem.py` is accurate but not differentiable. Which one an optimization should use is open (Chapter 8).

The 2-D plate decoders of this repo do not fit this template: they have `geom_dimension 2` and each one is a whole plate, not a cell for `LatticeSDFStruct` to tile; a `--dim 3` plate decoder is still a whole plate. A first run uses a shipped 3-D decoder, or one trained here on 3-D unit-cell data (SDF maker).

9.7 Testing and conventions

File in <code>tests/</code>	What it covers	Needs
<code>test_deepsdfstruct_env.py</code>	Package import, SDF values, booleans, a gradient, FlexiCubes, lattice tiling; pretrained check only with <code>DEEPSDF_TEST_PRETRAINED=1</code>	torch, fast
<code>test_datagen.py</code>	Design spaces, exact fields, file layout, overwrite safety, both families	one 2-epoch training
<code>test_app_unattended.py</code>	Argument parsing, log tee, pid wait, <code>prepare_run(_like)</code> , <code>code_vs_parameter</code>	no trainer
<code>test_app_hyperparams.py</code>	Schema, validator vs real decoder, sheet import, 4-epoch training, window	torch, Tk
<code>test_app_runs.py</code>	Rename, notes, delete; run lists	Tk
<code>test_app_explore.py</code>	Explore tab regressions	torch, Tk
<code>test_app_explore2d.py</code>	Tiny dataset, 2-epoch Train-tab run, Explore 2-D	torch, Tk
<code>test_app_launcher.py</code>	Console-less launch (real <code>pythonw</code>), env sync, instance lock, <code>.lnk</code> files	Windows

Tk tests run hidden off-screen and skip without a display; the tests write into pytest's temporary folders, never into the real `data/` or `runs/`.

```
uv run pytest tests/test_deepsdfstruct_env.py -v    # after any dependency change
uv run pytest tests/ -v                            # everything in this repo
uv run pytest DeepSDFStruct/tests/test_structural_optimization.py -v
```

One sheet-import test in `test_app_hyperparams.py` would fail as written (not run): it expects 300 epochs from the `xyr_d3` sheet, which says 600 (Chapter 10).

Conventions:

- **Always `uv run`, from `code/`**, so the locked environment is the one in use.
- **Format what you touch:** `uvx black <files>`. Black is not a project dependency; `uvx` runs it in a throwaway environment.
- **Differentiability** between design variables and objective (box above). The GUI and the K path of `fem.py` are outside that loop.
- **Never import from `experiments/`**. When a second caller needs script code, move it into `structsept/`; that is how `prepare_run_like` and `code_vs_parameter` got into `unattended.py`.
- **Leave `DeepSDFStruct/` alone** unless the change is meant for upstream, and ask before adding a dependency: the environment is heavy already.
- **Record every result in `experiments/IDEIAS.md`**.
- **`CLAUDE.md` is curated, not appended to.** It is the context of AI sessions, kept under 150 lines by `docs/context-maintenance.md` (run as `/update-context`, each pass logged in `docs/context-log.md`).
- **Start new work from `gui-train-tab`**, not `main` (which has no `datagen/`, launcher or 2-D tabs), until the branch is merged; ask before pushing to the shared remote.
- **Commit before a big refactor:** the tree holds much uncommitted work, and a refactor on top of it cannot be undone cleanly.

Chapter 10

Where the research stands

This chapter is a snapshot taken on **2 October 2026**. Its source is the lab notebook `experiments/IDEIAS.md` (written in Portuguese), checked against the run folders in `runs/` and the logs in `outputs/logs/`. Those folders are gitignored, so the notebook is the only versioned record of what was found. If this chapter and the notebook disagree, the notebook is newer. Write your own findings there as well.

10.1 The research question

The paper treats the latent space as a design space. MMA moves the spline control points $\hat{\lambda}_i$ inside box bounds, and the library template uses `[0.15,0.75]` for every one of them (`DeepSDFstruct/tests/test_structural_optimization.py:65`). Those bounds make sense only if the box contains shapes the decoder was trained on. A DeepSDF auto-decoder does not guarantee that. It has no encoder. Each training shape gets a free code, initialized as noise and optimized together with the weights, so the network invents its own coordinates. Before building an optimization on self-trained decoders, this repo therefore asks:

Does the latent space that an auto-decoder learns on its own recover the parameters that generated the shapes, and how many latent dimensions does that take?

Why the answer matters for the optimization:

- **Reading a design back.** If the codes are an affine relabelling of the parameters, an optimized code can be mapped back to (x_c, y_c, r) with the fitted map, and a constraint written in parameter space stays linear in λ . If the codes are curved, folded, or have lost a parameter, neither is true.
- **The admissible set is not a box, even in parameter space.** A hole must keep a margin to the plate edge, so the largest radius shrinks as the centre moves outwards: the feasible set in (x_c, y_c, r) is a pyramid (module docstring of `datagen/plate_hole_params.py:1`). `datagen` therefore also stores each shape as a point (u_x, u_y, t) of the unit cube, where every point maps to an admissible shape. Freezing the codes to these unit-cube coordinates during training would keep the design space a box all the way to MMA. `HoleParameters.latent_codes` (`datagen/plate_hole_params.py:688`) and `TriParameters.latent_codes` (`datagen/plate_tri_params.py:573`) compute such codes. **No training path uses them yet.**
- **Gaps.** A box drawn around the learned codes also contains regions with no training code in them, where the decoder extrapolates (rule 4 below).

That leaves two routes: learn the codes and check that they are a clean relabelling of the parameters, or prescribe them. All experiments so far take the first route. Every dataset writes `params.csv`, whose row i belongs to latent code i . After training, `code_vs_parameter` (`structsept/app/unattended.py:525`) and `compare_codes_with_params` (`experiments/train_plate_xyr_4h.py:261`) compute per-component correlations, the share of variance on each principal axis, and a linear or affine fit of each parameter on all code components. The verdict is the R^2 of that fit. How to run these checks is in Chapter 6. The test bed is 2-D plates with exact SDFs (Chapter 5). A plate of constant thickness with a through-hole is an extrusion of the 2-D shape, so the 2-D field holds all the information.

10.2 Timeline

The project ran in two phases (Figure 22). Until about 21 September the work was learning the library and building the pieces of the online pipeline (geometry, FEM up to K , the point-cloud fit). From 23 September on, it has been the latent-space question. All training runs on the laptop's CPU (`torch 2.13.0+cpu`; the GPU is not used), which is why the long runs take hours and are chained overnight (Chapter 6).

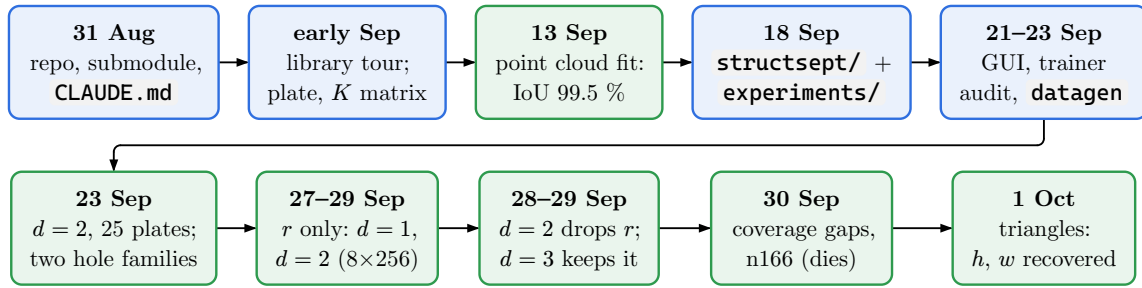


Figure 22: Milestones from the first commit to the triangle results. Blue: code and tooling; green: experiments with a measured result. Top row: learning the library; bottom row: the latent-space question. Commit **6c07cfc** (29 Sep) is the last one; everything from 30 Sep on is uncommitted.

10.3 Families, datasets and runs

Two plate families exist, each in a one-parameter and a multi-parameter variant (Chapter 5). The table lists every run that produced a result or was meant to. Numbers come from `code_vs_*.csv/json`, `metadata.json` and the run logs; R^2 is the fit of the parameter on all code components. Presets in `runs/preset_*` hold only a `specs.json` and a `metadata.json`, no checkpoints, and are left out.

Dataset (shapes)	Run; d , decoder, epochs	Headline result
<i>Circular hole, free: x_c, y_c, r</i>		
<code>plate_hole_2d_n25</code> (25)	First_test $d = 2, 4 \times 64, 250$	smoke test: R^2 x_c 0.07, y_c 0.87, r 0.50; keeps y_c , drops x_c
<code>plate_hole_2d_n134</code> (134)	<code>plate_hole_2d_n134_d2_20260928_1338</code> $d = 2, 8 \times 256, 600$	R^2 x_c 0.759, y_c 0.794, r 0.009 ; keeps the centre, drops the radius
<code>plate_hole_2d_xyrr</code> (134)	<code>plate2d_xyrr_d3_8x256_4h</code> $d = 3, 8 \times 256, 300$	R^2 x_c 0.988 , r 0.857 , y_c 0.741; principal axes 49/28/23 %
same	<code>plate_hole_2d_xyrr_d3_20260930_0124</code> $d = 3, 8 \times 256, 800$	complete, loss 0.0058; codes not yet compared with the parameters
<code>plate_hole_2d_xyrr_n166</code> (166)	<code>plate_hole_2d_xyrr_n166_d3_ep800</code> $d = 3, 8 \times 256, 800$	died at epoch 80 ; no result
<i>Circular hole, centred: r only</i>		
<code>plate_hole_2d_r_only</code> (40)	<code>plate2d_r_only_d1_8x256_4h</code> $d = 1, 8 \times 256, 600$	code linear in r : Pearson -1.000 , monotonic
same	<code>plate2d_r_only_d2_8x256_4h</code> $d = 2, 8 \times 256, 600$	codes on a line (84.8 % of variance on PC1, Spearman 1.000 along it); the other direction is frozen initial noise
<i>Four triangular holes: height h, then h and width w</i>		
<code>plate_tri_2d_h</code> (40)	<code>plate_tri_2d_h_d1_4x64</code> $d = 1, 4 \times 64, 1500$	code linear in h : Pearson -1.000 , monotonic (20.6 min)
<code>plate_tri_2d_hw</code> (133)	<code>plate_tri_2d_hw_d2_4x64</code> $d = 2, 4 \times 64, 1500$	R^2 h 1.000 , w 0.996 ; no single component is monotonic (49.9 min)

`plate_hole_2d_n134` and `plate_hole_2d_xyrr` are byte-identical (same seed); only the name differs. **First_test** has no analysis file; its R^2 values were recomputed from its saved codes with the same affine fit. The small 4×64 preset recipe was enough for the triangle families; the analysed hole runs after **First_test** used the 8×256 recipe of the unattended scripts.

10.4 Rules of thumb learned so far

1. **With d equal to the number of generating parameters, the decoder recovers them up to an arbitrary affine basis.** The centred hole and the triangle height give a straight line ($d = 1$, Pearson -1.000 both). The triangles with (h, w) give $R^2 = 1.000$ and 0.996 , yet single components correlate only 0.47 to 0.88 with either parameter: the codes fill the (h, w) feasible set (a rectangle with one corner cut off, Chapter 5), rotated and sheared. Consequence: judge by a fit on all components, never component by component. In the Explore 2-D tab, no single slider is “height” or “radius”.

2. **With d smaller than the number of parameters, the decoder keeps what dominates the loss.** On 134 free-hole plates with $d = 2$ it kept the centre and dropped the radius ($R^2 = 0.009$). Moving the hole changes the SDF over the whole plate; changing the radius changes only a ring around the hole. The small first run (25 plates, 4×64 , 250 epochs) instead kept y_c and dropped x_c : which parameter survives also depends on the data and the training budget.
3. **A latent direction the decoder does not need keeps its random initialization.** In the centred-hole run with $d = 2$, the coordinate across the line of codes correlates 0.96 between epoch 1 and epoch 600 and not at all with r ; its spread only fell from 0.092 to 0.053. The gradient along that direction is about zero, and only the weak code regularization pulls it in. For an optimizer this would be a design variable the geometry may not respond to (still to be measured, see below).
4. **Gaps in the training data become gaps in the latent space.** The Explore tabs shade the widest stretch of an axis without a trained code when it exceeds 12 % of the range (`GAP_FRACTION`, [structsept/app/viz.py:28](#)). In the $d = 3$ hole runs the first axis, which mostly carries the radius, has a 29 % gap: only 8 of 134 shapes have $r > 0.25$, because a large hole needs a central position and t near 1, a corner of the unit cube that a 128-point Sobol draw barely reaches. Not every gap is missing data: a 23 % gap on the third axis sits next to a corner shape whose nearest neighbour is close in parameter space; there the latent space is stretched.

10.5 Open threads and next steps

1. **Rerun the n166 training** (the run died, see below) and check that the radius gap closes. The dataset adds 32 holes with $r \in [0.25, 0.45]$: 40 shapes above $r = 0.25$ instead of 8, and the largest jump between neighbouring radii falls from 0.108 to 0.017. The script copies the reference run's `specs.json` key for key and writes `latent_coverage.json` (`latent_coverage`, [experiments/train_plate_xy_r_n166.py:160](#)). The expectation is that the first-axis gap disappears and the third-axis gap stays. Budget more than a night: the first launch ran 80 epochs in 83 min with the laptop in use (62 s per epoch), and the script's docstring now says to plan for 14 h for the 800 epochs. Resuming from epoch 80 is not an option: the library trainer accepts `continue_from` ([DeepSDFStruct/DeepSDFStruct/deep_sdf/training.py:299](#)) but discards the latent codes it loads (line 517), so the codes would restart from noise, and `structsept.app.training.train` does not pass it on anyway ([structsept/app/training.py:157](#)). Relaunch with `--force`.
2. **Find out why y_c is recovered worst at $d = 3$.** The error sits in the lower half (RMSE 0.164 for $y_c < 0.45$ against 0.051 above); the worst shapes are holes near the bottom edge that the fit places near the middle. The plate is symmetric in x and y and x_c comes out at 0.988, so the notebook's hypothesis is a training accident, not geometry. Three cheap checks: compare the codes of the complete 800-epoch run, look in Explore 2-D whether the hole moves down to the bottom edge, and train a second seed.
3. **Measure the dead direction** of the centred-hole $d = 2$ decoder: sweep ± 0.1 across the line of codes from a middle code and see whether the SDF changes.
4. **Four independent triangle heights.** The SDF code already takes one height per triangle (`triangles_in_frame`, [datagen/plate_tri_sdf.py:78](#)); what is missing is a sampler in `plate_tri_params.py` and the extra `params.csv` columns.
5. **Prescribed codes.** A training path that freezes the codes to `latent_codes` (the second route above), so that MMA gets box bounds by construction.
6. **Towards an optimization loop.** `structsept/fem.py` stops at K , and the 2-D plate decoders do not plug into `LatticeSDFStruct`; Chapter 8 lists what is missing.
7. **Small items.** Two GUI windows can collide on automatic run names (minute resolution) and do not refresh each other's run lists; a seconds-plus-PID suffix and a refresh timer are about 30 lines (Chapter 7). Two cheap trainer experiments are queued: a decoder learning-rate sweep from `1e-4` to `3e-3` with a step decay about every 50 epochs, and `clampedL1` with $\delta = 0.1$ against `L1` with $\delta = 0.3$. The early queue at the top of `IDEIAS.md` (radius sweep against the analytic volume, a hole through the border as a `CappedBorderSDF` test, several holes, other hole shapes, a graded lattice with `SplineParametrization`) is also still open.

The first two threads need no new code. For the second, `<run>` is `plate_hole_2d_xyr_d3_20260930_0124`:

```
# n166 from scratch; add --after outputs/logs/<run>.pid to chain it after another run
uv run python experiments/train_plate_xyr_n166.py --force
# code-vs-parameter check of an existing d = 3 run, no training
uv run python experiments/train_plate_xyr_4h.py --check-only --run <run>
```

10.6 Broken or incomplete

1. **The n166 run died at epoch 80.** `runs/plate_hole_2d_xyr_n166_d3_ep800` was started on 30 Sep for 800 epochs. Its `latest.pth` was last written the same evening, at epoch 80. The log ends with the start-up lines and shows no error, and the notebook records that the machine did not reboot; the cause is unknown. `metadata.json` has `"final_loss": null` and there is no `latent_coverage.json`, but the GUI still lists the run as loadable. Do not read its codes as a result.
2. **The “ep800” run stopped at epoch 680.** `runs/plate_hole_2d_xyr_d3_ep800`, started from the GUI, has `latest.pth` at epoch 680 and no `800.pth`, `metadata.json` or `training_summary.json`. The complete 800-epoch run with an identical `specs.json` is `plate_hole_2d_xyr_d3_20260930_0124`. `train_plate_xyr_n166.py` still takes ep800 as its default `--like`; that is harmless: its `specs.json` is identical, and its epoch-680 codes, which the script also reads for the reference coverage, show the same 29 / 5 / 23 % gaps as the complete run. Three more GUI runs are leftovers: `plate_hole_2d_xyr_d3_20260929_0759` (epoch-1 checkpoint only), `Test2` (500 epochs on `plate_hole_2d_n25`, never analysed) and `Test_2` (no checkpoints).
3. **Uncommitted work.** The triangle results and the n166 dataset rest on uncommitted code, and `gui-train-tab` is not merged (Chapter 3, Git state).

Watch out A test would fail as written (not run).

`test_the_xyr_d3_sheet_is_the_same_recipe_sized_for_134_shapes` (`tests/test_app_hyperparams.py:543`) expects 300 epochs, but `from_sheet` reads 600 from `docs/hyperparameters/NN_Training_Hyperparameters_plate2d_xyr_d3_8x256_4h.xlsx` (the decay interval, 75, agrees). The run the sheet documents trained 300 epochs, so the sheet is the likely error; see the open questions below.

10.7 Open questions for the supervisor

Recorded in the notebook and not yet answered:

- What plate thickness fits the real HiWi case? The 0.1 m in `structsept.plate_with_hole` was a guess.
- Is the hole a requirement of the problem or an exercise?
- Point-cloud input: is the target part already porous, so that fitting the latent field to the cloud is right (this is what `experiments/pointcloud_to_lattice.py` does), or is it solid, so that the lattice should only fill it (a trim, which leaves the latents free for MMA)? Orienting the normals of a raw cloud properly is also not implemented.

Not recorded in the notebook, but the next phase depends on them:

- Which route into the optimization: learned codes read back through the fitted affine map, or prescribed unit-cube codes?
- How do the plate-level 2-D decoders connect to the paper’s setting, where the decoder describes a 3-D unit cell that `LatticeSDFStruct` tiles?

From a check of the training recipes against the paper (2 October 2026); recorded here, not decided:

- **Decay interval 75.** The `xyr_d3` sheet halves the learning rate every 75 epochs, where the two `r_only` sheets use 150. The 75 went into the three 800-epoch runs (`plate_hole_2d_xyr_d3_ep800`, `plate_hole_2d_xyr_d3_20260930_0124`, the `n166` run), so from epoch 750 the rate is 1/1024 of its start. In the complete run the loss falls only from 0.0060 at epoch 450 to 0.0058 at epoch 800.

- **Points per step.** The sheet row *Points per training step* is divided by the batch (16 384 / 4 = 4096 per shape), whereas the shipped `round_cross` specs use the paper's 16 000 per shape (`SamplesPerScene` 16000).
- **Regularization ramp.** The paper's Eq. 9 prints the code-norm weight as $\sigma_0 \min(1, 1/n_{\text{epoch}})$, a decay; the library ramps it up, $\min(1, \text{epoch} / 100)$, at [DeepSDFStruct/DeepSDFStruct/deep_sdf/training.py:622](#) (also noted in [docs/paper_context.md](#)).

10.8 Upstream bug candidates

Found while reading or running the library, not yet reported to `DeepSDFStruct`. The hardcoded gradient clip and the late seed (trap table in Chapter 6) are candidates too; the other trainer traps there are documented behaviour rather than bugs.

Problem and location	Effect and workaround
<code>create_3D_mesh(mesh_type="volume")</code> leaves cavities: a solid cube keeps only about 70 % of its volume at any resolution. DeepSDFStruct/DeepSDFStruct/mesh.py:669	Mesh the surface and call <code>tetrahedralize_surface</code> . The library's optimization test uses the lossy path (line 77 of the test).
<code>plot_reconstruction_loss</code> smooths over 41 steps and, when given a CSV path, fails with fewer than 40. DeepSDFStruct/DeepSDFStruct/deep_sdf/plotting.py:121	<code>ValueError</code> after short reconstructions that write <code>loss_csv_path</code> ; run at least 40 steps or drop the CSV.

Appendix A

Glossary and notation

The paper, the library and this repository name the same things in three ways. The first table maps the paper's symbols to the code; the second explains the words used in dataset names, run names, scripts and `experiments/IDEIAS.md`.

A.1 Paper symbols and their names in the code

Symbol	Meaning	In the code
$f_\theta(\lambda, \mathbf{x}), \theta$	DeepSDF decoder: signed distance at $\mathbf{x}$ of the shape with code λ ; θ its weights	<code>NetworkArch</code> <code>deep_sdf_decoder</code> ; weights in <code>ModelParameters/<epoch>.pth</code> ; <code>get_model</code> loads a <code>DeepSDFModel</code> (not an <code>nn.Module</code>), <code>SDFfromDeepSDF</code> wraps it as an SDF
$\lambda \in \mathbb{R}^d, \lambda_j$	latent code; λ_j the code of training shape j	row j of <code>latent_codes["weight"]</code> in <code>LatentCodes/<epoch>.pth</code> ; j = position in the split = row j of <code>params.csv</code>
d	latent dimension	<code>CodeLength</code> (specs), <code>latent_dim</code> (hyperparameters, metadata), <code>_d1</code> , <code>_d2</code> , <code>_d3</code> in run names
φ	signed distance, $\varphi < 0$ inside the material	last column of every npz row; <code>sdf(points)</code> returns shape <code>(N, 1)</code>
δ, c_δ	clamp of the training loss	<code>ClampingDistance</code> ; <code>clampedL1</code> also clamps at its own 0.1
σ	initial spread, $\lambda_j \sim \mathcal{N}(0, \sigma^2/d)$ per component	<code>CodeInitStdDev</code> ; presets and script runs: $\sqrt{0.01d}$
λ_{reg}	weight of the code-norm penalty (the paper's σ)	<code>CodeRegularizationLambda</code> , default <code>1e-4</code> , ramped in over 100 epochs
$\hat{\lambda}_i$	spline control points: the design variables	parameters of <code>SplineParametrization</code> , flattened x -fastest, row $i + n_x(j + n_y k)$
$\lambda(\mathbf{x}), \varphi_i$	latent field $\sum_i \varphi_i(\mathbf{x}) \hat{\lambda}_i$, B-spline basis φ_i	<code>SplineParametrization</code> (or <code>Constant</code>), evaluated by a <code>TorchSpline</code>
$T(\mathbf{x})$	tiling map, parametric $[0, 1]^3$ to decoder cube $[-1, 1]^3$, cells mirrored	<code>LatticeSDFStruct(tiling, microtile, parametrization)</code>
$\bar{\Omega}$	deformed, physical domain (FFD)	a <code>TorchSpline</code> as <code>deformation_function</code> of <code>create_3D_mesh</code> ; moves vertices only
$K, \mathbf{u}, \mathbf{f}$	stiffness matrix, displacements, forces	<code>assemble_stiffness</code> in <code>structsept/fem.py</code> ; <code>solve(...)</code> in the library template
J, V, V_{target}	compliance $\mathbf{f}^T \mathbf{u}$; volume and its bound	<code>torch.inner(f.ravel(), u.ravel());</code> <code>G = vol - target_vol, target_vol = 0.5;</code> <code>optimization.MMA(param, bounds)</code>
$x_c, y_c, r; h, w$	hole centre and radius; triangle height and base width (design units)	<code>params.csv</code> columns; <code>u_x, u_y, t</code> and <code>t_h, t_w</code> are the same shape as a point of the unit cube

A.2 Project jargon

Term	Meaning
run	A folder <code>runs/<run>/</code> with a <code>specs.json</code> . It counts as <i>trained</i> once <code>ModelParameters/latest.pth</code> exists, which says nothing about the epochs it holds.
preset	An untrained run: only <code>specs.json</code> and <code>metadata.json</code> (<code>"preset": true</code>). The four written by <code>experiments/make_plate_presets.py</code> are named <code>preset_plate2d_<variant>_d<d></code> (<code>r_only</code> , <code>xyr</code> , <code>tri_h</code> , <code>tri_hw</code>).
recipe	A <code>specs.json</code> minus <code>DataSource</code> , <code>TrainSplit</code> , <code>TestSplit</code> , <code>Description</code> . <code>prepare_run_like</code> copies a recipe and refuses to train if any other key differs.

family	A parametric shape family with a design space and an exact SDF: plate_hole (one circular hole) or plate_tri (four triangular holes); in datagen/ the triple <family>_params.py , <family>_sdf.py , make_<family>.py .
dataset, class, instance	data/SdfSamples/<dataset>/<class>/<instance>.npz + data/splits/<dataset>.json . Datasets are named <family>_2d_<variant> ; the class is always plate ; one instance (the trainer says <i>scene</i>) = one shape = one latent code.
geom dimension	Coordinates per sample, geom_dimension 2 or 3, taken from the dataset. Every decoder trained here is 2 (a <i>planar</i> decoder of a whole plate, shown in Explore 2-D, not tileable); every shipped one is 3.
shipped decoder	A pretrained model in DeepSDFStruct/DeepSDFStruct/trained_models/ , named by PretrainedModels: AnalyticRoundCross, RoundCross, ChiAndCross, Primitives, ...
dataset variants	r_only , xyr , xyr_n166 , n25 , n134 (holes); tri_h , tri_hw (triangles). What each draws and how many shapes: the table in Chapter 5. The <i>extremes</i> of the design box come first in the latent order, the <i>large holes</i> of xyr_n166 last.
design units, normalized frame	params.csv uses plate units, origin at the lower-left corner. Samples live in $[-1, 1]^d$: normalized = $1.8 \cdot (\text{design} - \text{plate centre})$, a 0.1 pad.
margin, ligament	The thinnest material allowed between a hole and an edge or another hole. Holes: 0.05 for training, 0.1 for the design space; triangles: 0.1.
8x256 , 4x64 , skip	Hidden layers $\times$ width (NetworkSpecs.dims). The long runs use 8x256 with the skip (latent_in : where $(\lambda, \mathbf{x})$ re-enter) at layer 4, the presets 4x64 with the skip at layer 2.
4h , unattended	experiments/train_plate*.py wait, train, analyse and log to outputs/logs/ ; the two 4h ones also stop at a deadline (--max-hours , default 4).
ep800	The 800-epoch $d = 3$ recipe of runs/plate_hole_2d_xyr_d3_ep800 (that run stopped at 680; the finished one is plate_hole_2d_xyr_d3_20260930_0124).
smoke test	A 2-epoch throwaway run (--epochs 2 --run smoke) before every real launch; also the environment check tests/test_deepsdfstruct_env.py .
code-vs-parameter	Comparing learned codes with params.csv : correlation per component and, for $d > 1$, the R^2 of an affine fit of each parameter on all components.
trained range, latent coverage	Per-component min/max of a run's codes (GUI sliders span it $\pm 10\%$; optimization bounds must come from it). Coverage: the widest gap between neighbouring codes as a share of the range; above 12% (viz.GAP_FRACTION) the GUI shades it.

Appendix B

On-disk formats

Checked against the files on disk on 2 October 2026. The layout of `data/` is the only interface between `datagen` and everything downstream (Chapter 5); a run folder is the library trainer’s layout plus a few files of this repo (Chapter 6). `data/`, `runs/` and `outputs/` are gitignored.

B.1 Datasets: `data/`

From each name in the split the trainer builds the path

`<DataSource>/SdfSamples/<dataset>/<class>/<instance>.npz`, so `SdfSamples` and `splits` are fixed names (the tree: Chapter 5). Writers: `write_instance`, `write_split`, `write_manifest` (`datagen/dataset.py:138` and below). Instance names carry the parameters with four decimals and `p` for the point (`hole_x0p1200_y0p1200_r0p0700`, `tri_h0p0500_w0p1000`). A name in the split without its file makes the trainer log a warning, then fail when it loads the samples into RAM.

File, key	Content
npz <code>pos</code> , <code>neg</code>	float32 arrays of shape <code>(n, dim + 1)</code> , one row per sample: the coordinates, then φ . <code>pos</code> holds $\varphi \geq 0$ (void, boundary included), <code>neg</code> $\varphi < 0$ (material). Coordinates in $[-1, 1]^d$; band samples stray slightly past it (up to 1.16 seen, the SDF maker’s <i>Validate</i> allows 1.2). Every dataset so far: 25 000 uniform + 25 000 band rows per shape. Legacy keys <code>pos.npy</code> , <code>neg.npy</code> are also read.
<code>splits/<dataset>.json</code>	<code>{"<dataset>": {"plate": [names]}}</code> , names in latent order.
<code>params.csv</code> , holes	<code>name</code> , <code>x_c</code> , <code>y_c</code> , <code>r</code> , <code>u_x</code> , <code>u_y</code> , <code>t</code> , <code>r_max</code> , <code>clearance</code>
<code>params.csv</code> , <code>tri_h</code>	<code>name</code> , <code>h</code> , <code>t</code> , <code>base</code> , <code>wall</code> , <code>arm</code>
<code>params.csv</code> , <code>tri_hw</code>	<code>name</code> , <code>h</code> , <code>w</code> , <code>t_h</code> , <code>t_w</code> , <code>w_max</code> , <code>wall</code> , <code>arm</code> , <code>side</code>

In `params.csv` all lengths are design units; join through `name`, not row order. `u_x`, `u_y`, `t` (`t`; `t_h`, `t_w`) are the same shape as a point of the unit cube the sampler draws from: the box an optimizer would see. `r_max` and `w_max` are the largest admissible radius or base there; `clearance`, `wall`, `arm`, `side` are ligament widths. The trainer ignores the file’s inside/outside ratio: it draws `SamplesPerScene / 2` rows from each array (`DeepSDFStruct/DeepSDFStruct/deep_sdf/data.py:169`).

B.1.1 `dataset.json`

The two oldest datasets, `n25` and `n134`, lack `parameters.varied`, `fixed_centre` and `parameter_sampling.family`: code that reads a manifest must tolerate missing keys.

Key	Content
<code>created_by</code> , <code>created</code> , <code>command</code>	Writer, ISO time, and the exact command line that rebuilds the dataset.
<code>geom_dimension</code>	2 or 3. The only key any consumer reads (<code>geom_dimension</code> in <code>structsept/app/datasets.py</code> ; without a manifest the npz row width decides).
<code>family</code> , <code>dataset</code> , <code>class</code> , <code>n_instances</code> , <code>columns</code> , <code>sign</code> , <code>order</code>	<code>plate_with_hole</code> or <code>plate_with_four_triangles</code> ; name; <code>plate</code> ; shape count; <code>["x", "y", "phi"]</code> ; the sign rule; “split order = latent code index = params.csv row”.
<code>parameters</code>	<code>names</code> , <code>varied</code> , <code>fixed_centre</code> (holes), <code>table</code> , <code>units</code> , <code>shown_to_network: false</code> .
<code>geometry</code>	Triangles only: <code>triangles</code> , <code>centres</code> , <code>shared_height</code> , <code>base</code> , <code>base_ratio</code> , <code>tip</code> .
<code>design_space</code>	Holes: <code>length</code> , <code>width</code> , <code>margins</code> (four), <code>r_min</code> . Triangles: <code>size</code> , <code>margin</code> , <code>h_min</code> , <code>h_max</code> (+ <code>w_min</code> , <code>w_cap</code> , <code>w_max</code>).
<code>parameter_sampling</code>	<code>family</code> , <code>n_requested</code> , <code>method</code> , <code>seed</code> , <code>t_power</code> , <code>include_extremes</code> ; <code>n166</code> adds <code>large_holes</code> .

`frame`, `sdf_sampling` `pad` 0.1, `scale` 1.8, `mapping` (information only: nothing converts back to design units for you); `n_uniform`, `n_band`, `hole_fraction`, `stds` (0.05, 0.025), `seeding` (shape *i* uses `default_rng([seed, i])`).

B.2 Runs: `runs/<run>/`

A run folder holds `specs.json`, `metadata.json`, the trainer's files and the scripts' analysis files, in this order below (the tree: Chapter 4); a preset holds only the first two. `latest.pth` and `Logs.pth` are rewritten every `LogFrequency` epochs; numbered snapshots are kept at the multiples of `SnapshotFrequency` and at the epochs in `AdditionalSnapshots`.

B.2.1 `specs.json`

Written by `write_specs` via `to_specs` (`structsept/app/training.py:28`, `structsept/app/hyperparams.py:1118`) or copied by `prepare_run_like`; read by `train_deep_sdf`. Examples from `plate2d_r_only_d1_8x256_4h`.

Key	Meaning (example)
<code>DataSource</code> , <code>TrainSplit</code>	Absolute paths of <code>data/</code> and of the split. <code>TestSplit</code> (a copy) and <code>ReconstructionSplit</code> ("") are not read.
<code>Description</code> , <code>NetworkArch</code>	Free text (run editor); <code>deep_sdf_decoder</code> .
<code>NetworkSpecs</code>	<code>dims</code> (width per hidden layer, <code>[256]*8</code>), <code>latent_in</code> (skip, <code>[4]</code>), <code>dropout</code> (layer indices), <code>dropout_prob</code> (0.2), <code>norm_layers</code> , <code>weight_norm</code> , <code>xyz_in_all</code> , <code>use_tanh</code> , <code>latent_dropout</code> , <code>geom_dimension</code> (2).
<code>CodeLength</code> , <code>CodeInitStdDev</code> , <code>CodeBound</code>	<i>d</i> (1); σ of the initial codes (0.1); largest code norm (1.0), applied by <code>nn.Embedding(max_norm= ...)</code> .
<code>CodeRegularization</code> , <code>CodeRegularizationLambda</code>	The bool is never read; the lambda (<code>1e-4</code>) alone sets the penalty.
<code>NumEpochs</code> , <code>LogFrequency</code> , <code>SnapshotFrequency</code> , <code>AdditionalSnapshots</code>	600, 10, 100, <code>[1]</code> .
<code>LearningRateSchedule</code>	Two entries, decoder then codes, e.g. <code>"Type": "Step", "Initial": 0.0005, "Interval": 150, "Factor": 0.5</code> ; types <code>Step</code> , <code>Warmup</code> , <code>Constant</code> .
<code>SamplesPerScene</code> , <code>ScenesPerBatch</code>	4096 points per shape and epoch (must be even); 4 shapes per batch.
<code>ClampingDistance</code> , <code>LossFunction</code>	0.1; <code>clampedL1</code> (or <code>leakyClampedL1</code> , <code>L1</code> , <code>MSE</code> , <code>huber</code>).
<code>DataLoaderThreads</code> , <code>seed</code>	0 (workers would re-import the app on Windows); 42, applied by <code>training.train</code> before the library builds the decoder.

B.2.2 `metadata.json`

Written by `write_metadata` (`structsept/app/training.py:185`), read by the runs table (`list_runs`). Keys: `timestamp` (when written; a run without the file is dated by its `specs.json` and shown with a ~), `dataset`, `split_path`, `data_source`, `latent_dim`, `geom_dimension`, `arch`, `dims`, `epochs` (asked for, not reached), `final_loss` (from `training_summary.json`, else `null`), `deepsdffstruct_version`. Presets and scripts add `preset`; at the end the scripts add `last_epoch`, `wall_seconds` and (4h only) `stopped_at_deadline`. The GUI writes the file after a successful training; a script writes it before training (so the run shows while it waits) and again at the end, so a killed run keeps `final_loss = null`.

B.2.3 Files the trainer writes

File	Content
<code>ModelParameters/*.pth</code>	<code>{"epoch", "model_state_dict"}</code> ; 1.86 MB for 8×256.

<code>LatentCodes/*.pth</code>	<code>{"epoch", "latent_codes": {"weight": (n_shapes, d)}};</code> row <i>i</i> = split entry <i>i</i> . The latent-field decoders <code>primitives_cl*</code> store zeros here and their codes under <code>latent_fields_state_dict</code> .
<code>OptimizerParameters/*.pth</code>	<code>{"epoch", "optimizer_state_dict"}</code> , Adam with two groups (decoder, codes); 3.7 MB for 8×256 . Only for resuming, which nothing here does.
<code>LatentCodes/ latent_code_data_map.json</code>	Which file each code was fitted to, written at the start: <code>data_source</code> , <code>sdf_samples_subdir</code> , <code>latent_codes (latent_index</code> , <code>relative_npz_filename</code> , absolute <code>npz_filename</code>).
<code>Logs.pth</code> , <code>Logs.png</code>	<code>epoch</code> ; <code>loss</code> , one float per step ($600 \text{ epochs} \times 10 \text{ steps} = 6000$); per epoch <code>learning_rate</code> (decoder, codes), <code>timing</code> , <code>latent_magnitude</code> , <code>param_magnitude</code> (per parameter). The Train tab polls it every 1.5 s. <code>Logs.png</code> , the loss plot, is redrawn at each snapshot.
<code>training_summary.json</code>	<code>loss</code> (mean batch loss of the last epoch), <code>num_epochs</code> , <code>timestamp</code> , <code>host_name</code> , <code>device</code> , <code>training_duration ("1:35:59")</code> , <code>data_dir</code> , <code>version</code> . Only after the last epoch: a stopped or killed run has none.

B.2.4 Analysis files

File	Content
<code>code_vs_<p>.csv</code> , <code>.png</code>	<code>unattended.code_vs_parameter</code> : columns <code>name</code> , <code><p></code> , <code>code_0</code> , <code>code_1</code> , ...; the older <code>plate2d_r_only_d1_8x256_4h</code> has <code>name</code> , <code>r</code> , <code>code</code> .
<code>code_vs_params.*</code>	<code>train_plate_xyr_4h.py</code> : CSV of parameters, codes and fits; JSON with <code>pearson</code> , <code>spearman</code> , <code>params_from_codes (r2, rmse, W)</code> , <code>codes_from_params</code> , <code>pc_variance_share</code> , <code>code_min</code> , <code>code_max</code> .
<code>latent_coverage.json</code>	<code>train_plate_xyr_n166.py</code> : <code>run</code> and <code>reference</code> , each a list of <code>components</code> with <code>min</code> , <code>max</code> , <code>gap_from</code> , <code>gap_to</code> , <code>gap_share</code> , <code>shaded</code> , <code>shapes_at_the_ends</code> . Not produced yet.

B.3 Logs: `outputs/logs/`

File	Content
<code><run>.log</code>	Appended; per launch a header with date, time, run and pid, then <code>HH:MM:SS</code> lines and the analysis numbers. Tee of stdout and stderr by <code>unattended.open_log (structsept/app/unattended.py:73)</code> .
<code><run>.pid</code>	The process id as text, never removed. Used by <code>--after</code> and by <code>taskkill</code> .
<code>explorer.log</code> , <code>sdf_maker.log</code>	Shortcut launches only: interpreter, <code>uv sync</code> output, tracebacks, the output of a GUI training. Moved to <code>.log.1</code> above 2 MB.

Appendix C

Command cheat sheet

Run everything from the repository root, through `uv run`, never with a bare `python`: several scripts write to paths relative to the working directory, and `uv` started inside `DeepSDFstruct/` builds a second environment. Type each command on one line.

C.1 Setup

```
git clone --recursive https://github.com/tomito6/struct_opt_nn.git code
cd code
git checkout gui-train-tab           # the work since 23/09; main is older
git submodule update --init --recursive # if cloned without --recursive
uv sync                             # .venv from pyproject.toml + uv.lock
uv run pytest tests/test_deepsdfstruct_env.py -v      # offline smoke test
.\install_shortcuts.bat              # or double-click: uv sync + Desktop\NN
```

C.2 Data

Each `dataset.json` stores its rebuild command under `"command"`. All share one stem (`<module>` is `make_plate_hole` or `make_plate_tri`), plus `--plot` for the preview PNGs; the extra flags of each dataset are in the table of Chapter 5. `--dry-run` prints the plan; `--help` lists the rest.

```
uv run python -m datagen.<module> --dim 2 --n-uniform 25000 --n-band 25000 --name <ds>
```

C.3 Training

```
uv run python experiments/make_plate_presets.py           # presets; after the datasets
uv run python experiments/train_plate_tri.py --epochs 2 --run smoke # smoke test
uv run python experiments/train_plate_tri.py --family hw --start-at 23:30
uv run python experiments/train_plate_r_only_4h.py --latent-dim 2 --start-at now
uv run python experiments/train_plate_xyr_4h.py --after outputs/logs/<run>.pid
uv run python experiments/train_plate_xyr_4h.py --check-only --run <run>
uv run python experiments/train_plate_xyr_n166.py --force # retrain from scratch
uv run python experiments/train_plate_xyr_n166.py --check-only
Get-Content outputs/logs/<run>.log -Wait -Tail 20        # follow a log
taskkill /PID <pid> /F                                   # pid in outputs/logs/<run>.pid
```

To keep a job alive after the terminal closes, start it hidden from PowerShell:

```
$cmd = "run python experiments/train_plate_tri.py --family hw"
Start-Process uv -ArgumentList $cmd -WindowStyle Hidden
```

All four scripts take `--start-at` (`now`, `HH:MM`, or date and time in quotes), `--after` (a pid or a `.pid` file), `--run`, `--epochs`, `--force`, `--data-root`, `--runs-dir`, `--log-dir`. The rest differs:

Flag	train_plate_r_only_4h	train_plate_xyr_4h	train_plate_tri	train_plate_xyr_n166
<code>--max-hours</code>	default 4	default 4	–	–
<code>--latent-dim</code>	default 1	default 3	–	–
<code>--decay-interval</code>	–	default epochs/4	–	–
<code>--family</code>	–	–	h (default), hw	–
<code>--like</code>	–	–	the family's preset	<code>..._xyr_d3_ep800</code>
<code>--dataset</code>	–	–	–	<code>..._xyr_n166</code>
<code>--check-only</code>	–	yes	yes	yes
<code>--epochs</code>	changes the recipe	changes the recipe	smoke test only	smoke test only

C.4 GUI

```
uv run python -m structsept.app.main           # Explore, Explore 2-D, Train
uv run python -m structsept.app.sdf_maker      # meshes → SdfSamples dataset
uv run python -m structsept.app.launcher explorer # as the shortcut: splash first
```

C.5 Experiments: geometry, stiffness, point clouds

```
uv run python experiments/plate_geometry.py
uv run python -m structsept.plate_with_hole
uv run python experiments/plate_with_hole_network.py
uv run python experiments/plate_with_hole_stiffness.py
uv run python experiments/pointcloud_to_lattice.py --noise 0.01 --validate
```

What each one shows and writes: Chapter 8; `--help` lists the flags.

C.6 Tests and docs

```
uv run pytest tests/ -v                # all project tests, offline
uv run python tests/test_deepsdfstruct_env.py # smoke test without pytest
uv run pytest tests/test_datagen.py -v    # includes a 2-epoch training
uv run pytest tests/test_app_hyperparams.py tests/test_app_runs.py -v
uv run pytest tests/test_app_launcher.py tests/test_app_unattended.py -v
uv run pytest tests/test_app_explore.py tests/test_app_explore2d.py -v # need a display
uv run pytest DeepSDFStruct/tests/test_structural_optimization.py -v # sim_out.vtk
uv run --project .. pytest tests/test_mesh_functions.py -v # inside DeepSDFStruct/
uv run --no-project --with typst python docs/handbook/build.py # handbook.pdf
uv run python docs/stiffness_theory/make_figures.py docs/stiffness_theory/figures
```

The smoke test's pretrained-decoder check is opt-in (PowerShell: `$env:DEEPSDF_TEST_PRETRAINED = "1"`). The `--project ..` line is for library tests that open `tests/data` relative to the working directory.

Appendix D

Gotchas checklist

The traps that have cost time in this project, each with the chapter that explains it.

D.1 Environment and repository

1. Run `uv` only from the repository root. Inside `DeepSDFStruct/` it builds a second environment from the submodule's `pyproject.toml`, without the root's `link-mode = "copy"` (on OneDrive hardlinks fail with os error 396); use `uv run --project ..` there. → Chapter 3
2. A clone has no data: `data/`, `runs/` and `outputs/` are gitignored. Rebuild every dataset from its `command`, then the presets. → Chapter 9
3. `specs.json` and `latent_code_data_map.json` store absolute Windows paths: a run folder's data paths break on another machine or after moving the repo, and presets must be regenerated per machine. → Chapter 6
4. `main` predates `datagen`, the GUI tabs and the launcher; the work is on `gui-train-tab` plus uncommitted files, so some commands of Appendix C are missing on a fresh clone. Clone with `--recursive`: the submodule needs its git history for its version number. → Chapter 3
5. The geometry and FEM scripts' outputs and the default `--input` of `pointcloud_to_lattice.py` are relative to the working directory (the training scripts and `datagen` resolve theirs from the repo root). → Chapter 3

D.2 Data

6. `--dim` has no default, and the hole family's default names (`plate_hole_2d`, `plate_hole_2d_r`) and sample counts (10 000 + 10 000) match no existing dataset: pass `--name` and both counts. → Chapter 5
7. Latent index = split order = `params.csv` row; extremes come first, large holes last. Join codes and parameters by `name`, never by assumption. → Chapter 5
8. The plate-with-hole SDF is exact only for admissible parameters (every ligament at least the margin); for a notch it is only a bound. → Chapter 5
9. The trainer rebalances every shape to half inside, half outside; the file's ratio never reaches the network. → Chapter 5

D.3 Training

10. Training always runs on CPU (`device="cpu"` in `training.train`). Two trainings at once share every core and both slow down: chain them with `--after`. → Chapter 6
11. Trainer traps, each a row of the trap table: `latest.pth`, the checkpoint everything loads, is written only at multiples of `LogFrequency` (leave the frequencies on `auto`); `SamplesPerScene` must be even; `clampedL1` clamps at its own 0.1 (use `L1` for a wider band); `GradientClipNorm` and the `CodeRegularization` bool are ignored; `drop_last` leaves out `n_shapes % ScenesPerBatch` shapes per epoch. → Chapter 6
12. *Trained* only means `latest.pth` exists, so a dead run loads like a finished one. Check the checkpoint epoch (`unattended.last_epoch`): the `n166` run stopped at epoch 80 of its planned 800. → Chapter 10
13. There is no resume. `--force` retrains from scratch, and the library's `continue_from` would restart the codes from random. → Chapter 6
14. Reusing a run name rewrites `specs.json` (the scripts also `metadata.json`) and nothing else: old snapshots, `Logs.pth` and, in the GUI, `metadata.json` stay until replaced. → Chapter 6

15. `--epochs` is a smoke-test override in `train_plate_tri.py` and `train_plate_xyr_n166.py` (it skips the recipe check) but changes the recipe in the `4h` scripts. Only the `4h` scripts have a deadline; stopping there loses up to `LogFrequency` epochs. → Chapter 6
16. Under `pythonw` (desktop shortcut) `sys.stderr` is `None` and `tqdm` crashes the trainer. Scripts set `TQDM_DISABLE` and `MPLBACKEND=Agg` before importing `torch`; the GUI forces the `Agg` backend. → Chapter 7
17. Auto run names have minute resolution, so two windows training the same dataset in the same minute collide, and a window does not see the other's new runs until *Refresh*. → Chapter 7

D.4 Latent space and analysis

18. The learned latent basis is arbitrary. Judge whether a parameter is encoded by an affine fit on all components (R^2), never component by component. → Chapter 10
19. The bounds `[0.15, 0.75]` belong to `AnalyticRoundCross`, which is no network: its latent *is* the strut radius. Trained codes sit near 0 (`r_only`, $d = 1$: -0.17 to $+0.22$), so take bounds from the trained range. → Chapter 4
20. The mean of the trained codes can lie in an untrained gap while the min/max box test passes. Look at the coverage strip and the nearest-code distance. → Chapter 7
21. Every decoder trained here is a planar whole-plate decoder: it cannot be tiled by `LatticeSDFStruct` or handed to MMA as it is. → Chapter 1

D.5 Geometry, FEM and optimization

22. Three coordinate spaces: decoder cube $[-1, 1]^3$, parametric $[0, 1]^3$, metres. The FFD moves only mesh vertices, so anything sized in metres goes through `ScaledSpaceSDF`; wrap the lattice in `CappedBorderSDF` before meshing for FEM. → Chapter 8
23. `create_3D_mesh(mesh_type="volume")` loses about 30 % of the volume to cavities; `fem.py` uses surface + tetgen instead, which breaks the gradient chain. Neither path is ready for an optimization loop. → Chapter 2
24. `LatticeSDFStruct` writes per-point codes into its microtile: never evaluate one lattice SDF twice at once, and re-wrap an `SDFfromDeepSDF` after using it in a lattice. → Chapter 4
25. `DeepSDFModel` is not an `nn.Module`: `sdf.to(device)` does not move the decoder, and `get_model` picks CUDA when it exists. Pass the device to `create_3D_mesh`. → Chapter 4
26. MMA bounds need shape `(numel, 2)`: use `np.tile([lo, hi], (n, 1))`, the template's broadcast works only for $d = 1$. MMA scales only the objective (normalize the constraint yourself) and has no stopping rule. → Chapter 4
27. `torch-fem` needs float64: call it inside `default_dtype(torch.float64)`. The unconstrained K is singular by design (six rigid-body modes). → Chapter 8